

2、并发三大特性

并发编程Bug的源头：可见性、原子性和有序性问题

可见性

当一个线程修改了共享变量的值，其他线程能够看到修改的值。Java 内存模型是通过在变量修改后将新值同步回主内存，在变量读取前从主内存刷新变量值这种依赖主内存作为传递媒介的方法来实现可见性的。

如何保证可见性

- 通过 volatile 关键字保证可见性。
- 通过 内存屏障保证可见性。
- 通过 synchronized 关键字保证可见性。
- 通过 Lock保证可见性。
- 通过 final 关键字保证可见性

有序性

即程序执行的顺序按照代码的先后顺序执行。JVM 存在指令重排，所以存在有序性问题。

如何保证有序性

- 通过 volatile 关键字保证可见性。
- 通过 内存屏障保证可见性。
- 通过 synchronized关键字保证有序性。
- 通过 Lock保证有序性。

原子性

一个或多个操作，要么全部执行且在执行过程中不被任何因素打断，要么全部不执行。在Java 中，对基本数据类型的变量的读取和赋值操作是原子性操作（64位处理器）。不采取任何的原子性保障措施 self-increment 操作并不是原子性的。

如何保证原子性

- 通过 synchronized 关键字保证原子性。
- 通过 Lock保证原子性。
- 通过 CAS保证原子性。

2.1 可见性问题

先看可见性问题：

```
public class VisibilityTest {  
  
    private boolean flag = true;  
  
    public void refresh() {  
        flag = false;  
        System.out.println(Thread.currentThread().getName() + "修改flag");  
    }  
}
```

```

public void load() {
    System.out.println(Thread.currentThread().getName() + "开始执行.....");
    int i = 0;
    while (flag) {
        i++;
        //TODO 业务逻辑
    }
    System.out.println(Thread.currentThread().getName() + "跳出循环: i=" + i);
}

public static void main(String[] args) throws InterruptedException {
    visibilityTest test = new VisibilityTest();

    // 线程threadA模拟数据加载场景
    Thread threadA = new Thread(() -> test.load(), "threadA");
    threadA.start();

    // 让threadA执行一会儿
    Thread.sleep(1000);
    // 线程threadB通过flag控制threadA的执行时间
    Thread threadB = new Thread(() -> test.refresh(), "threadB");
    threadB.start();

}

}

```

简写了，实际上编译器会认为这lambda表达式重写了run方法

获取Unsafe工具类:

```

public class UnsafeFactory {

    /**
     * 获取 Unsafe 对象
     * @return
     */
    public static Unsafe getUnsafe() {
        try {
            Field field = Unsafe.class.getDeclaredField("theUnsafe");
            field.setAccessible(true);
            return (Unsafe) field.get(null);
        } catch (Exception e) {
            e.printStackTrace();
        }
        return null;
    }

    /**
     * 获取字段的内存偏移量
     * @param unsafe
     * @param clazz
     * @param fieldName
     */
}

```

```

        * @return
        */
        public static long getFieldOffset(Unsafe unsafe, Class clazz, String
fieldName) {
            try {
                return unsafe.objectFieldOffset(clazz.getDeclaredField(fieldName));
            } catch (NoSuchFieldException e) {
                throw new Error(e);
            }
        }
    }
}

```

解决办法:

方法0: 使用volatile 修饰变量

方法一:

//System.out.println("1"); 这里可以停止循环, 原因是println 是synchronized的

方法二: 内存屏障

```
UnsafeFactory.getUnsafe().storeFence();
```

方法三: 释放时间片, 上下文切换 加载上下文

```
Thread.yield();
```

方法四: 底层也是使用的unsafe

```
LockSupport.unpark(Thread.currentThread());
```

方法五: 加入如下代码

```

try {
    Thread.sleep(1); //内存屏障
} catch (InterruptedException e) {
    e.printStackTrace();
}

```

方法六:

```
int i = 0; 改为Integer i = 0; //原因: Integer 内部的value是final修饰
```

以上解决办法的原理是什么?

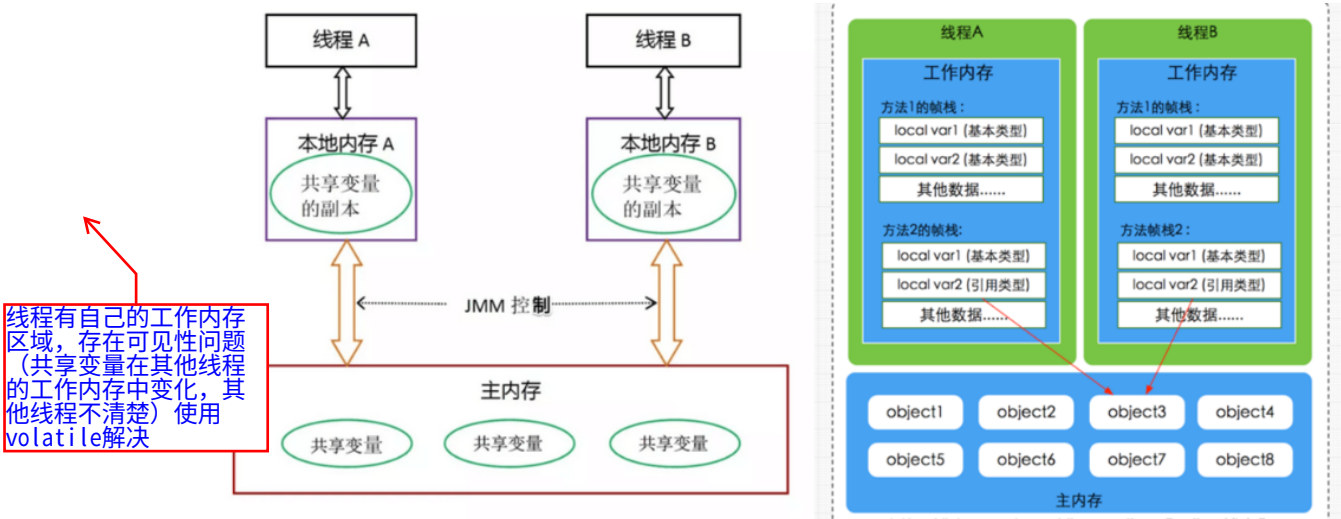
2.2 Java内存模型 (JMM)

Java内存模型(Java Memory Model简称JMM)是一种抽象的概念, 并不真实存在, 它描述的是一组规则或规范, 通过这组规范定义了程序中各个变量(包括实例字段, 静态字段和构成数组对象的元素)的访问方式。JVM运行程序的实体是线程, 而每个线程创建时JVM都会为其创建一个工作内存(有些地方称为栈空间), 用于存储线程私有的数据, 而Java内存模型中规定所有变量都存储在主内存, 主内存是共享内存区域, 所有线程都可以访问, 但线程对变量的操作(读取赋值等)必须在工作内存中进行, 首先要将变量从主内存拷贝的自己的工作内存空间, 然后对变量进行操作, 操作完成后再将变量写回主内存, 不能直接操作主内存中的变量, 工作内存中存储着主内存中的变量副本拷贝, 工作内存是每个线程的私有数据区域, 因此不同的线程间无法访问对方的工作内存, 线程间的通信(传值)必须通过主内存来完成。

JMM描述的是一组规则, 通过这组规则控制程序中各个变量在共享数据区域和私有数据区域的访问方式, JMM是围绕原子性, 有序性、可见性展开。

JMM定义

Java虚拟机规范中定义了Java内存模型（Java Memory Model, JMM），用于屏蔽掉各种硬件和操作系统的内存访问差异，以实现让Java程序在各种平台下都能达到一致的并发效果，JMM规范了Java虚拟机与计算机内存是如何协同工作的：**规定了一个线程如何和何时可以看到由其他线程修改过后的共享变量的值，以及在必须时如何同步的访问共享变量。**JMM描述的是一种抽象的概念，一组规则，通过这组规则控制程序中各个变量在共享数据区域和私有数据区域的访问方式，JMM是围绕原子性、有序性、可见性展开的。



2.3 volatile的内存语义

volatile的特性

- **可见性：对一个volatile变量的读，总是能看到（任意线程）对这个volatile变量最后的写入。**
- **原子性：对任意单个volatile变量的读/写具有原子性，但类似于volatile++这种复合操作不具有原子性（基于这点，我们会认为volatile不具备原子性）。volatile仅仅保证对单个volatile变量的读/写具有原子性，而锁的互斥执行的特性可以确保对整个临界区代码的执行具有原子性。**

64位的long型和double型变量，只要它是volatile变量，对该变量的读/写就具有原子性。

- **有序性：对volatile修饰的变量的读写操作前后加上各种特定的内存屏障来禁止指令重排序来保障有序性。**

在JSR-133之前的旧Java内存模型中，虽然不允许volatile变量之间重排序，但旧的Java内存模型允许volatile变量与普通变量重排序。为了提供一种比锁更轻量级的线程之间通信的机制，JSR-133专家组决定增强volatile的内存语义：严格限制编译器和处理器对volatile变量与普通变量的重排序，确保volatile的写-读和锁的释放-获取具有相同的内存语义。

volatile写-读的内存语义

- **当写一个volatile变量时，JMM会把该线程对应的本地内存中的共享变量值刷新到主内存。**
- **当读一个volatile变量时，JMM会把该线程对应的本地内存置为无效，线程接下来将从主内存中读取共享变量。**

总结：

volatile 是通过jvm层面 storeLoad内存屏障，对应到汇编就是lock前缀指令

2.6 有序性

```
public class ReOrderTest {

    private static int x = 0, y = 0;

    private static int a = 0, b = 0;

    public static void main(String[] args) throws InterruptedException{
        int i=0;
        while (true) {
            i++;
            x = 0;
            y = 0;
            a = 0;
            b = 0;

            /**
             * x,y:
             */
            Thread thread1 = new Thread(new Runnable() {
                @Override
                public void run() {
                    shortWait(20000);
                    a = 1;
                    x = b;
                }
            });
            Thread thread2 = new Thread(new Runnable() {
                @Override
                public void run() {
                    b = 1;
                    y = a;
                }
            });

            thread1.start();
            thread2.start();
            thread1.join();
            thread2.join();

            System.out.println("第" + i + "次 (" + x + "," + y + ")");

            if (x==0&&y==0){
                break;
            }
        }

        public static void shortWait(long interval){
            long start = System.nanoTime();
            long end;
```

```

do{
    end = System.nanoTime();
}while(start + interval >= end);
}
}

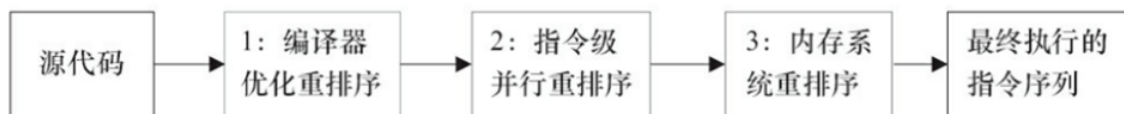
```

指令重排序

Java语言规范规定JVM线程内部维持顺序化语义。即只要程序的最终结果与它顺序化情况的结果相等，那么指令的执行顺序可以与代码顺序不一致，此过程叫指令的重排序。

指令重排序的意义：JVM能根据处理器特性（CPU多级缓存系统、多核处理器等）适当的对机器指令进行重排序，使机器指令能更符合CPU的执行特性，最大限度的发挥机器性能。

在编译器与CPU处理器中都能执行指令重排优化操作



3、List、Set、Map

3.1 ArrayList\LinkedList

ArrayList：底层数组存/取元素效率非常的高(get/set)，时间复杂度是O(1)，而查找（比如：indexOf, contain），插入和删除元素效率不太高，时间复杂度为O(n)。

array内存连续

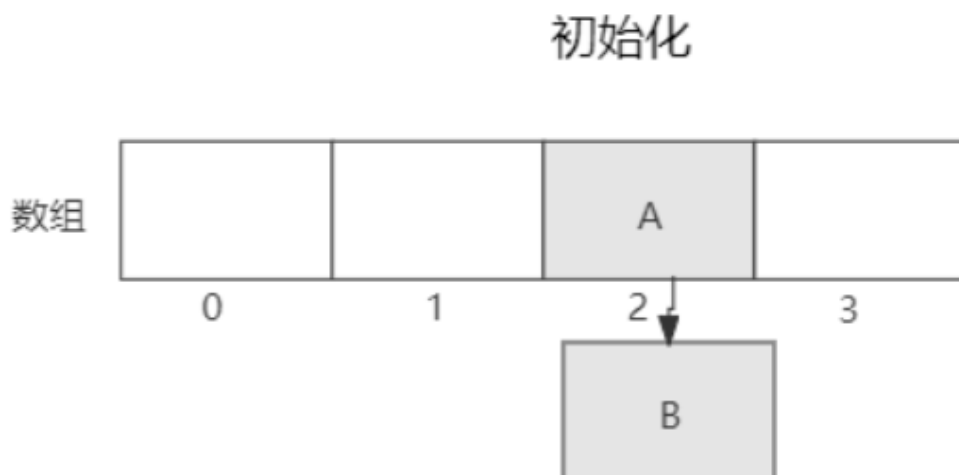
linkedList：底层采用链表来实现的。

链表内存连续不？

3.3 jdk7 HashMap cpu 100%问题

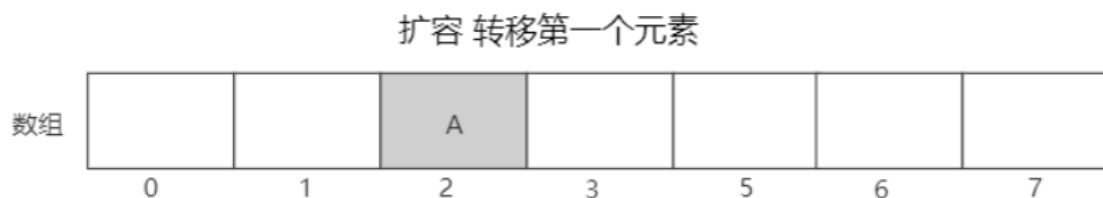
jdk7：HashMap采用的头插法，也就是产生hash冲突的时候，新插入的元素插在链表的头。

比如：

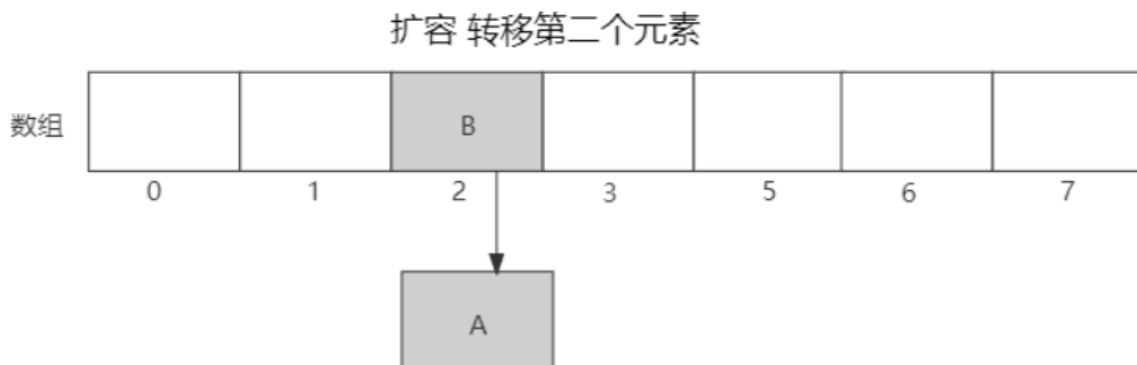


最开始的时候在2号位置发生了冲突 A 指向B

但是现在正好要扩容：

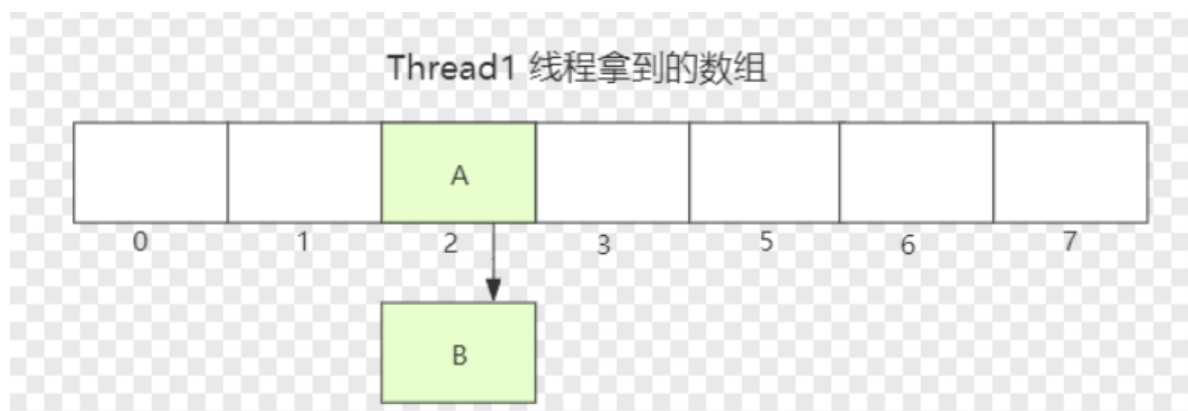


因为采用头插法，先转移第一个元素 A，这时A插到链表的头部，接下来转移B，头插法，B就插到头部，如下图：



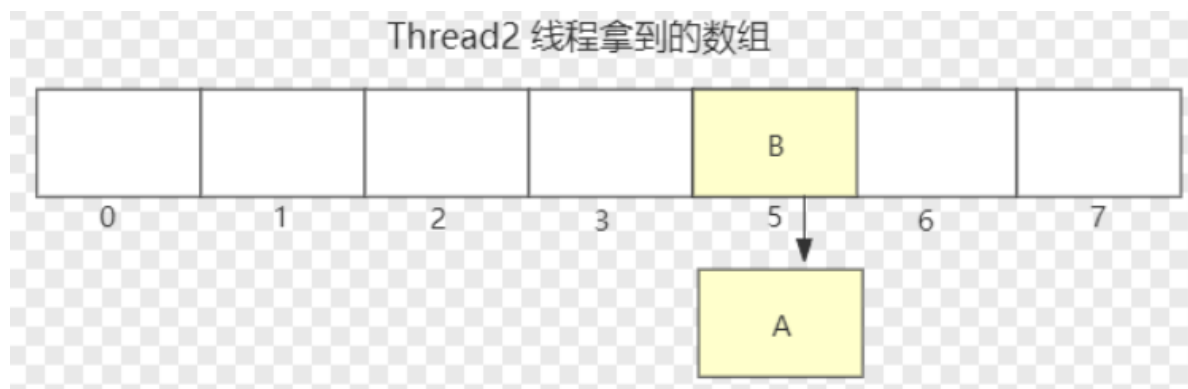
如果这时是两个线程做操作：

第一个线程：

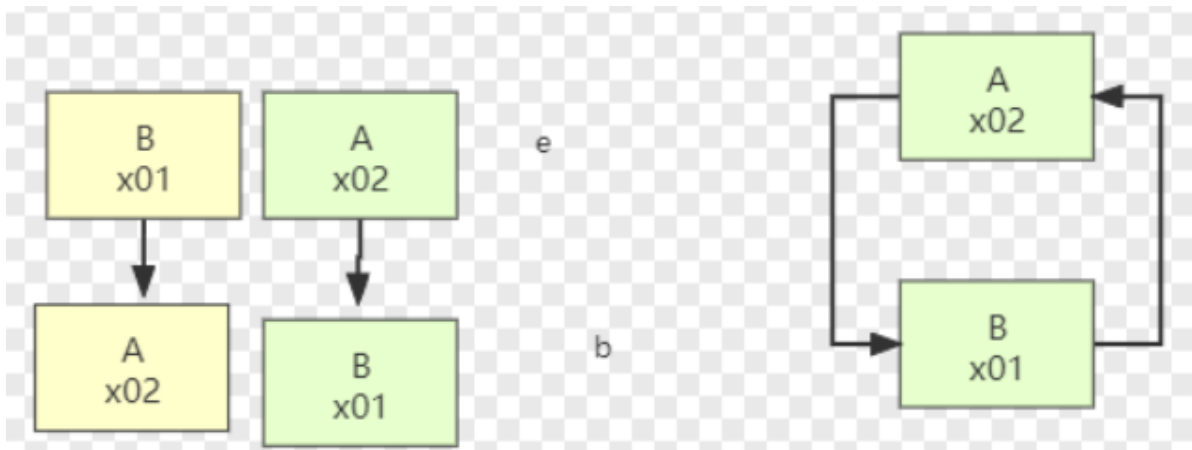


第二个线程：

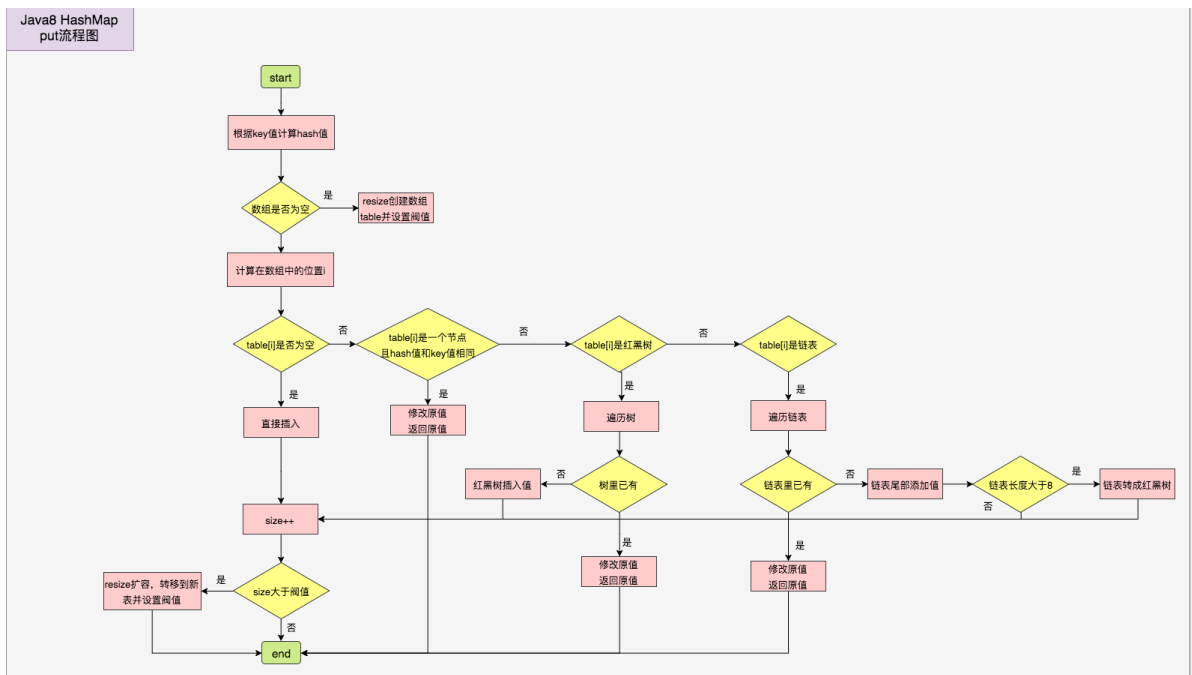
来操作的时候比如需要扩容，就会导致：



就会导致如下现象：



3.4 HashMap源码



3.5 ConcurrentHashMap

存储结构:

底层采用数组、链表、红黑树 内部大量采用CAS操作。并发控制使用synchronized 和 CAS 来操作来实现的。

3.6 HashMap面试题

1、JDK8中的HashMap与JDK7的HashMap有什么不一样？

- JDK8中新增了红黑树，JDK8是通过数组+链表+红黑树来实现的
- JDK7中链表的插入是用的头插法，而JDK8中则改为了尾插法
- JDK8中的因为使用了红黑树保证了插入和查询的效率，所以实际上JDK8中的Hash算法实现的复杂度降低了
- JDK8中数组扩容的条件也发了变化，只会判断是否当前元素个数是否查过了阈值，而不再判断当前put进来的元素对应的数组下标位置是否有值。
- JDK7中是先扩容再添加新元素，JDK8中是先添加新元素然后再扩容

2、HashMap中PUT方法的流程？

1. 通过key计算出一个hashcode

2. 通过hashCode与“与操作”计算出一个数组下标
3. 在把put进来的key,value封装为一个entry对象
4. 判断数组下标对应的位置，是不是空，如果是空则把entry直接存在该数组位置
5. 如果该下标对应的位置不为空，则需要把entry插入到链表中
6. 并且还需要判断该链表中是否存在相同的key，如果存在，则更新value
7. 如果是JDK7，则使用头插法
8. 如果是JDK8，则会遍历链表，并且在遍历链表的过程中，统计当前链表的元素个数，如果超过8个，则先把链表转变为红黑树，并且把元素插入到红黑树中

3、JDK8中链表转变为红黑树的条件？

1. 链表中的元素的个数为8个或超过8个
2. 同时，还要满足当前数组的长度大于或等于64才会把链表转变为红黑树。

为什么？因为链表转变为红黑树的目的是为了解决链表过长，导致查询和插入效率慢的问题，而如果要解决这个问题，也可以通过数组扩容，把链表缩短也可以解决这个问题。所以在数组长度还不太长的情况，可以先通过数组扩容来解决链表过长的问题。

4、HashMap扩容流程是怎样的？

1. HashMap的扩容指的就是数组的扩容，因为数组占用的是连续内存空间，所以数组的扩容其实只能新开一个新的数组，然后把老数组上的元素转移到新数组上来，这样才是数组的扩容
2. 在HashMap中也是一样，先新建一个2被数组大小的数组
3. 然后遍历老数组上的每一个位置，如果这个位置上是一个链表，就把这个链表上的元素转移到新数组上去
4. 在这个过程中就需要遍历链表，当然jdk7，和jdk8在这个实现时是有不一样的，jdk7就是简单的遍历链表上的每一个元素，然后按每个元素的hashCode结合新数组的长度重新计算得出一个下标，而重新得到的这个数组下标很可能和之前的数组下标是不一样的，这样子就达到了一种效果，就是扩容之后，某个链表会变短，这也就达到了扩容的目的，缩短链表长度，提高了查询效率
5. 而在jdk8中，因为涉及到红黑树，这个其实比较复杂，jdk8中其实还会用到一个双向链表来维护红黑树中的元素，所以jdk8中在转移某个位置上的元素时，会去判断如果这个位置是一个红黑树，那么会遍历该位置的双向链表，遍历双向链表统计哪些元素在扩容完之后还是原位置，哪些元素在扩容之后在新位置，这样遍历完双向链表后，就会得到两个子链表，一个放在原下标位置，一个放在新下标位置，如果原下标位置或新下标位置没有元素，则红黑树不用拆分，否则判断这两个子链表的长度，如果超过八，则转成红黑树放到对应的位置，否则把单向链表放到对应的位置。
6. 元素转移完了之后，在把新数组对象赋值给HashMap的table属性，老数组会被回收。

5、为什么HashMap的数组的大小是2的幂次方数？

JDK7的HashMap是数组+链表实现的

JDK8的HashMap是数组+链表+红黑树实现的

当某个key-value对需要存储到数组中时，需要先生成一个数组下标index，并且这个index不能越界。

在HashMap中，先得到key的hashCode，hashCode是一个数字，然后通过hashCode & (table.length - 1) 运算得到一个数组下标index，是通过与运算计算出来一个数组下标的，而不是通过取余，与运算相比于取余运算速度更快，但是也有一个前提条件，就是数组的长度得是一个2的幂次方数。

4、线程池

4.1 线程池与线程对比：

使用线程：



```

/**
 * 使用线程的方式去执行程序
 */
public class ThreadTest {

    public static void main(String[] args) throws InterruptedException {
        Long start = System.currentTimeMillis();
        final Random random = new Random();
        final List<Integer> list = new ArrayList<Integer>();
        for (int i = 0; i < 100000; i++) {
            Thread thread = new Thread() {
                @Override
                public void run() {
                    list.add(random.nextInt());
                }
            };
            thread.start();
            thread.join();
        }
        System.out.println("时间: " + (System.currentTimeMillis() - start));
        System.out.println("大小: " + list.size());
    }
}

```

线程池:

```

/**
 * 线程池执行
 */
public class ThreadPoolTest {

    public static void main(String[] args) throws InterruptedException {
        Long start = System.currentTimeMillis();
        final Random random = new Random();
        final List<Integer> list = new ArrayList<Integer>();
        ExecutorService executorService = Executors.newSingleThreadExecutor();
        for (int i = 0; i < 100000; i++) {
            executorService.execute(new Runnable() {
                @Override
                public void run() {
                    list.add(random.nextInt());
                }
            });
        }
        executorService.shutdown();
        executorService.awaitTermination(1, TimeUnit.DAYS);
        System.out.println("时间: " + (System.currentTimeMillis() - start));
        System.out.println("大小: " + list.size());
    }
}

```

4.2 线程池参数

corePoolSize



线程池中的核心线程数，当提交一个任务时，线程池创建一个新线程执行任务，直到当前线程数等于corePoolSize；如果当前线程数为corePoolSize，继续提交的任务被保存到阻塞队列中，等待被执行；如果执行了线程池的prestartAllCoreThreads()方法，线程池会提前创建并启动所有核心线程。

maximumPoolSize

线程池中允许的最大线程数。如果当前阻塞队列满了，且继续提交任务，则创建新的线程执行任务，前提是当前线程数小于maximumPoolSize；

keepAliveTime

线程池维护线程所允许的空闲时间。当线程池中的线程数量大于corePoolSize的时候，如果这时没有新的任务提交，核心线程外的线程不会立即销毁，而是会等待，直到等待的时间超过了keepAliveTime；

unit

keepAliveTime的单位；

workQueue

用来保存等待被执行的任务的阻塞队列，且任务必须实现Runnable接口，在JDK中提供了如下阻塞队列：

- 1、ArrayBlockingQueue：基于数组结构的有界阻塞队列，按FIFO排序任务；
- 2、LinkedBlockingQueue：基于链表结构的阻塞队列，按FIFO排序任务，吞吐量通常要高于ArrayBlockingQueue；
- 3、SynchronousQueue：一个不存储元素的阻塞队列，每个插入操作必须等到另一个线程调用移除操作，否则插入操作一直处于阻塞状态，吞吐量通常要高于LinkedBlockingQueue；
- 4、PriorityBlockingQueue：具有优先级的无界阻塞队列；

threadFactory

它是ThreadFactory类型的变量，用来创建新线程。默认使用Executors.defaultThreadFactory() 来创建线程。使用默认的ThreadFactory来创建线程时，会使新创建的线程具有相同的NORM_PRIORITY优先级并且是非守护线程，同时也设置了线程的名称。

handler

线程池的饱和策略，当阻塞队列满了，且没有空闲的工作线程，如果继续提交任务，必须采取一种策略处理该任务，线程池提供了4种策略：

- 1、AbortPolicy：直接抛出异常，默认策略；
- 2、CallerRunsPolicy：用调用者所在的线程来执行任务；
- 3、DiscardOldestPolicy：丢弃阻塞队列中靠最前的任务，并执行
- 4、DiscardPolicy：直接丢弃任务；

谁执行submit();【这是线程池中的提交方法】
这个代码，谁就是提交者

上面的4种策略都是ThreadPoolExecutor的内部类。

当然也可以根据应用场景实现RejectedExecutionHandler接口，自定义饱和策略，如记录日志或持久化存储不能处理的任务。

线程池监控

```
public long getTaskCount() //线程池已执行与未执行的任务总数
public long getCompletedTaskCount() //已完成的任务数
public int getPoolSize() //线程池当前的线程数
public int getActiveCount() //线程池中正在执行任务的线程数量
```

线程池的工作原理：



简单来说，在执行execute()方法时如果状态一直是RUNNING时，的执行过程如下：

1. 如果workerCount < corePoolSize，则创建并启动一个线程来执行新提交的任务；
2. 如果workerCount >= corePoolSize，且线程池内的阻塞队列未满，则将任务添加到该阻塞队列中；
3. 如果workerCount >= corePoolSize && workerCount < maximumPoolSize，且线程池内的阻塞队列已满，则创建并启动一个线程来执行新提交的任务；
4. 如果workerCount >= maximumPoolSize，并且线程池内的阻塞队列已满，则根据拒绝策略来处理该任务，默认的处理方式是直接抛异常。

ThreadPoolExecutor内部有实现4个拒绝策略：

- (1)、CallerRunsPolicy，由调用execute方法提交任务的线程来执行这个任务；
- (2)、AbortPolicy，抛出异常RejectedExecutionException拒绝提交任务；
- (3)、DiscardPolicy，直接抛弃任务，不做任何处理；
- (4)、DiscardOldestPolicy，去除任务队列中的第一个任务（最旧的），重新提交；

4.3 ScheduledThreadPoolExecutor：

schedule：延迟多长时间之后只执行一次；

scheduledAtFixedRate固定：延迟指定时间后执行一次，之后按照固定的时长周期执行；

scheduledWithFixedDelay非固定：延迟指定时间后执行一次，之后按照：上一次任务执行时长 + 周期的时长 的时间去周期执行

示例：

```
public class ScheduledThreadPoolExecutorExample {
    public static void main(String[] args) {
        ScheduledThreadPoolExecutor executor = (ScheduledThreadPoolExecutor)
        Executors.newScheduledThreadPool(5);
        Task task = new Task("任务");
        System.out.println("Created : " + task.getName());
        //      executor.schedule(task, 2, TimeUnit.SECONDS);
        //      executor.scheduleWithFixedDelay(task, 0, 2, TimeUnit.SECONDS); //任务+延
        迟
        executor.scheduleAtFixedRate(task, 0, 3, TimeUnit.SECONDS); //任务，延迟取最
        大值 稳定定时器
    }
}

class Task implements Runnable {
    private String name;
```

```

    public Task(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    AtomicInteger atomicInteger=new AtomicInteger();

    public void run() {
        atomicInteger.incrementAndGet();
        //        if(true){
        //
        //            throw new NullPointerException();
        //        }
        System.out.println("Executing : " + name + ", Current Seconds : " + new
Date().getSeconds());
        //        try {
        //            Thread.sleep(5000);
        //        } catch (InterruptedException e) {
        //            e.printStackTrace();
        //        }
    }
}

```

5、线程基础

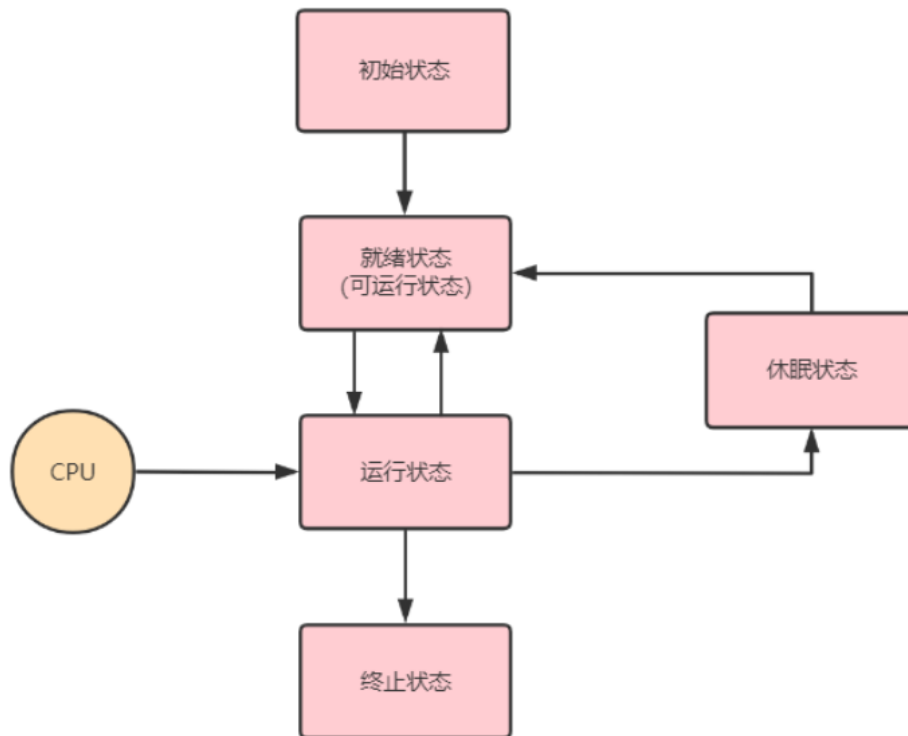
5.1 进程间通信的方式

1. **管道 (pipe) 及有名管道 (named pipe)**：管道可用于具有亲缘关系的父子进程间的通信，有名管道除了具有管道所具有的功能外，它还允许无亲缘关系进程间的通信。
2. **信号 (signal)**：信号是在软件层次上对中断机制的一种模拟，它是比较复杂的通信方式，用于通知进程有某事件发生，一个进程收到一个信号与处理器收到一个中断请求效果上可以说是一致的。
3. **消息队列 (message queue)**：消息队列是消息的链接表，它克服了上两种通信方式中信号量有限的缺点，具有写权限得进程可以按照一定得规则向消息队列中添加新信息；对消息队列有读权限得进程则可以从消息队列中读取信息。
4. **共享内存 (shared memory)**：可以说这是最有用的进程间通信方式。它使得多个进程可以访问同一块内存空间，不同进程可以及时看到对方进程中对共享内存中数据得更新。这种方式需要依靠某种同步操作，如互斥锁和信号量等。
5. **信号量 (semaphore)**：主要作为进程之间及同一种进程的不同线程之间得同步和互斥手段。
6. **套接字 (socket)**：这是一种更为一般得进程间通信机制，它可用于网络中不同机器之间的进程间通信，应用非常广泛。

5.2 线程生命周期

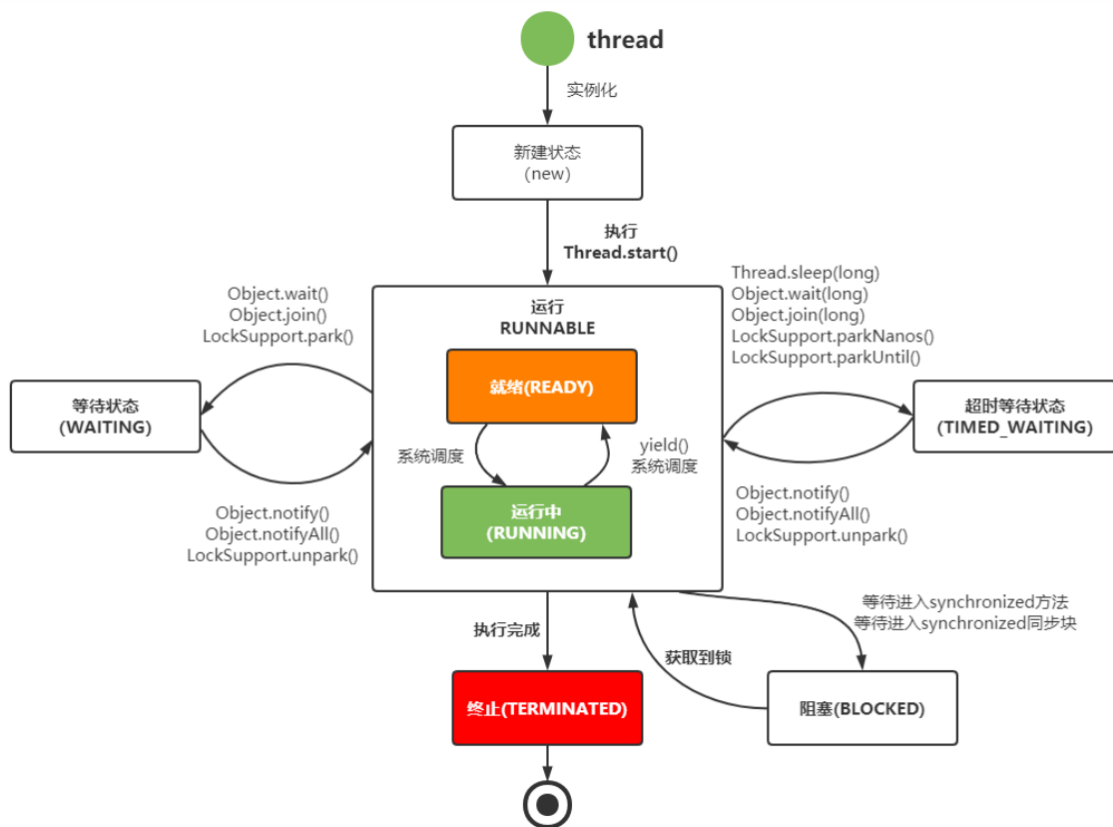
- 1、**操作系统层面**的线程生命周期基本上可以用下图这个“五态模型”来描述。这五态分别是：初始状态、可运行状态、运行状态、休眠状态和终止状态。

操作系统层面



1. 初始状态，指的是线程已经被创建，但是还不允许分配 CPU 执行。这个状态属于编程语言特有的，不过这里所谓的被创建，仅仅是在编程语言层面被创建，而在操作系统层面，真正的线程还没有创建。
2. 可运行状态，指的是线程可以分配 CPU 执行。在这种状态下，真正的操作系统线程已经被成功创建了，所以可以分配 CPU 执行。
3. 当有空闲的 CPU 时，操作系统会将其分配给一个处于可运行状态的线程，被分配到 CPU 的线程的状态就转换成了运行状态。
4. 运行状态的线程如果调用一个阻塞的 API（例如以阻塞方式读文件）或者等待某个事件（例如条件变量），那么线程的状态就会转换到休眠状态，同时释放 CPU 使用权，休眠状态的线程永远没有机会获得 CPU 使用权。当等待的事件出现了，线程就会从休眠状态转换到可运行状态。
5. 线程执行完或者出现异常就会进入终止状态，终止状态的线程不会切换到其他任何状态，进入终止状态也就意味着线程的生命周期结束了。

2、java层面



5.3 实现线程的方式

本质上Java中实现线程只有一种方式，都是通过new Thread()创建线程，调用Thread#start启动线程最终都会调用Thread#run方法

方式1：使用 Thread类或继承Thread类

```
// 创建线程对象
Thread t = new Thread() {
    public void run() {
        // 要执行的任务
    }
};
// 启动线程
```

方式2：实现 Runnable 接口配合Thread

把【线程】和【任务】（要执行的代码）分开

- Thread 代表线程
- Runnable 可运行的任务（线程要执行的代码）

```
Runnable runnable = new Runnable() {
    public void run(){
        // 要执行的任务
    }
};
// 创建线程对象
Thread t = new Thread( runnable );
// 启动线程
```

方式3: 使用有返回值的 Callable

```
@Slf4j
public class ThreadExecuteTest {

    public static void main(String[] args) throws ExecutionException,
        InterruptedException {

        //      FutureTask task = new FutureTask(new Callable() {
        //          @Override
        //          public Object call() throws Exception {
        //              log.debug("通过Callable方式执行任务");
        //              Thread.sleep(3000);
        //              return "返回任务结果";
        //          }
        //      });
        //
        //      new Thread(task).start();
        //      log.debug("结果: {}", task.get());

        ExecutorService executor = Executors.newFixedThreadPool(2);

        log.debug("张三下班回家做饭");
        Future<String> future = executor.submit(new Callable<String>() {
            @Override
            public String call() {
                //真正的任务在这里执行, 这里的返回值类型为String, 可以为任意类型
                log.debug("开始煮饭");
                try {
                    Thread.sleep(5000);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
                return "饭熟了";
            }
        });
        Future<String> future2 = executor.submit(new Callable<String>() {
            @Override
            public String call() {
                //真正的任务在这里执行, 这里的返回值类型为String, 可以为任意类型
                log.debug("开始做菜");
                try {
                    Thread.sleep(3000);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
                return "菜好了";
            }
        });
        log.debug("{} , {}, 张三开始吃饭", future.get(), future2.get());

        executor.shutdown();
    }
}
```



```
}  
}
```

方式4：使用 lambda

```
new Thread(() -> System.out.println(Thread.currentThread().getName())) .start();
```

5.6 Java线程的调度机制

线程调度是指系统为线程分配处理器使用权的过程，主要调度方式分两种，分别是协同式线程调度和抢占式线程调度

协同式线程调度

线程执行时间由线程本身来控制，线程把自己的工作执行完之后，要主动通知系统切换到另外一个线程上。最大好处是实现简单，且切换操作对线程自己是可知的，没啥线程同步问题。坏处是线程执行时间不可控制，如果一个线程有问题，可能一直阻塞在那里。

抢占式线程调度

每个线程将由系统来分配执行时间，线程的切换不由线程本身来决定（Java中，Thread.yield()可以让出执行时间，但无法获取执行时间）。线程执行时间系统可控，也不会有一个线程导致整个进程阻塞。

Java线程调度就是抢占式调度

希望系统能给某些线程多分配一些时间，给一些线程少分配一些时间，可以通过设置线程优先级来完成。

Java语言一共10个级别的线程优先级（Thread.MIN_PRIORITY至Thread.MAX_PRIORITY），在两线程同时处于ready状态时，优先级越高的线程越容易被系统选择执行。但优先级并不是很靠谱，因为Java线程是通过映射到系统的原生线程上实现的，所以线程调度最终还是取决于操作系统。

```
public class SellTicketDemo implements Runnable {  
    /**  
     * 车票  
     */  
    private int ticket;  
  
    public SellTicketDemo() {  
        this.ticket = 1000;  
    }  
  
    @Override  
    public void run() {  
        while (ticket > 0) {  
            synchronized (this) {  
                if (ticket > 0) {  
                    try {  
                        // 线程进入暂时的休眠  
                        Thread.sleep(20);  
                    } catch (InterruptedException e) {  
                        e.printStackTrace();  
                    }  
                }  
            }  
        }  
    }  
}
```

```

    }
    // 获取到当前正在执行的程序的名称，打印余票
    System.out.println(Thread.currentThread().getName()
        + ":正在执行操作，余票:" + ticket--);
    }
    }
    Thread.yield();
}
}

public static void main(String[] args) {
    SellTicketDemo demo = new SellTicketDemo();

    Thread thread1 = new Thread(demo, "thread1");
    Thread thread2 = new Thread(demo, "thread2");
    Thread thread3 = new Thread(demo, "thread3");
    Thread thread4 = new Thread(demo, "thread4");
    //priority优先级默认是5，最低1，最高10
    thread1.setPriority(Thread.MIN_PRIORITY);
    thread2.setPriority(Thread.MAX_PRIORITY);
    thread3.setPriority(Thread.MIN_PRIORITY);
    thread4.setPriority(Thread.MAX_PRIORITY);
    thread1.start();
    thread2.start();
    thread3.start();
    thread4.start();
}
}

```

5.7 Thread常用方法

sleep方法

- 调用 sleep 会让当前线程从 *Running* 进入 *TIMED_WAITING* 状态，**不会释放对象锁**
- 其它线程可以使用 interrupt 方法打断正在睡眠的线程，这时 sleep 方法会抛出 InterruptedException，并且会**清除中断标志**
- 睡眠结束后的线程未必会立刻得到执行
- sleep当传入参数为0时，和yield相同

yield方法

- yield会释放CPU资源，让当前线程从 *Running* 进入 *Runnable* 状态，让优先级更高（至少是相同）的线程获得执行机会，
- **不会释放对象锁；**
- 假设当前进程只有main线程，当调用yield之后，main线程会继续运行，因为没有比它优先级更高的线程；
- 具体的实现依赖于操作系统的任务调度器

join方法

等待调用join方法的线程结束之后，程序再继续执行，一般用于等待异步线程执行完结果之后才能继续运行的场景。

```
public class ThreadJoinDemo {
```

```

public static void main(String[] args) throws InterruptedException {

    Thread t = new Thread(new Runnable() {
        @Override
        public void run() {
            System.out.println("t begin");
            try {
                Thread.sleep(5000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
            System.out.println("t finished");
        }
    });
    long start = System.currentTimeMillis();
    t.start();
    //主线程等待线程t执行完成
    t.join();

    System.out.println("执行时间:" + (System.currentTimeMillis() - start));
    System.out.println("Main finished");
}
}

```

5.8 停止线程

1、stop方法[废弃]

stop()方法已经被jdk废弃，原因就是stop()方法太过于暴力，强行把执行到一半的线程终止。

stop会释放对象锁，可能会造成数据不一致。

```

public class ThreadStopDemo {

    private static Object lock = new Object();

    public static void main(String[] args) throws InterruptedException {

        Thread thread = new Thread(new Runnable() {
            @Override
            public void run() {
                synchronized (lock) {
                    System.out.println(Thread.currentThread().getName() + "获取
锁");

                    try {
                        Thread.sleep(60000);
                    } catch (InterruptedException e) {
                        e.printStackTrace();
                    }
                }
                System.out.println(Thread.currentThread().getName() + "执行完成");
            }
        });
        thread.start();
        Thread.sleep(2000);
    }
}

```

```

        // 停止thread，并释放锁
        thread.stop();

        new Thread(new Runnable() {
            @Override
            public void run() {
                System.out.println(Thread.currentThread().getName() + "等待获取
锁");

                synchronized (lock) {
                    System.out.println(Thread.currentThread().getName() + "获取
锁");

                }
            }
        }).start();
    }
}

```

2. 中断

Java没有提供一种安全、直接的方法来停止某个线程，而是提供了中断机制。**中断机制是一种协作机制，也就是说通过中断并不能直接终止另一个线程，而需要被中断的线程自己处理。**被中断的线程拥有完全的自主权，它既可以选择立即停止，也可以选择一段时间后停止，也可以选择压根不停止。

API的使用

- interrupt(): 将线程的中断标志位设置为true，不会停止线程
- isInterrupted(): 判断当前线程的中断标志位是否为true，不会清除中断标志位
- Thread.interrupted(): 判断当前线程的中断标志位是否为true，并清除中断标志位，重置为false

```

public class ThreadInterruptTest {

    static int i = 0;

    public static void main(String[] args) {
        System.out.println("begin");
        Thread t1 = new Thread(new Runnable() {
            @Override
            public void run() {
                while (true) {
                    i++;
                    System.out.println(i);
                    try {
                        Thread.sleep(10000); // 会清除中断标识
                    } catch (InterruptedException e) {
                        e.printStackTrace();
                    }

                    // Thread.interrupted() 清除中断标志位
                    // Thread.currentThread().isInterrupted() 不会清除中断标志位
                    if (Thread.interrupted()) {
                        System.out.println("=====");
                    }

                    if (i == 10) {
                        break;
                    }
                }
            }
        });
    }
}

```

```

        }

    }

});

t1.start();
//不会停止线程t1,只会设置一个中断标志位 flag=true
t1.interrupt();

}

}

```

处于休眠中的线程被中断，线程是可以感受到中断信号的，并且会抛出一个 `InterruptedException` 异常，同时清除中断信号，将中断标记位设置成 `false`。

`sleep`可以被中断 抛出中断异常：`sleep interrupted`，清除中断标志位

`wait`可以被中断 抛出中断异常：`InterruptedException`，清除中断标志位

```

public class ThreadStopDemo2 implements Runnable {

    @Override
    public void run() {
        int count = 0;
        while (!Thread.currentThread().isInterrupted() && count < 1000) {
            System.out.println("count = " + count++);

            try {
                Thread.sleep(1); //清除中断标识
            } catch (InterruptedException e) {
                e.printStackTrace();
                //重新设置线程中断状态为true
                Thread.currentThread().interrupt();
            }
        }
        System.out.println("线程停止: stop thread");
    }

    public static void main(String[] args) throws InterruptedException {
        Thread thread = new Thread(new ThreadStopDemo2());
        thread.start();
        Thread.sleep(5);
        thread.interrupt();
    }
}

```

利用中断机制优雅地停止线程

```

while (!Thread.currentThread().isInterrupted() && more work to do) {
    do more work
}

```

```

public class StopThread implements Runnable {

```

```

@Override
public void run() {
    int count = 0;
    while (!Thread.currentThread().isInterrupted() && count < 1000) {
        System.out.println("count = " + count++);
    }
    System.out.println("线程停止:  stop thread");
}

public static void main(String[] args) throws InterruptedException {
    Thread thread = new Thread(new StopThread());
    thread.start();
    Thread.sleep(5);
    thread.interrupt();
}
}

```

5.9 java线程间通信方式

1、volatile

volatile有两大特性，一是可见性，二是有序性，禁止指令重排序，其中可见性就是可以让线程之间进行通信。

```

public class VolatileDemo {

    private static volatile boolean flag = true;

    public static void main(String[] args) {

        new Thread(new Runnable() {
            @Override
            public void run() {
                while (true){
                    if (flag){
                        System.out.println("trun on");
                        flag = false;
                    }
                }
            }
        }).start();

        new Thread(new Runnable() {
            @Override
            public void run() {
                while (true){
                    if (!flag){
                        System.out.println("trun off");
                        flag = true;
                    }
                }
            }
        }).start();
    }
}

```

```
}
```

2、等待唤醒(等待通知)机制

等待唤醒机制可以基于wait和notify方法来实现，在一个线程内调用该线程锁对象的wait方法，线程将进入等待队列进行等待直到被唤醒。

```
public class waitDemo {

    private static Object lock = new Object();
    private static boolean flag = true;

    public static void main(String[] args) {
        new Thread(new Runnable() {
            @Override
            public void run() {
                synchronized (lock){
                    while (flag){
                        try {
                            System.out.println("wait start .....");
                            //等待
                            lock.wait();
                        } catch (InterruptedException e) {
                            e.printStackTrace();
                        }
                    }

                    System.out.println("wait end ..... ");
                }
            }
        }).start();

        new Thread(new Runnable() {
            @Override
            public void run() {
                if (flag){
                    synchronized (lock){
                        if (flag){

                            try {
                                Thread.sleep(2000);
                            } catch (InterruptedException e) {
                                e.printStackTrace();
                            }
                            //通知
                            lock.notifyAll();
                            System.out.println("notify .....");
                            flag = false;
                        }
                    }
                }
            }
        }).start();
    }
}
```

```
}  
}
```

LockSupport是JDK中用来实现线程阻塞和唤醒的工具，线程调用**park**则等待“许可证”，调用**unpark**则为指定线程提供“许可”。使用它可以在任何场合使线程阻塞，可以指定任何线程进行唤醒，并且不用担心阻塞和唤醒操作的顺序，但要注意连续多次唤醒的效果和一次唤醒是一样的。

```
public class LockSupportTest {  
  
    public static void main(String[] args) {  
        Thread parkThread = new Thread(new ParkThread());  
        parkThread.start();  
  
        System.out.println("唤醒parkThread");  
        //为指定线程parkThread提供“许可”  
        LockSupport.unpark(parkThread);  
    }  
  
    static class ParkThread implements Runnable{  
  
        @Override  
        public void run() {  
            System.out.println("ParkThread开始执行");  
            // 等待“许可证”  
            LockSupport.park();  
            System.out.println("ParkThread执行完成");  
        }  
    }  
}
```

3、管道输入输出流

管道输入/输出流和普通的文件输入/输出流或者网络输入/输出流不同之处在于，它主要用于线程之间的数据传输，而传输的媒介为内存。管道输入/输出流主要包括了如下4种具体实现：

PipedOutputStream、PipedInputStream、PipedReader和PipedWriter，前两种面向字节，而后两种面向字符。

```
public class PipedTest {  
    public static void main(String[] args) throws Exception {  
        PipedWriter out = new PipedWriter();  
        PipedReader in = new PipedReader();  
        // 将输出流和输入流进行连接，否则在使用时会抛出IOException  
        out.connect(in);  
  
        Thread printThread = new Thread(new Print(in), "PrintThread");  
        printThread.start();  
  
        int receive = 0;  
        try {  
            while ((receive = System.in.read()) != -1) {  
                out.write(receive);  
            }  
        } finally {  
            out.close();  
        }  
    }  
}
```



```

        out.close();
    }
}

static class Print implements Runnable {
    private PipedReader in;

    public Print(PipedReader in) {
        this.in = in;
    }

    @Override
    public void run() {
        int receive = 0;
        try {
            while ((receive = in.read()) != -1) {
                System.out.print((char) receive);
            }
        } catch (IOException ex) {
        }
    }
}
}

```

4、Thread.join

join可以理解成是线程合并，当在一个线程调用另一个线程的join方法时，当前线程阻塞等待被调用join方法的线程执行完毕才能继续执行，所以join的好处能够保证线程的执行顺序，但是如果调用线程的join方法其实已经失去了并行的意义，虽然存在多个线程，但是本质上还是串行的，最后**join的实现其实是基于等待通知机制的**。

6、CAS

6.1 什么是 CAS

无锁，能保证线程安全的机制，内部依赖volatile可见性

CAS (Compare And Swap, 比较并交换)，通常指的是这样一种原子操作：针对一个变量，首先比较它的内存值与某个期望值是否相同，如果相同，就给它赋一个新值。

CAS 的逻辑用伪代码描述如下：

```

if (value == expectedValue) {
    value = newValue;
}

```

以上伪代码描述了一个由比较和赋值两阶段组成的复合操作，**CAS 可以看作是它们合并后的整体——一个不可分割的原子操作，并且其原子性是直接在硬件层面得到保障的。**

CAS可以看做是乐观锁（对比数据库的悲观、乐观锁）的一种实现方式，Java原子类中的递增操作就通过CAS自旋实现的。

CAS是一种无锁算法，在不使用锁（没有线程被阻塞）的情况下实现多线程之间的变量同步。



6.2 CAS应用

在 Java 中，CAS 操作是由 Unsafe 类提供支持的，该类定义了三种针对不同类型变量的 CAS 操作，如图：

```
public final native boolean compareAndSwapObject(Object var1, long var2, Object var4, Object var5);

public final native boolean compareAndSwapInt(Object var1, long var2, int var4, int var5);

public final native boolean compareAndSwapLong(Object var1, long var2, long var4, long var6);
```

它们都是 native 方法，由 Java 虚拟机提供具体实现，这意味着不同的 Java 虚拟机对它们的实现可能会略有不同。

以 compareAndSwapInt 为例，Unsafe 的 compareAndSwapInt 方法接收 4 个参数，分别是：对象实例、内存偏移量、字段期望值、字段新值。该方法会针对指定对象实例中的相应偏移量的字段执行 CAS 操作

工具类：

```
public class UnsafeFactory {

    /**
     * 获取 Unsafe 对象
     * @return
     */
    public static Unsafe getUnsafe() {
        try {
            Field field = Unsafe.class.getDeclaredField("theUnsafe");
            field.setAccessible(true);
            return (Unsafe) field.get(null);
        } catch (Exception e) {
            e.printStackTrace();
        }
        return null;
    }

    /**
     * 获取字段的内存偏移量
     * @param unsafe
     * @param clazz
     * @param fieldName
     * @return
     */
    public static long getFieldOffset(Unsafe unsafe, Class clazz, String
fieldName) {
        try {
            return unsafe.objectFieldOffset(clazz.getDeclaredField(fieldName));
        } catch (NoSuchFieldException e) {
            throw new Error(e);
        }
    }

}
```

```

public class CASTest {

    public static void main(String[] args) {
        Entity entity = new Entity();

        Unsafe unsafe = UnsafeFactory.getUnsafe();

        //获取偏移量 这里指的是这个对象占用的多少字节【对象标记8个字节+4个字节的类型指针】
        long offset = UnsafeFactory.getFieldOffset(unsafe, Entity.class, "x");
        System.out.println(offset);
        boolean successful;

        // 4个参数分别是：对象实例、字段的内存偏移量、字段期望值、字段更新值
        successful = unsafe.compareAndSwapInt(entity, offset, 0, 3);
        System.out.println(successful + "\t" + entity.x);

        successful = unsafe.compareAndSwapInt(entity, offset, 3, 5);
        System.out.println(successful + "\t" + entity.x);

        successful = unsafe.compareAndSwapInt(entity, offset, 3, 8);
        System.out.println(successful + "\t" + entity.x);

    }

}

class Entity{
    int x;
}

```

6.4 CAS缺陷

CAS 虽然高效地解决了原子操作，但是还是存在一些缺陷的，主要表现在三个方面：

- 自旋 CAS 长时间地不成功，则会给 CPU 带来非常大的开销
- 只能保证一个共享变量原子操作
- ABA 问题

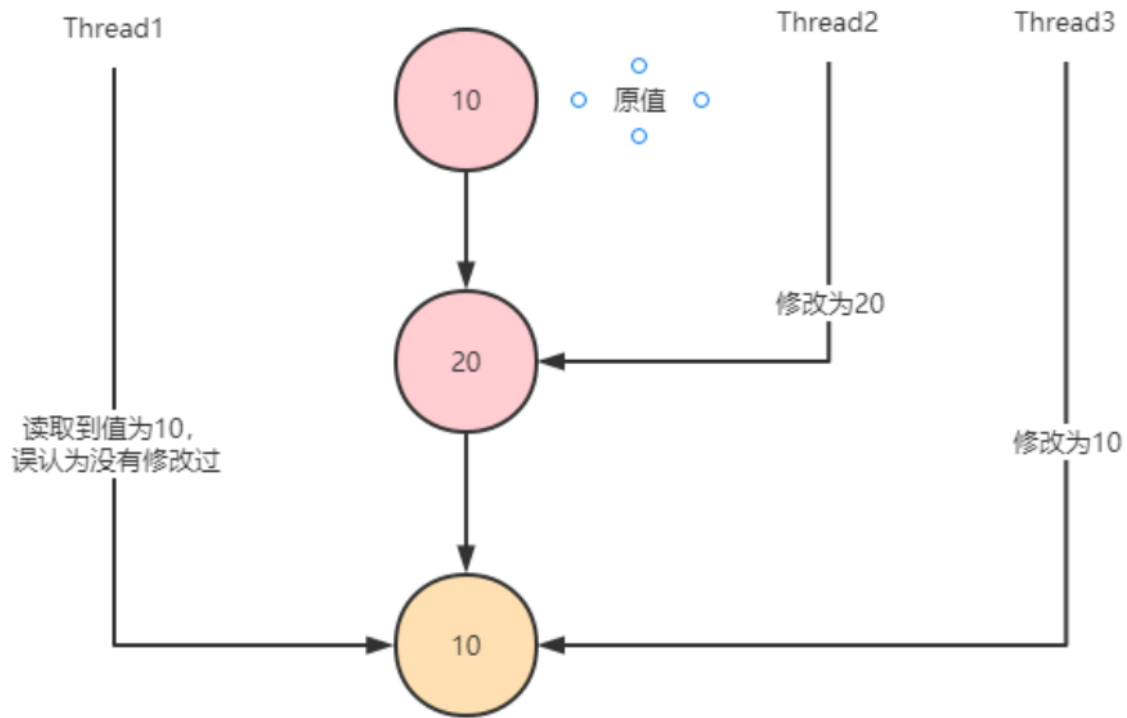
6.5 ABA问题及其解决方案

CAS算法实现一个重要前提需要取出内存中某时刻的数据，而在下时刻比较并替换，那么在这个时间差类会导致数据的变化。

什么是ABA问题

两个线程同时对一个变量修改，修改后的结果是和初始值一样，导致其他线程以为该变量没有修改

当有多个线程对一个原子类进行操作的时候，某个线程在短时间内将原子类的值A修改为B，又马上将其修改为A，此时其他线程不感知，还是会修改成功。



```
@Slf4j
public class ABATest {

    public static void main(String[] args) {
        AtomicInteger atomicInteger = new AtomicInteger(1);

        new Thread()->{
            int value = atomicInteger.get();
            log.debug("Thread1 read value: " + value);

            // 阻塞1s
            LockSupport.parkNanos(1000000000L);

            // Thread1通过CAS修改value值为3
            if (atomicInteger.compareAndSet(value, 3)) {
                log.debug("Thread1 update from " + value + " to 3");
            } else {
                log.debug("Thread1 update fail!");
            }
        }, "Thread1").start();

        new Thread()->{
            int value = atomicInteger.get();
            log.debug("Thread2 read value: " + value);
            // Thread2通过CAS修改value值为2
            if (atomicInteger.compareAndSet(value, 2)) {
                log.debug("Thread2 update from " + value + " to 2");

                // do something
                value = atomicInteger.get();
                log.debug("Thread2 read value: " + value);
                // Thread2通过CAS修改value值为1
                if (atomicInteger.compareAndSet(value, 1)) {
                    log.debug("Thread2 update from " + value + " to 1");
                }
            }
        }, "Thread2").start();

        new Thread()->{
            int value = atomicInteger.get();
            log.debug("Thread3 read value: " + value);
            // Thread3通过CAS修改value值为10
            if (atomicInteger.compareAndSet(value, 10)) {
                log.debug("Thread3 update from " + value + " to 10");
            }
        }, "Thread3").start();
    }
}
```

```

    }
    }
    }, "Thread2").start();
}
}

```

Thread1不清楚Thread2对value的操作，误以为value=1没有修改过

```

22:15:05.964 [Thread2] DEBUG com.gxa.concurrent.cas.ABATest - Thread2 read value: 1
22:15:05.964 [Thread1] DEBUG com.gxa.concurrent.cas.ABATest - Thread1 read value: 1
22:15:05.966 [Thread2] DEBUG com.gxa.concurrent.cas.ABATest - Thread2 update from 1 to 2
22:15:05.966 [Thread2] DEBUG com.gxa.concurrent.cas.ABATest - Thread2 read value: 2
22:15:05.966 [Thread2] DEBUG com.gxa.concurrent.cas.ABATest - Thread2 update from 2 to 1
22:15:06.966 [Thread1] DEBUG com.gxa.concurrent.cas.ABATest - Thread1 update from 1 to 3

```

ABA问题的解决方案

加入 时间戳 进行比对

数据库有个锁称为乐观锁，是一种基于数据版本实现数据同步的机制，每次修改一次数据，版本就会进行累加。

同样，Java也提供了相应的原子引用类AtomicStampedReference

```

public class AtomicStampedReference<V> {

    13 usages
    private static class Pair<T> {
        7 usages
        final T reference;
        7 usages
        final int stamp;
        1 usage
        private Pair(T reference, int stamp) {
            this.reference = reference;
            this.stamp = stamp;
        }
        4 usages
        static <T> Pair<T> of(T reference, int stamp) { return new Pair<T>(reference, stamp); }
    }

    8 usages
    private volatile Pair<V> pair;
}

```

reference即我们实际存储的变量，stamp是版本，每次修改可以通过+1保证版本唯一性。这样就可以保证每次修改后的版本也会往上递增。

```

@Slf4j
public class AtomicStampedReferenceTest {

    public static void main(String[] args) {
        // 定义AtomicStampedReference Pair.reference值为1, Pair.stamp为1
        AtomicStampedReference atomicStampedReference = new
        AtomicStampedReference(1,1);

        new Thread()->{
            int[] stampHolder = new int[1];
            int value = (int) atomicStampedReference.get(stampHolder);
            int stamp = stampHolder[0];
            log.debug("Thread1 read value: " + value + ", stamp: " + stamp);
        }
    }
}

```

性

```
// 阻塞1s
LockSupport.parkNanos(1000000000L);
// Thread1通过CAS修改value值为3    stamp是版本，每次修改可以通过+1保证版本唯一

if (atomicStampedReference.compareAndSet(value, 3, stamp, stamp+1)) {
    log.debug("Thread1 update from " + value + " to 3");
} else {
    log.debug("Thread1 update fail!");
}

}, "Thread1").start();

new Thread()->{
    int[] stampHolder = new int[1];
    int value = (int)atomicStampedReference.get(stampHolder);
    int stamp = stampHolder[0];
    log.debug("Thread2 read value: " + value + ", stamp: " + stamp);
    // Thread2通过CAS修改value值为2
    if (atomicStampedReference.compareAndSet(value, 2, stamp, stamp+1)) {
        log.debug("Thread2 update from " + value + " to 2");

        // do something

        value = (int) atomicStampedReference.get(stampHolder);
        stamp = stampHolder[0];
        log.debug("Thread2 read value: " + value + ", stamp: " + stamp);
        // Thread2通过CAS修改value值为1
        if (atomicStampedReference.compareAndSet(value,
1, stamp, stamp+1)) {
            log.debug("Thread2 update from " + value + " to 1");
        }
    }
    }, "Thread2").start();
}
}
```

6.6 Atomic原子操作类介绍

自己使用cas来实现锁

```
public class CASLock {

    //加锁标记
    private volatile int state;
    private static final Unsafe UNSAFE;
    private static final long OFFSET;

    static {
        try {
            UNSAFE = UnsafeFactory.getUnsafe();
            OFFSET = UnsafeFactory.getFieldOffset(
                UNSAFE, CASLock.class, "state");
        } catch (Exception e) {
```

```

        throw new Error(e);
    }
}

public boolean cas() {
    return UNSAFE.compareAndSwapInt(this, OFFSET, 0, 1);
}

public int getState() {
    return state;
}

public void setState(int state) {
    this.state = state;
}
}

```

在java.util.concurrent.atomic包里提供了一组原子操作类：

基本类型： AtomicInteger、AtomicLong、AtomicBoolean；

引用类型： AtomicReference、AtomicStampedRerence、AtomicMarkableReference；

数组类型： AtomicIntegerArray、AtomicLongArray、AtomicReferenceArray

对象属性原子修改器： AtomicIntegerFieldUpdater、AtomicLongFieldUpdater、AtomicReferenceFieldUpdater

原子类型累加器 (jdk1.8增加的类)： DoubleAccumulator、DoubleAdder、LongAccumulator、LongAdder、Striped64

6.6.1 原子更新基本类型

```

//以原子的方式将实例中的原值加1，返回的是自增前的旧值；
public final int getAndIncrement() {
    return unsafe.getAndAddInt(this, valueOffset, 1);
}

//getAndSet(int newValue)：将实例中的值更新为新值，并返回旧值；
public final boolean getAndSet(boolean newValue) {
    boolean prev;
    do {
        prev = get();
    } while (!compareAndSet(prev, newValue));
    return prev;
}

//incrementAndGet()：以原子的方式将实例中的原值进行加1操作，并返回最终相加后的结果；
public final int incrementAndGet() {
    return unsafe.getAndAddInt(this, valueOffset, 1) + 1;
}

```

```
//addAndGet(int delta) : 以原子方式将输入的数值与实例中原本的值相加，并返回最后的结果；
public final int addAndGet(int delta) {
    return unsafe.getAndAddInt(this, valueOffset, delta) + delta;
}
```

测试:

```
public class AtomicIntegerTest {

    static AtomicInteger sum = new AtomicInteger(0);

    public static void main(String[] args) {

        for (int i = 0; i < 10; i++) {
            Thread thread = new Thread(() -> {
                for (int j = 0; j < 10000; j++) {
                    // 原子自增 CAS
                    sum.incrementAndGet();
                    //count++;
                }
            });
            thread.start();
        }

        try {
            Thread.sleep(3000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println(sum.get());
    }

}
```

incrementAndGet()方法通过CAS自增实现，如果CAS失败，自旋直到成功+1。

```
public final int getAndAddInt(Object var1, long var2, int var4) {
    int var5;
    do {
        var5 = this.getIntVolatile(var1, var2);
    } while(!this.compareAndSwapInt(var1, var2, var5, var5: var5 + var4));

    return var5;
}
```

存在的问题：并发量很高，都要去修改该值,会因为自旋导致性能下降

6.6.2 原子更新数组类型

AtomicIntegerArray为例总结常用的方法

```
//addAndGet(int i, int delta): 以原子更新的方式将数组中索引为i的元素与输入值相加;
public final int addAndGet(int i, int delta) {
    return getAndAdd(i, delta) + delta;
}

//getAndIncrement(int i): 以原子更新的方式将数组中索引为i的元素自增加1;
public final int getAndIncrement(int i) {
    return getAndAdd(i, 1);
}

//compareAndSet(int i, int expect, int update): 将数组中索引为i的位置的元素进行更新
public final boolean compareAndSet(int i, int expect, int update) {
    return compareAndSetRaw(byteOffset(i), expect, update);
}
```

测试

```
public class AtomicIntegerArrayTest {

    static int[] value = new int[]{ 1, 2, 3, 4, 5 };
    static AtomicIntegerArray atomicIntegerArray = new
AtomicIntegerArray(value);

    public static void main(String[] args) throws InterruptedException {

        //设置索引0的元素为100
        atomicIntegerArray.set(0, 100);
        System.out.println(atomicIntegerArray.get(0));
        //以原子更新的方式将数组中索引为1的元素与输入值相加
        atomicIntegerArray.getAndAdd(1, 5);

        System.out.println(atomicIntegerArray);
    }
}
```

6.6.3 原子更新引用类型

AtomicReference作用是对普通对象的封装，它可以保证你在修改对象引用时的线程安全性。

```
public class AtomicReferenceTest {

    public static void main( String[] args ) {
        User user1 = new User("张三", 23);
        User user2 = new User("李四", 25);
        User user3 = new User("王五", 20);

        //初始化为 user1
        AtomicReference<User> atomicReference = new AtomicReference<>();
        atomicReference.set(user1);
    }
}
```

```

        //把 user2 赋给 atomicReference
        atomicReference.compareAndSet(user1, user2);
        System.out.println(atomicReference.get());

        //把 user3 赋给 atomicReference
        atomicReference.compareAndSet(user1, user3);
        System.out.println(atomicReference.get());

    }

}

@Data
@AllArgsConstructor
class User {
    private String name;
    private Integer age;
}

```

6.6.4 对象属性原子修改器

AtomicIntegerFieldUpdater可以线程安全地更新对象中的整型变量。

并不想整个对象原子修改，只需要里面的某个字段

```

public class AtomicIntegerFieldUpdaterTest {

    public static class Candidate {
        //字段必须是volatile类型
        volatile int score = 0;

        AtomicInteger score2 = new AtomicInteger();
    }

    public static final AtomicIntegerFieldUpdater<Candidate> scoreUpdater =
        AtomicIntegerFieldUpdater.newUpdater(Candidate.class, "score");

    public static AtomicInteger realScore = new AtomicInteger(0);

    public static void main(String[] args) throws InterruptedException {

        final Candidate candidate = new Candidate();

        Thread[] t = new Thread[10000];
        for (int i = 0; i < 10000; i++) {
            t[i] = new Thread(new Runnable() {
                @Override
                public void run() {
                    if (Math.random() > 0.4) {
                        candidate.score2.incrementAndGet();
                        scoreUpdater.incrementAndGet(candidate);
                    }
                }
            });
        }
    }
}

```

```

        realScore.incrementAndGet();
    }
}
});
t[i].start();
}
for (int i = 0; i < 10000; i++) {
    t[i].join();
}
System.out.println("AtomicIntegerFieldUpdater Score=" +
candidate.score);
System.out.println("AtomicInteger Score=" + candidate.score2.get());
System.out.println("realScore=" + realScore.get());

}
}

```

对于AtomicIntegerFieldUpdater 的使用稍微有一些限制和约束，约束如下：

- (1) 字段必须是volatile类型的，在线程之间共享变量时保证立即可见。eg: volatile int value = 3
- (2) 字段的描述类型（修饰符public/protected/default/private）与调用者与操作对象字段的关系一致。也就是说调用者能够直接操作对象字段，那么就可以反射进行原子操作。但是对于父类的字段，子类是不能直接操作的，尽管子类可以访问父类的字段。
- (3) 只能是实例变量，不能是类变量，也就是说不能加static关键字。
- (4) 只能是可修改变量，不能使final变量，因为final的语义就是不可修改。实际上final的语义和volatile是有冲突的，这两个关键字不能同时存在。
- (5) 对于AtomicIntegerFieldUpdater和AtomicLongFieldUpdater只能修改int/long类型的字段，不能修改其包装类型（Integer/Long）。如果要修改包装类型就需要使用AtomicReferenceFieldUpdater。

6.6.5 LongAdder/DoubleAdder详解

AtomicLong是利用了底层的CAS操作来提供并发性的：

方法调用了Unsafe类的getAndAddLong方法，该方法内部是个native方法，它的逻辑是采用自旋的方式不断更新目标值，直到更新成功。

在并发量较低的环境下，线程冲突的概率比较小，自旋的次数不会很多。但是，高并发环境下，N个线程同时进行自旋操作，会出现大量失败并不断自旋的情况，此时AtomicLong的自旋会成为瓶颈。

这就是LongAdder引入的初衷——解决高并发环境下AtomicInteger，AtomicLong的自旋瓶颈问题。

性能测试：

```

public class LongAdderTest {

    public static void main(String[] args) {
        testAtomicLongVSLongAdder(10, 10000);
        System.out.println("=====");
        testAtomicLongVSLongAdder(10, 200000);
        System.out.println("=====");
        testAtomicLongVSLongAdder(100, 200000);
    }
}

```

```

        static void testAtomicLongVSLongAdder(final int threadCount, final int
times) {
            try {
                long start = System.currentTimeMillis();
                testLongAdder(threadCount, times);
                long end = System.currentTimeMillis() - start;
                System.out.println("条件>>>>>线程数:" + threadCount + ", 单线程操作计数"
+ times);
                System.out.println("结果>>>>>LongAdder方式增加计数" + (threadCount *
times) + "次,共计耗时:" + end);

                long start2 = System.currentTimeMillis();
                testAtomicLong(threadCount, times);
                long end2 = System.currentTimeMillis() - start2;
                System.out.println("条件>>>>>线程数:" + threadCount + ", 单线程操作计数"
+ times);
                System.out.println("结果>>>>>AtomicLong方式增加计数" + (threadCount *
times) + "次,共计耗时:" + end2);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }

```

```

        static void testAtomicLong(final int threadCount, final int times) throws
InterruptedException {
            CountDownLatch countDownLatch = new CountDownLatch(threadCount);
            AtomicLong atomicLong = new AtomicLong();
            for (int i = 0; i < threadCount; i++) {
                new Thread(new Runnable() {
                    @Override
                    public void run() {
                        for (int j = 0; j < times; j++) {
                            atomicLong.incrementAndGet();
                        }
                        countDownLatch.countDown();
                    }
                }, "my-thread" + i).start();
            }
            countDownLatch.await();
        }
    }

```

```

        static void testLongAdder(final int threadCount, final int times) throws
InterruptedException {
            CountDownLatch countDownLatch = new CountDownLatch(threadCount);
            LongAdder longAdder = new LongAdder();
            for (int i = 0; i < threadCount; i++) {
                new Thread(new Runnable() {
                    @Override
                    public void run() {
                        for (int j = 0; j < times; j++) {
                            longAdder.add(1);
                        }
                        countDownLatch.countDown();
                    }
                }, "my-thread" + i).start();
            }
            countDownLatch.await();
        }
    }

```

```

        }, "my-thread" + i).start();
    }

    countDownLatch.await();
}
}

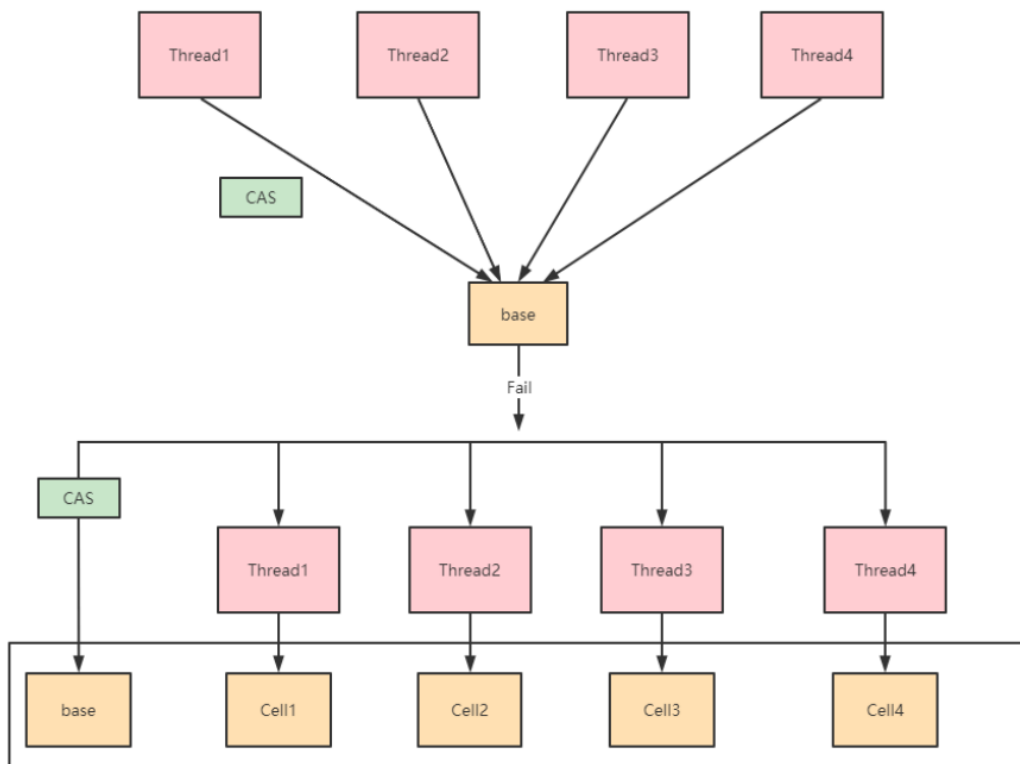
```

测试结果：线程数越多，并发操作数越大，LongAdder的优势越明显

LongAdder原理

设计思路

AtomicLong中有个内部变量value保存着实际的long值，所有的操作都是针对该变量进行。也就是说，高并发环境下，value变量其实是一个热点，也就是N个线程竞争一个热点。LongAdder的基本思路就是分散热点，将value值分散到一个数组中，不同线程会命中到数组的不同槽中，各个线程只对自己槽中的那个值进行CAS操作，这样热点就被分散了，冲突的概率就小很多。如果要获取真正的long值，只要将各个槽中的变量值累加返回。



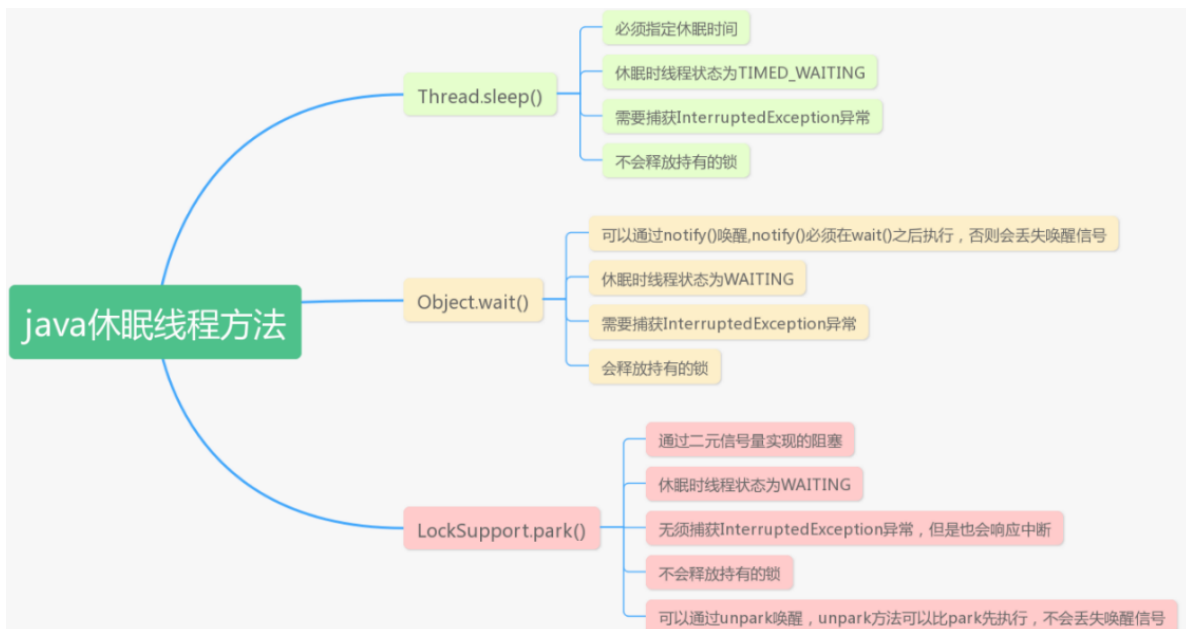
LongAdder的内部结构

LongAdder内部有一个base变量，一个Cell[]数组：

base变量：非竞态条件下，直接累加到该变量上

Cell[]数组：竞态条件下，累加各个线程自己的槽Cell[i]中

7、java线程休眠



7.1 使用wait/notify阻塞唤醒线程

使用wait, notify来实现等待唤醒功能至少有两个缺点:

- wait和notify都是Object中的方法,在调用这两个方法前必须先获得锁对象, 这限制了其使用场合:只能在同步代码块中。
- 当对象的等待队列中有多个线程时, notify只能随机选择一个线程唤醒, 无法唤醒指定的线程

使用LockSupport的话, 我们可以在任何场合使线程阻塞, 同时也可以指定要唤醒的线程, 相当的方便。

7.2 使用park/unpark阻塞唤醒线程

LockSupport是JDK中用来实现线程阻塞和唤醒的工具。使用它可以在任何场合使线程阻塞, 可以指定任何线程进行唤醒, 并且不用担心阻塞和唤醒操作的顺序, 但要注意连续多次唤醒的效果和一次唤醒是一样的。

JDK并发包下的锁和其他同步工具的底层实现中大量使用了LockSupport进行线程的阻塞和唤醒, 掌握它的用法和原理可以让我们更好的理解锁和其它同步工具的底层实现。

```
public class LockSupportTest {

    public static void main(String[] args) {
        Thread parkThread = new Thread(new ParkThread());
        parkThread.start();

        System.out.println("唤醒parkThread");
        LockSupport.unpark(parkThread);
    }

    static class ParkThread implements Runnable{

        @Override
        public void run() {
            System.out.println("ParkThread开始执行");
            LockSupport.park();
        }
    }
}
```

```
        System.out.println("ParkThread执行完成");
    }
}
}
```

8、synchronized

i++的JVM 字节码指令

```
getstatic i // 获取静态变量i的值
iconst_1 // 将int常量1压入操作数栈
iadd // 自增
putstatic i // 将修改后的值存入静态变量i
```

i--的JVM 字节码指令

```
getstatic i // 获取静态变量i的值
iconst_1 // 将int常量1压入操作数栈
isub // 自减
putstatic i // 将修改后的值存入静态变量i
```

8.1 Monitor（管程/监视器）

Monitor，直译为“监视器”，而操作系统领域一般翻译为“管程”。**管程是指管理共享变量以及对共享变量操作的过程，让它们支持并发。**在Java 1.5之前，Java语言提供的唯一并发语言就是管程，**Java 1.5之后提供的SDK并发包也是以管程为基础的。**除了Java之外，C/C++、C#等高级语言也都是支持管程的。synchronized关键字和wait()、notify()、notifyAll()这三个方法是Java中实现管程技术的组成部分。

java中是以管程来实现的。管程是遵循**MESA模型**的

MESA模型是说：大家在访问某个共享变量的时候，引入了等待队列，等待的线程放到等待队列，后面来进行唤醒

MESA模型：

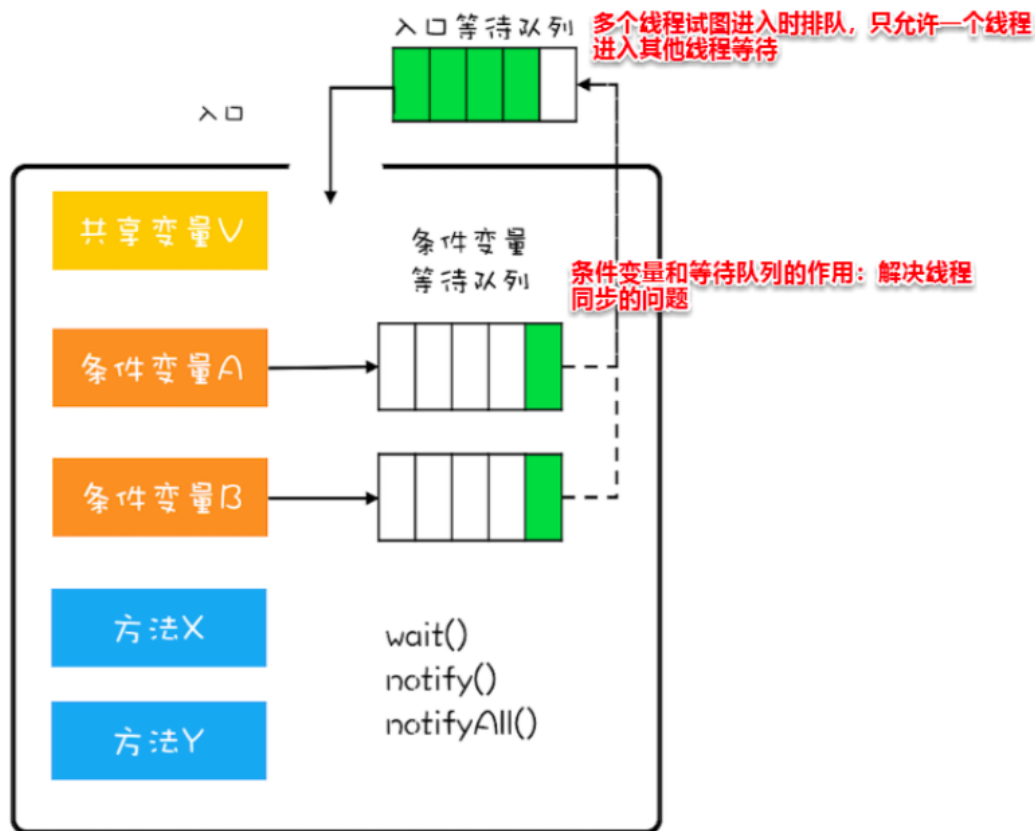
共享变量：多个线程要访问的变量

入口等待队列：只会有一个线程获取到锁，其他没获取到的放到等待队列

条件等待队列：当某个线程处于wait之后，会进入到条件等待队列，被唤醒之后进入到等待队列。

在jvm层面：使用synchronized对管程做了实现，利用的就是monitor，每个对象有一个ObjectMonitor，里面有cxq(入口等待队列)、waitset：等待队列 唤醒之后，放到entrylist

在java层面：java层面对管程的实现，就是AQS，那么他也应该具有入口等待队列，条件变量等待队列，共享变量等。具体情况AQS部分。



MESA 管程模型

管程中引入了条件变量的概念，而且每个条件变量都对应有一个等待队列。条件变量和等待队列的作用是解决线程之间的同步问题。

Monitor机制在Java中的实现

java.lang.Object 类定义了 wait(), notify(), notifyAll() 方法，这些方法的具体实现，依赖于 ObjectMonitor 实现，这是 JVM 内部基于 C++ 实现的一套机制。

ObjectMonitor其主要数据结构如下（hotspot源码ObjectMonitor.hpp）：

```
ObjectMonitor() {
    _header      = NULL; //对象头  markOop
    _count       = 0;
    _waiters     = 0,
    _recursions  = 0;    // 锁的重入次数
    _object      = NULL; //存储锁对象
    _owner       = NULL; // 标识拥有该monitor的线程（当前获取锁的线程）
    _waitSet     = NULL; // 等待线程（调用wait）组成的双向循环链表，_waitSet是第一个节点
    _waitSetLock = 0 ;
    _Responsible = NULL ;
    _succ        = NULL ;
    _cxq         = NULL ; //多线程竞争锁会先存到这个单向链表中 （FILO栈结构）
    FreeNext     = NULL ;
    _EntryList   = NULL ; //存放在进入或重新进入时被阻塞(blocked)的线程（也是存竞争锁失败的线程）
    _SpinFreq    = 0 ;
    _SpinClock   = 0 ;
}
```

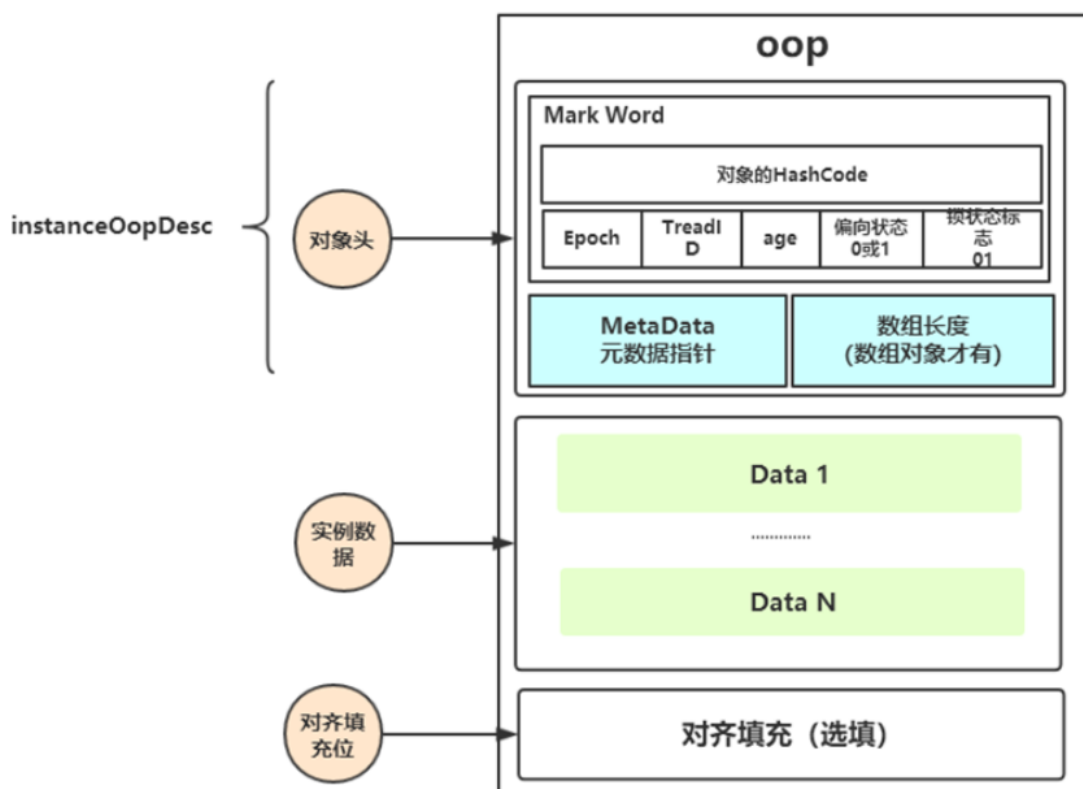


```
OwnerIsThread = 0 ;
_previous_owner_tid = 0;
```

8.2 对象的内存布局

Hotspot虚拟机中，对象在内存中存储的布局可以分为三块区域：**对象头 (Header)**、**实例数据 (Instance Data)** 和**对齐填充 (Padding)**。

- 对象头：比如 hash码，对象所属的年代，对象锁，锁状态标志，偏向锁（线程）ID，偏向时间，数组长度（数组对象才有）等。
 - hash：保存对象的哈希码。运行期间调用System.identityHashCode()来计算，延迟计算，并把结果赋值到这里。
 - age：保存对象的分代年龄。表示对象被GC的次数，当该次数到达阈值的时候，对象就会转移到老年代。
 - biased_lock：偏向锁标识位。由于无锁和偏向锁的锁标识都是 01，没办法区分，这里引入一位的偏向锁标识位。
 - lock：锁状态标识位。区分锁状态，比如11时表示对象待GC回收状态，只有最后2位锁标识(11)有效。
 - JavaThread*：保存持有偏向锁的线程ID。偏向模式的时候，当某个线程持有对象的时候，对象这里就会被置为该线程的ID。在后面的操作中，就无需再进行尝试获取锁的动作。这个线程ID并不是JVM分配的线程ID号，和Java Thread中的ID是两个概念。
 - epoch：保存偏向时间戳。偏向锁在CAS锁操作过程中，偏向性标识，表示对象更偏向哪个锁。
- 实例数据：存放类的属性数据信息，包括父类的属性信息；
- 对齐填充：由于虚拟机要求**对象起始地址必须是8字节的整数倍**。填充数据不是必须存在的，仅仅是为了字节对齐。



8.2.1 对象头详解

HotSpot虚拟机的对象头包括：

- Mark Word

用于存储对象自身的运行时数据，如哈希码（HashCode）、GC分代年龄、锁状态标志、线程持有的锁、偏向线程ID、偏向时间戳等，这部分数据的长度在32位和64位的虚拟机中分别为32bit和64bit，官方称它为“Mark Word”。

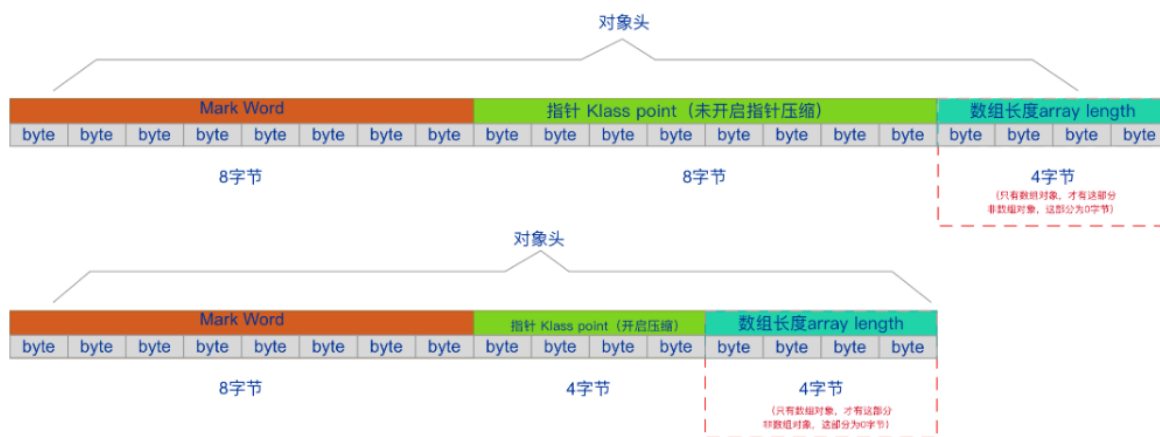
- Klass Pointer

对象头的另外一部分是klass类型指针，即对象指向它的类元数据的指针，虚拟机通过这个指针来确定这个对象是哪个类的实例。32位4字节，64位开启指针压缩或最大堆内存<32g时4字节，否则8字节。

jdk1.8默认开启指针压缩后为4字节，当在VM参数中关闭指针压缩（-XX:-UseCompressedOops）后，长度为8字节。

- 数组长度（只有数组对象有）

如果对象是一个数组，那在对象头中还必须有一块数据用于记录数组长度。4字节



使用JOL工具查看内存布局

查看普通java对象的内部布局工具JOL(JAVA OBJECT LAYOUT)，使用此工具可以查看new出来的一个java对象的内部布局,以及一个普通的java对象占用多少字节。

依赖：

```
<dependency>
  <groupId>org.openjdk.jol</groupId>
  <artifactId>jol-core</artifactId>
  <version>0.10</version>
</dependency>
```

```
public class TestObject {
    public static void main(String[] args) throws InterruptedException {
        Object obj = new Object();
        //查看对象内部信息
        System.out.println(ClassLayout.parseInstance(obj).toPrintable());
    }
}
```

```
E:\software\java\java8\bin\java.exe ...
```

OFFSET	SIZE	TYPE DESCRIPTION	VALUE
0	4	(object header)	01 00 00 00 (00000001 00000000 00000000 00000000) (1)
4	4	(object header)	00 00 00 00 (00000000 00000000 00000000 00000000) (0)
8	4	(object header)	28 0f 00 00 (00101000 00001111 00000000 00000000) (3880)
12	4	(class due to the next object alignment)	

Annotations:
 - **markword** points to the first 4 bytes of the first object header.
 - **kclass pointer** points to the 8th byte of the first object header.

- OFFSET: 偏移地址, 单位字节;
- SIZE: 占用的内存大小, 单位为字节;
- TYPE DESCRIPTION: 类型描述, 其中object header为对象头;
- VALUE: 对应内存中当前存储的值, 二进制32位;

这里需要注意, 使用的是操作系统的小端存储, 高位在左边, 低位在右边。如果我们要看, 就要反过来

(00000001 00000000 00000000 00000000) ==> 00000000 00000000 00000000 00000001

2. 关闭指针压缩后, 对象头为16字节: -XX:-UseCompressedOops

```
java.lang.Object object internals:
```

OFFSET	SIZE	TYPE DESCRIPTION	VALUE
0	4	(object header)	01 00 00 00 (00000001 00000000 00000000 00000000) (1)
4	4	(object header)	00 00 00 00 (00000000 00000000 00000000 00000000) (0)
8	4	(object header)	00 1c 47 1c (00000000 00011100 00000000 00000000) (3880)
12	4	(object header)	00 00 00 00 (00000000 00000000 00000000 00000000) (0)

Annotations:
 - **mark word** points to the first 4 bytes of the first object header.
 - **kclass point** points to the 8th byte of the first object header.

Instance size: 16 bytes
 Space losses: 0 bytes internal + 0 bytes external = 0 bytes total

8.3 JVM下的对象结构

32位JVM下的对象结构描述

锁状态	25bit		4bit	1bit	2bit
	23bit	2bit		是否偏向锁	锁标志位
无锁态	对象的hashCode		分代年龄	0	01
偏向锁	线程ID	Epoch	分代年龄	1	01
轻量级锁	指向栈中锁记录的指针				00
重量级锁	指向互斥锁（重量级锁）的指针				10
GC标记	空				11

64位JVM下的对象结构描述

锁状态	56bit		1bit	4bit	1bit	2bit
	unused: 25bit	对象的hashCode: 31bit	unused	分代年龄	是否偏向锁	锁标志位
无锁态	unused: 25bit	对象的hashCode: 31bit	unused	分代年龄	0	01
偏向锁	线程ID: 54bit	Epoch: 2bit	unused	分代年龄	1	01
轻量级锁	指向栈中锁记录的指针 (ptr_to_lock_record)					00
重量级锁	指向互斥锁（重量级锁）的指针 (ptr_to_heavyweight_monitor)					10
GC标记	空					11

- ptr_to_lock_record: 轻量级锁状态下, 指向栈中锁记录的指针。当锁获取是无竞争时, JVM使用原子操作而不是OS互斥, 这种技术称为轻量级锁定。在轻量级锁定的情况下, JVM通过CAS操作在对象的Mark Word中设置指向锁记录的指针。

- ptr_to_heavyweight_monitor：重量级锁状态下，指向对象监视器Monitor的指针。如果两个不同的线程同时在一个对象上竞争，则必须将轻量级锁定升级到Monitor以管理等待的线程。在重量级锁定的情况下，JVM在对象的ptr_to_heavyweight_monitor设置指向Monitor的指针

更直观的理解方式：

锁状态	存储内容	偏向锁标识位	锁标识位
无锁	哈希码、分代年龄	0	01
偏向锁	线程ID、时间戳、分代年龄	1	01
轻量级锁	指向栈中锁记录的指针ptr	无	00
重量级锁	指向monitor的指针ptr	无	10
GC标记	无	无	11

8.4 锁升级过程

8.4.1 无锁

```
public class Juc_PrintMarkWord {

    public static void main(String[] args) throws InterruptedException {
        T t = new T();
        //未出现任何获取锁的时候
        System.out.println(ClassLayout.parseInstance(t).toPrintable());
    }

    class T{
        int i = 0;
    }
}
```

OFFSET	SIZE	TYPE	DESCRIPTION	VALUE
0	4		(object header)	01 00 00 00 (00000001 00000000 00000000 00000000) (1)
4	4		(object header)	00 00 00 00 (00000000 00000000 00000000 00000000) (0)
8	4		(object header)	18 0a 06 00 (00011000 00001010 00000110 00000000) (395800)
12	4	int T.i		0

Instance size: 16 bytes

8.4.2 偏向锁

偏向锁是一种针对加锁操作的优化手段，经过研究发现，在大多数情况下，锁不仅不存在多线程竞争，而且总是由同一线程多次获得，因此为了消除数据在无竞争情况下锁重入（CAS操作）的开销而引入偏向锁。对于没有锁竞争的场合，偏向锁有很好的优化效果。

```
public class Juc_PrintMarkWord {

    public static void main(String[] args) throws InterruptedException {
        T t = new T();
        //未出现任何获取锁的时候
        System.out.println(ClassLayout.parseInstance(t).toPrintable());
        synchronized (t){
            // 获取一次锁之后
            System.out.println(ClassLayout.parseInstance(t).toPrintable());
        }
    }

    class T{
        int i = 0;
    }
}
```

```

    }

    }

}

class T{
    int i = 0;
}

```

com.gxa.concurrent.syn.T object internals:

OFFSET	SIZE	TYPE DESCRIPTION	VALUE
0	4	(object header)	38 f6 ef 9b (00111000) 11110110 11101111 10011011) (-1678772680)
4	4	(object header)	e9 00 00 00 (11101001 00000000 00000000 00000000) (233)
8	4	(object header)	18 0a 06 00 (00011000 00001010 00000110 00000000) (395800)
12	4	int T.i	0

发现还是无锁? 为什么?

原因是因为 偏向锁有锁延迟, 因为在jvm启动过程中也有很多地方用到了synchronized的, 所以不是一启动就开启了, 默认延迟4秒, 我们启动休眠一下, 再看效果

```

public class Juc_PrintMarkWord {

    public static void main(String[] args) throws InterruptedException {
        Thread.sleep(5000);
        T t = new T();
        //未出现任何获取锁的时候
        System.out.println(ClassLayout.parseInstance(t).toPrintable());
        synchronized (t){
            // 获取一次锁之后
            System.out.println(ClassLayout.parseInstance(t).toPrintable());
        }

    }

}

class T{
    int i = 0;
}

```

com.gxa.concurrent.syn.T object internals:

OFFSET	SIZE	TYPE DESCRIPTION	VALUE
0	4	(object header)	05 00 00 00 (00000101) 00000000 00000000 00000000) (5)
4	4	(object header)	00 00 00 00 (00000000 00000000 00000000 00000000) (0)
8	4	(object header)	18 0a 06 00 (00011000 00001010 00000110 00000000) (395800)
12	4	int T.i	0

这里已经是预偏向, 还没偏向线程

Instance size: 16 bytes

Space losses: 0 bytes internal + 0 bytes external = 0 bytes total

com.gxa.concurrent.syn.T object internals:

OFFSET	SIZE	TYPE DESCRIPTION	VALUE
0	4	(object header)	05 20 17 df (00000101) 00100000 00010111 11011111) (-552132603)
4	4	(object header)	d1 02 00 00 (11010001 00000010 00000000 00000000) (721)
8	4	(object header)	18 0a 06 00 (00011000 00001010 00000110 00000000) (395800)
12	4	int T.i	0

已经偏向某个线程了

Instance size: 16 bytes

Space losses: 0 bytes internal + 0 bytes external = 0 bytes total

最开始还没偏向某个线程的时候, 状态是偏向锁, 但是偏向的线程还没有, 这种称为匿名偏向

默认开启偏向锁，只是开启了延迟，延迟4秒

//关闭延迟开启偏向锁

-XX:BiasedLockingStartupDelay=0

//禁止偏向锁

-XX:-UseBiasedLocking

//开启偏向锁

-XX:+UseBiasedLocking

8.4.3 轻量级锁

倘若偏向锁失败，虚拟机并不会立即升级为重量级锁，它还会尝试使用一种称为轻量级锁的优化手段，此时Mark Word 的结构也变为轻量级锁的结构。**轻量级锁所适应的场景是线程交替执行同步块的情况**，如果存在同一时间多个线程访问同一把锁的情况，就会导致轻量级锁膨胀为重量级锁。

偏向锁到 轻量级锁 是 偏向锁获取失败就直接升级，没有自旋操作。

```
@Slf4j
public class T0_BasicLock {
    public static void main(String[] args) {
        try {
            Thread.sleep(5000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        Object o = new Object();
        log.info(ClassLayout.parseInstance(o).toPrintable());

        new Thread(()->{
            synchronized (o){
                log.info(ClassLayout.parseInstance(o).toPrintable());
            }
        }).start();

        try {
            Thread.sleep(2000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        log.info(ClassLayout.parseInstance(o).toPrintable()); //出现让
        new Thread(()->{
            synchronized (o){
                log.info(ClassLayout.parseInstance(o).toPrintable());
            }
        }).start();

    }
}
```

因为现在有两个线程要获取锁，没有出现激烈的竞争，因为我们是2秒后启动的另外一个线程。

8	4	(object header)	28 0f 00 00 (00101000 00001111 00000000 00000000) (3880)
12	4	(loss due to the next object alignment)	
Instance size: 16 bytes			
Space losses: 0 bytes internal + 4 bytes external = 4 bytes total			
09:56:38.558 [Thread-1] INFO com.gxa.concurrent.syn.T0_BasicLock - java.lang.Object object internals:			
OFFSET	SIZE	TYPE DESCRIPTION	VALUE
0	4	(object header)	70 ee bf 72 (01110000 11101110 10111111 01110010) (1925181040)
4	4	(object header)	74 00 00 00 (01110100 00000000 00000000 00000000) (116)
8	4	(object header)	28 0f 00 00 (00101000 00001111 00000000 00000000) (3880)
12	4	(loss due to the next object alignment)	

8.4.4 重量级锁

在升级到重量级锁之前，先创建一个monitor对象，进行自旋，自旋失败再阻塞[重量级锁里面的 park 和unpark]。

```
public class T0_heavyweightMonitor {

    public static void main(String[] args) throws InterruptedException {
        Thread.sleep(5000);
        Object a = new Object();

        Thread thread1 = new Thread(){
            @Override
            public void run() {
                synchronized (a){
                    System.out.println("thread1 locking");

                    System.out.println(ClassLayout.parseInstance(a).toPrintable());
                    try {
                        //让线程晚点儿死亡，造成锁的竞争
                        Thread.sleep(2000);
                    } catch (InterruptedException e) {
                        e.printStackTrace();
                    }
                }
            }
        };

        Thread thread2 = new Thread(){
            @Override
            public void run() {
                synchronized (a){
                    System.out.println("thread2 locking");

                    System.out.println(ClassLayout.parseInstance(a).toPrintable());
                    try {
                        Thread.sleep(2000);
                    } catch (InterruptedException e) {
                        e.printStackTrace();
                    }
                }
            }
        };

        thread1.start();
        thread2.start();
    }
}
```



```
}
```

OFFSET	SIZE	TYPE	DESCRIPTION	VALUE
0	4		(object header)	0a 1e 4e be (00001010 00011110 01001110 10111110) (-1102176758)
4	4		(object header)	46 02 00 00 (01000110 00000010 00000000 00000000) (582)
8	4		(object header)	28 0f 00 00 (00101000 00001111 00000000 00000000) (3880)
12	4		(loss due to the next object alignment)	

Instance size: 16 bytes

8.5 自旋优化

重量级锁竞争的时候，还可以使用自旋来进行优化，如果当前线程自旋成功（即这时候持锁线程已经退出了同步块，释放了锁），这时当前线程就可以避免阻塞。

- 自旋会占用 CPU 时间，单核 CPU 自旋就是浪费，多核 CPU 自旋才能发挥优势。
- 在 Java 6 之后自旋是自适应的，比如对象刚刚的一次自旋操作成功过，那么认为这次自旋成功的可能性会高，就多自旋几次；反之，就少自旋甚至不自旋，比较智能。
- Java 7 之后不能控制是否开启自旋功能

注意：**自旋的目的是为了减少线程挂起的次数，尽量避免直接挂起线程**（挂起操作涉及系统调用，存在用户态和内核态切换，这才是重量级锁最大的开销）

8.6 锁粗化

假设一系列的连续操作都会对同一个对象反复加锁及解锁，甚至加锁操作是出现在循环体中的，即使没有出现线程竞争，频繁地进行互斥同步操作也会导致不必要的性能损耗。如果JVM检测到有一连串零碎的操作都是对同一对象的加锁，将会扩大加锁同步的范围（即锁粗化）到整个操作序列的外部。

```
StringBuffer buffer = new StringBuffer();
/**
 * 锁粗化
 */
public void append(){
    buffer.append("aaa").append(" bbb").append(" ccc");
}
```

上述代码每次调用 `buffer.append` 方法都需要加锁和解锁，如果JVM检测到有一连串的对同一个对象加锁和解锁的操作，就会将其合并成一次范围更大的加锁和解锁操作，即在第一次`append`方法时进行加锁，最后一次`append`方法结束后进行解锁。

8.7 锁消除

锁消除即删除不必要的加锁操作。锁消除是Java虚拟机在JIT编译期间，通过对运行上下文的扫描，去除不可能存在共享资源竞争的锁，通过锁消除，可以节省毫无意义的请求锁时间。

```
public class LockEliminationTest {
    /**
     * 锁消除
     * -XX:+EliminateLocks 开启锁消除(jdk8默认开启)
     * -XX:-EliminateLocks 关闭锁消除
     * @param str1
     * @param str2
     */
}
```



```

    */
    public void append(String str1, String str2) {
        StringBuffer stringBuffer = new StringBuffer();
        stringBuffer.append(str1).append(str2);
    }

    public static void main(String[] args) throws InterruptedException {
        LockEliminationTest demo = new LockEliminationTest();
        long start = System.currentTimeMillis();
        for (int i = 0; i < 100000000; i++) {
            demo.append("aaa", "bbb");
        }
        long end = System.currentTimeMillis();
        System.out.println("执行时间: " + (end - start) + " ms");
    }
}

```

StringBuffer的append是个同步方法，但是append方法中的 StringBuffer 属于一个局部变量，不可能从该方法中逃逸出去，因此其实这过程是线程安全的，可以将锁消除。

测试结果： 关闭锁消除执行时间4688 ms 开启锁消除执行时间： 2601 ms

9、AQS

AbstractQueuedSynchronizer

← 用于构建锁和同步器的框架

AQS内部维护属性volatile int state

- state表示资源的可用状态

State三种访问方式：

- getState()
- setState()
- compareAndSetState()

AQS定义两种资源共享方式

- Exclusive-独占，只有一个线程能执行，如ReentrantLock
- Share-共享，多个线程可以同时执行，如Semaphore/CountDownLatch

AQS定义两种队列

- 同步等待队列： 主要用于维护获取锁失败时入队的线程
- 条件等待队列： 调用await()的时候会释放锁，然后线程会加入到条件队列，调用signal()唤醒的时候会吧条件队列中的线程节点移动到同步队列中，等待再次获得锁

AQS 定义了5个队列中节点状态：

1. 值为0，初始化状态，表示当前节点在sync队列中，等待着获取锁。
2. CANCELLED，值为1，表示当前的线程被取消；
3. SIGNAL，值为-1，表示当前节点的后继节点包含的线程需要运行，也就是unpark；
4. CONDITION，值为-2，表示当前节点在等待condition，也就是在condition队列中；
5. PROPAGATE，值为-3，表示当前场景下后续的acquireShared能够得以执行；

不同的自定义同步器竞争共享资源的方式也不同。自定义同步器在实现时只需要实现共享资源state的获取与释放方式即可，至于具体线程等待队列的维护（如获取资源失败入队/唤醒出队等），AQS已经在顶层实现好了。自定义同步器实现时主要实现以下几种方法：

- **isHeldExclusively()**：该线程是否正在独占资源。只有用到condition才需要去实现它。
- **tryAcquire(int)**：独占方式。尝试获取资源，成功则返回true，失败则返回false。
- **tryRelease(int)**：独占方式。尝试释放资源，成功则返回true，失败则返回false。
- **tryAcquireShared(int)**：共享方式。尝试获取资源。负数表示失败；0表示成功，但没有剩余可用资源；正数表示成功，且有剩余资源。
- **tryReleaseShared(int)**：共享方式。尝试释放资源，如果释放后允许唤醒后续等待结点返回true，否则返回false。

9.3 使用AQS自己实现锁

```
public class MyLock extends AbstractQueuedSynchronizer{

    //获取锁
    @Override
    protected boolean tryAcquire(int unused) {
        //cas 加锁 state初始为0，设置为1表示当前线程加锁
        if (compareAndSetState(0, 1)) {
            //设置锁的线程拥有者为当前线程
            setExclusiveOwnerThread(Thread.currentThread());
            //返回加锁成功
            return true;
        }
        //返回加锁失败
        return false;
    }

    //释放锁
    @Override
    protected boolean tryRelease(int unused) {
        //释放锁
        //将当前锁的线程拥有者 设置为null
        setExclusiveOwnerThread(null);
        //将状态恢复到0，表示没人获取锁，这里没有必要cas，因为只有获取到锁的才能释放锁，不存在竞争
        setState(0);
        //返回释放锁成功
        return true;
    }
}
```

```

    public void lock() {
        acquire(1);
    }

    public boolean tryLock() {
        return tryAcquire(1);
    }

    public void unlock() {
        release(1);
    }

    public boolean isLocked() {
        return isHeldExclusively();
    }

}

```

使用自己的锁来测试：

```

public class MyLockDemo {

    private static int sum = 0;
    private static MyLock lock = new MyLock();

    public static void main(String[] args) throws InterruptedException {

        for (int i = 0; i < 3; i++) {
            Thread thread = new Thread()->{
                //加锁
                lock.lock();
                try {
                    // 临界区代码
                    // TODO 业务逻辑：读写操作不能保证线程安全
                    for (int j = 0; j < 10000; j++) {
                        sum++;
                    }
                } finally {
                    // 解锁
                    lock.unlock();
                }
            };
            thread.start();

            Thread.sleep(2000);
            System.out.println(sum);
        }

    }
}

```

9.4 ReentrantLock使用

ReentrantLock是一种基于AQS框架的应用实现，是JDK中的一种线程并发访问的同步手段，它的功能类似于synchronized是一种互斥锁，可以保证线程安全。

相对于 synchronized， ReentrantLock具备如下特点：

- 可中断  中断的是等待该锁的线程
- 可以设置超时时间
- 可以设置为公平锁
- 支持多个条件变量
- 与 synchronized 一样，都支持可重入  同一把锁，能开一切用这把锁锁上的东西

顺便总结了几点synchronized和ReentrantLock的区别：

- synchronized是JVM层次的锁实现，ReentrantLock是JDK层面的锁实现；
- synchronized的锁状态是无法在代码中直接判断的，但是ReentrantLock可以通过ReentrantLock#isLocked判断；
- synchronized是非公平锁，ReentrantLock是可以是公平也可以是非公平的；
- synchronized是不可以被中断的，而ReentrantLock#lockInterruptibly方法是可以被中断的；
- 在发生异常时synchronized会自动释放锁，而ReentrantLock需要开发者在finally块中显示释放锁；
- ReentrantLock获取锁的形式有多种：如立即返回是否成功的tryLock(),以及等待指定时长的获取，更加灵活；
- synchronized在特定的情况下对于已经在等待的线程是后来的线程先获得锁（回顾一下synchronized的唤醒策略），而ReentrantLock对于已经在等待的线程是先来的线程先获得锁

9.4.1 ReentrantLock的基本使用

```
public class ReentrantLockDemo {

    private static int sum = 0;
    private static Lock lock = new ReentrantLock();

    public static void main(String[] args) throws InterruptedException {

        for (int i = 0; i < 3; i++) {
            Thread thread = new Thread()->{
                //加锁
                lock.lock();
                try {
                    // 临界区代码
                    // TODO 业务逻辑：读写操作不能保证线程安全
                    for (int j = 0; j < 10000; j++) {
                        sum++;
                    }
                } finally {
                    // 解锁
                    lock.unlock();
                }
            };
            thread.start();
        }

        Thread.sleep(2000);
    }
}
```

```
        System.out.println(sum);
    }
}
```

9.4.2 ReentrantLock可重入

```
@Slf4j
public class ReentrantLockDemo2 {

    public static ReentrantLock lock = new ReentrantLock();

    public static void main(String[] args) {
        method1();
    }

    public static void method1() {
        lock.lock();
        try {
            log.debug("execute method1");
            method2();
        } finally {
            lock.unlock();
        }
    }

    public static void method2() {
        lock.lock();
        try {
            log.debug("execute method2");
            method3();
        } finally {
            lock.unlock();
        }
    }

    public static void method3() {
        lock.lock();
        try {
            log.debug("execute method3");
        } finally {
            lock.unlock();
        }
    }
}
```

9.4.3 ReentrantLock可中断

```
@Slf4j
public class ReentrantLockDemo3 {

    public static void main(String[] args) throws InterruptedException {
        ReentrantLock lock = new ReentrantLock();

        Thread t1 = new Thread(() -> {
```

```

        log.debug("t1启动...");

        try {
            lock.lockInterruptibly();
            try {
                log.debug("t1获得了锁");
            } finally {
                lock.unlock();
            }
        } catch (InterruptedException e) {
            e.printStackTrace();
            log.debug("t1等锁的过程中被中断");
        }

    }, "t1");

    lock.lock();
    try {
        log.debug("main线程获得了锁");
        t1.start();
        //先让线程t1执行
        Thread.sleep(1000);

        t1.interrupt();
        log.debug("线程t1执行中断");
    } finally {
        lock.unlock();
    }

}

}

```

9.4.4 ReentrantLock锁超时

立即失败

```

@Slf4j
public class ReentrantLockDemo4 {

    public static void main(String[] args) throws InterruptedException {
        ReentrantLock lock = new ReentrantLock();

        Thread t1 = new Thread(() -> {

            log.debug("t1启动...");
            // 注意： 即使是设置的公平锁，此方法也会立即返回获取锁成功或失败，公平策略不生效
            if (!lock.tryLock()) {
                log.debug("t1获取锁失败，立即返回false");
                return;
            }

        })
    }
}

```

```

        try {
            log.debug("t1获得了锁");
        } finally {
            lock.unlock();
        }
    }, "t1");

    lock.lock();
    try {
        log.debug("main线程获得了锁");
        t1.start();
        //先让线程t1执行
        Thread.sleep(7000);
    } finally {
        lock.unlock();
    }
}
}

```

超时失败

```

@Slf4j
public class ReentrantLockDemo4 {

    public static void main(String[] args) throws InterruptedException {
        ReentrantLock lock = new ReentrantLock();

        Thread t1 = new Thread(() -> {

            log.debug("t1启动...");

            //超时
            try {
                if (!lock.tryLock(5, TimeUnit.SECONDS)) {
                    log.debug("等待 5s 后获取锁失败，返回");
                    return;
                }
            } catch (InterruptedException e) {
                e.printStackTrace();
                return;
            }

            try {
                log.debug("t1获得了锁");
            } finally {
                lock.unlock();
            }
        });
    }
}

```

```

    }, "t1");

    lock.lock();
    try {
        log.debug("main线程获得了锁");
        t1.start();
        //先让线程t1执行
        Thread.sleep(7000);
    } finally {
        lock.unlock();
    }
}

}

```

9.4.5 ReentrantLock公平锁与非公平锁

ReentrantLock 默认是不公平的，因为不公平效率高

```

@Slf4j
public class ReentrantLockDemo5 {

    public static void main(String[] args) throws InterruptedException {
        ReentrantLock lock = new ReentrantLock(); //非公平锁

        for (int i = 0; i < 500; i++) {
            new Thread(() -> {
                lock.lock();
                try {
                    try {
                        Thread.sleep(10);
                    } catch (InterruptedException e) {
                        e.printStackTrace();
                    }
                    log.debug(Thread.currentThread().getName() + " running...");
                } finally {
                    lock.unlock();
                }
            }, "t" + i).start();
        }
        // 1s 之后去争抢锁
        Thread.sleep(1000);

        for (int i = 0; i < 500; i++) {
            new Thread(() -> {
                lock.lock();
                try {
                    log.debug(Thread.currentThread().getName() + " running...");
                } finally {
                    lock.unlock();
                }
            }, "t" + i).start();
        }
    }
}

```



```

    }
    }, "强行插入" + i).start();

}

}

}

```

公平锁:

```

@Slf4j
public class ReentrantLockDemo5 {

    public static void main(String[] args) throws InterruptedException {
        ReentrantLock lock = new ReentrantLock(true); //公平锁

        for (int i = 0; i < 500; i++) {
            new Thread(() -> {
                lock.lock();
                try {
                    try {
                        Thread.sleep(10);
                    } catch (InterruptedException e) {
                        e.printStackTrace();
                    }
                    log.debug(Thread.currentThread().getName() + " running...");
                } finally {
                    lock.unlock();
                }
            }, "t" + i).start();
        }
        // 1s 之后去争抢锁
        Thread.sleep(1000);

        for (int i = 0; i < 500; i++) {
            new Thread(() -> {
                lock.lock();
                try {
                    log.debug(Thread.currentThread().getName() + " running...");
                } finally {
                    lock.unlock();
                }
            }, "强行插入" + i).start();
        }

    }

}

```

9.4.6 条件变量

java.util.concurrent类库中提供Condition类来实现线程之间的协调。调用Condition.await() 方法使线程等待，其他线程调用Condition.signal() 或 Condition.signalAll() 方法唤醒等待的线程。

注意：调用Condition的await()和signal()方法，都必须在lock保护之内。

基本使用：

```
@Slf4j
public class ConditionTest {

    public static void main(String[] args) {
        Lock lock = new ReentrantLock();
        Condition condition = lock.newCondition();

        new Thread(() -> {
            lock.lock();
            try {
                log.debug(Thread.currentThread().getName() + " 开始处理任务");
                //会释放当前持有的锁，然后阻塞当前线程
                condition.await();
                log.debug(Thread.currentThread().getName() + " 结束处理任务");
            } catch (InterruptedException e) {
                e.printStackTrace();
            } finally {
                lock.unlock();
            }
        }).start();

        new Thread(() -> {
            lock.lock();
            try {
                log.debug(Thread.currentThread().getName() + " 开始处理任务");

                Thread.sleep(2000);
                //唤醒因调用Condition#await方法而阻塞的线程
                condition.signal();
                log.debug(Thread.currentThread().getName() + " 结束处理任务");
            } catch (Exception e) {
                e.printStackTrace();
            } finally {
                lock.unlock();
            }
        }).start();

    }
}
```

```
@Slf4j
public class ReentrantLockDemo6 {
    private static ReentrantLock lock = new ReentrantLock();
    private static Condition cigCon = lock.newCondition();
    private static Condition takeCon = lock.newCondition();
```

```

private static boolean hashcig = false;
private static boolean hastakeout = false;

//送烟
public void cigratee(){
    lock.lock();
    try {
        while(!hashcig){
            try {
                log.debug("没有烟，歇一会");
                cigCon.await();

            }catch (Exception e){
                e.printStackTrace();
            }
        }
        log.debug("有烟了，干活");
    }finally {
        lock.unlock();
    }
}

//送外卖
public void takeout(){
    lock.lock();
    try {
        while(!hastakeout){
            try {
                log.debug("没有饭，歇一会");
                takeCon.await();

            }catch (Exception e){
                e.printStackTrace();
            }
        }
        log.debug("有饭了，干活");
    }finally {
        lock.unlock();
    }
}

public static void main(String[] args) throws Exception{
    ReentrantLockDemo6 test = new ReentrantLockDemo6();
    new Thread(() ->{
        test.cigratee();
    }).start();

    new Thread(() -> {
        test.takeout();
    }).start();

    TimeUnit.SECONDS.sleep(5);
    new Thread(() ->{
        lock.lock();

```

```

        try {
            hashcig = true;
            log.debug("唤醒送烟的等待线程");
            cigCon.signal();
        }finally {
            lock.unlock();
        }

}, "t1").start();

new Thread() ->{
    lock.lock();
    try {
        hastakeout = true;
        log.debug("唤醒送饭的等待线程");
        takeCon.signal();
    }finally {
        lock.unlock();
    }

}, "t2").start();
}
}

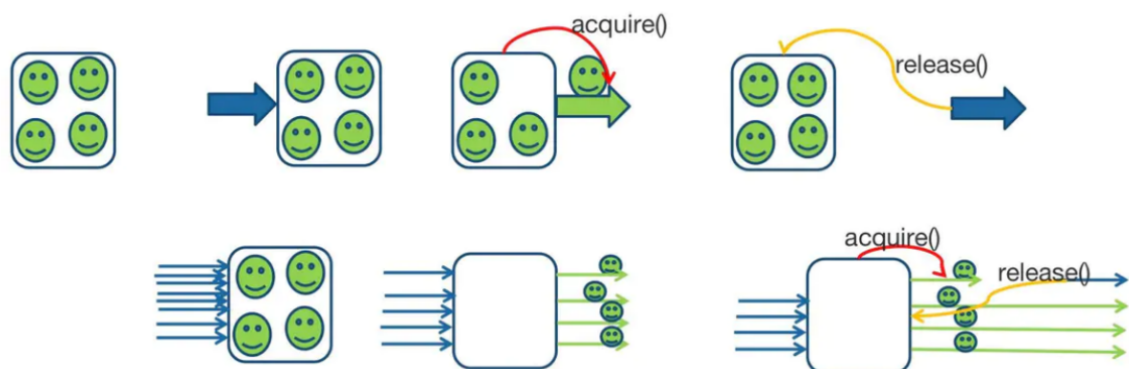
```

9.5 Semaphore 【基于AQS共享锁实现】

9.5.1 简介

Semaphore，俗称信号量，它是操作系统中PV操作的原语在java的实现，它也是基于AbstractQueuedSynchronizer实现的。

Semaphore的功能非常强大，大小为1的信号量就类似于互斥锁，通过同时只能有一个线程获取信号量实现。大小为n（ $n > 0$ ）的信号量可以实现限流的功能，它可以实现只能有n个线程同时获取信号量。



PV操作是操作系统一种实现进程互斥与同步的有效方法。

PV操作与信号量（S）的处理相关，P表示通过的意思，V表示释放的意思。

用PV操作来管理共享资源时，首先要确保PV操作自身执行的正确性。

P操作的主要动作是：

①S减1；

②若S减1后仍大于或等于0，则进程继续执行；

③若S减1后小于0，则该进程被阻塞后放入等待该信号量的等待队列中，然后转进程调度。

V操作的主要动作是：

①S加1；

②若相加后结果大于0，则进程继续执行；

③若相加后结果小于或等于0，则从该信号的等待队列中释放一个等待进程，然后再返回原进程继续执行或转进程调度。

9.5.2 Semaphore 常用方法

构造器：

```
@Contract(pure = true)
public Semaphore(int permits) { sync = new NonfairSync(permits); }

/** Creates a {@code Semaphore} with the given number of ...*/
@Contract(pure = true)
public Semaphore(int permits, boolean fair) { sync = fair ? new FairSync(permits) : new NonfairSync(permits); }
```

- permits 表示许可证的数量（资源数）
- fair 表示公平性，如果这个设为 true 的话，下次执行的线程会是等待最久的线程

常用方法

```
public void acquire() //表示阻塞并获取许可
public boolean tryAcquire() //方法在没有许可的情况下会立即返回 false，要获取许可的线程不会阻塞
public void release() //表示释放许可
public int availablePermits() //返回此信号量中当前可用的许可证数。
public final int getQueueLength() //返回正在等待获取许可证的线程数。
public final boolean hasQueuedThreads() //是否有线程正在等待获取许可证。
protected void reducePermits(int reduction) //减少 reduction 个许可证
Collection getQueuedThreads()//返回所有等待获取许可证的线程集合
```

应用场景

可以用于做流量控制，特别是公用资源有限的应用场景

基本使用

```
public class SemaphoreTest {

    public static void main(String[] args) {
        // 声明3个窗口 state: 资源数
        Semaphore windows = new Semaphore(3);

        for (int i = 0; i < 5; i++) {
            new Thread(new Runnable() {
                @Override
                public void run() {
```

```

        try {
            // 占用窗口    加锁
            windows.acquire();
            System.out.println(Thread.currentThread().getName() + ":
开始买票");

            //模拟买票流程
            Thread.sleep(5000);
            System.out.println(Thread.currentThread().getName() + ":
购票成功");

        } catch (InterruptedException e) {
            e.printStackTrace();
        } finally {
            // 释放窗口
            windows.release();
        }
    }
}).start();
}
}
}

```

限流

```

public class SemaphoreTest2 {

    /**
     * 实现一个同时只能处理5个请求的限流器
     */
    private static Semaphore semaphore = new Semaphore(5);

    /**
     * 定义一个线程池
     */
    private static ThreadPoolExecutor executor = new ThreadPoolExecutor
        (10, 50, 60,
         TimeUnit.SECONDS, new LinkedBlockingDeque<>(200));

    /**
     * 模拟执行方法
     */
    public static void exec() {
        try {
            //占用1个资源
            semaphore.acquire(1);
            //TODO 模拟业务执行
            System.out.println("执行exec方法");
            Thread.sleep(2000);
        } catch (Exception e) {
            e.printStackTrace();
        } finally {
            //释放一个资源
            semaphore.release(1);
        }
    }
}

```

```

    }

    public static void main(String[] args) throws InterruptedException {
        {
            for ( ; ; ) {
                Thread.sleep(100);
                // 模拟请求以10个/s的速度
                executor.execute(() -> exec());
            }
        }
    }
}

```

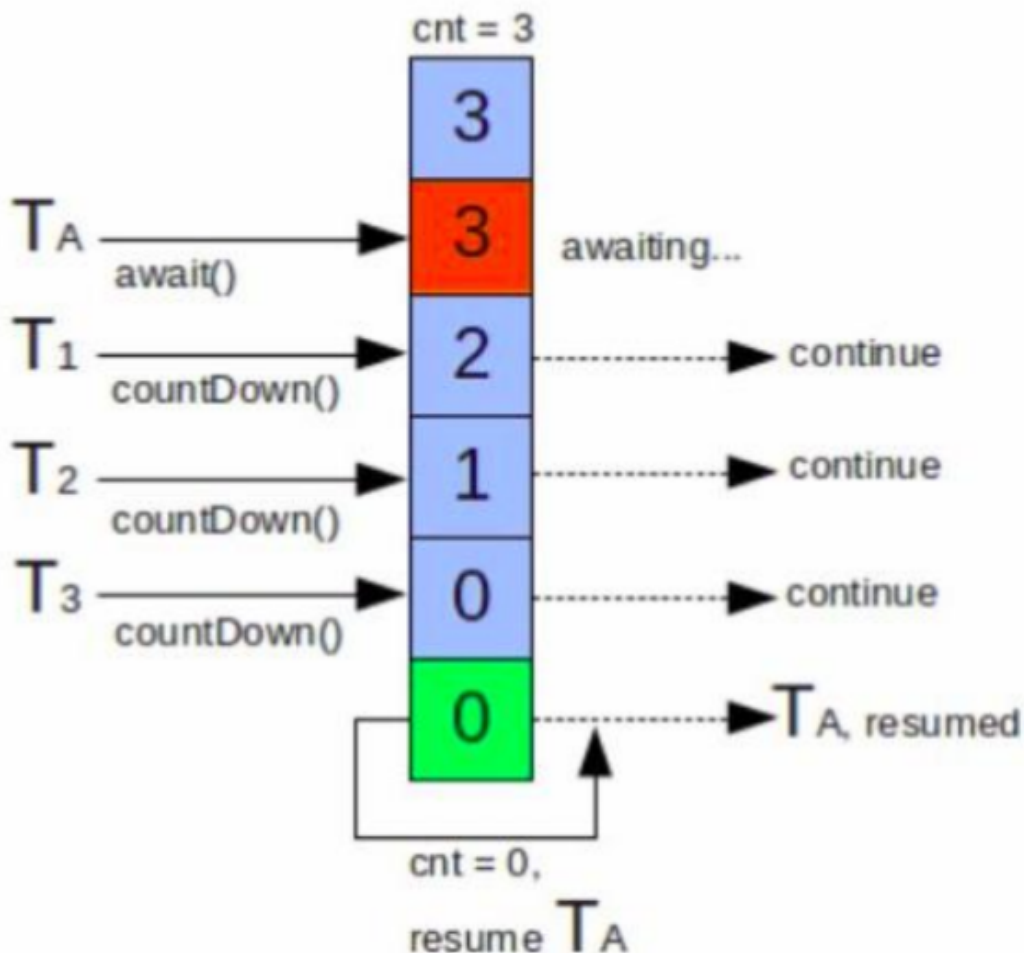
9.6 CountdownLatch 【基于AQS实现】

9.6.1 简介

类似于join 或者 wait
() notify()

CountDownLatch (闭锁) 是一个同步协助类，允许一个或多个线程等待，直到其他线程完成操作集。

CountDownLatch使用给定的计数值 (count) 初始化。**await方法会阻塞直到当前的计数值 (count) 由于countDown方法的调用达到0，count为0之后所有等待的线程都会被释放，并且随后对await方法的调用都会立即返回。这是一个一次性现象 —— count不会被重置。**如果你需要一个重置count的版本，那么请考虑使用CyclicBarrier。



9.6.2 CountdownLatch的使用

构造器

```
public CountdownLatch(int count) {  
    if (count < 0) throw new IllegalArgumentException("count < 0");  
    this.sync = new Sync(count);  
}
```

常用方法

```
// 调用 await() 方法的线程会被挂起，它会等待直到 count 值为 0 才继续执行  
public void await() throws InterruptedException { };  
// 和 await() 类似，若等待 timeout 时长后，count 值还是没有变为 0，不再等待，继续执行  
public boolean await(long timeout, TimeUnit unit) throws InterruptedException {  
};  
// 会将 count 减 1，直至为 0  
public void countDown();
```

CountDownLatch应用场景

CountDownLatch一般用作多线程倒计时计数器，强制它们等待其他一组（CountDownLatch的初始化决定）任务执行完成。

CountDownLatch的两种使用场景：

- 场景1：让多个线程等待
- 场景2：让单个线程等待。

场景1 让多个线程等待：模拟并发，让并发线程一起执行

```
public class CountdownLatchTest {  
    public static void main(String[] args) throws InterruptedException {  
  
        CountdownLatch countDownLatch = new CountdownLatch(1);  
        for (int i = 0; i < 5; i++) {  
            new Thread(() -> {  
                try {  
                    //准备完毕.....运动员都阻塞在这，等待号令  
                    countDownLatch.await();  
                    String parter = "[" + Thread.currentThread().getName() +  
                        " ] ";  
                    System.out.println(parter + "开始执行.....");  
                } catch (InterruptedException e) {  
                    e.printStackTrace();  
                }  
            }).start();  
        }  
  
        Thread.sleep(2000); // 裁判准备发令  
        countDownLatch.countDown(); // 发令枪：执行发令  
    }  
}
```


场景2 让单个线程等待：多个线程(任务)完成后，进行汇总合并

很多时候，我们的并发任务，存在前后依赖关系；比如数据详情页需要同时调用多个接口获取数据，并发请求获取到数据后、需要进行结果合并；或者多个数据操作完成后，需要数据check；这其实都是在多个线程(任务)完成后，进行汇总合并的场景。

```
/**
 * 让单个线程等待：多个线程(任务)完成后，进行汇总合并
 */
public class CountdownLatchTest2 {
    public static void main(String[] args) throws Exception {

        CountdownLatch countDownLatch = new CountdownLatch(5);
        for (int i = 0; i < 5; i++) {
            final int index = i;
            new Thread(() -> {
                try {
                    Thread.sleep(1000 +
                        ThreadLocalRandom.current().nextInt(1000));
                    System.out.println(Thread.currentThread().getName()
                        + " finish task" + index);

                    countDownLatch.countDown();
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }).start();
        }

        // 主线程在阻塞，当计数器==0，就唤醒主线程往下执行。
        countDownLatch.await();
        System.out.println("主线程:在所有任务运行完成后，进行结果汇总");

    }
}
```

```
public class CountdownLatchTest3 {

    //用于聚合所有的统计指标
    private static Map map = new ConcurrentHashMap();
    //创建计数器，这里需要统计4个指标
    private static CountdownLatch countDownLatch = new CountdownLatch(4);

    public static void main(String[] args) throws Exception {

        //记录开始时间
        long startTime = System.currentTimeMillis();

        Thread countUserThread = new Thread(() -> {
            try {
                System.out.println("正在统计新增用户数量");
                Thread.sleep(3000); //任务执行需要3秒
            }
        });
    }
}
```

```

        map.put("userNumber", 100); //保存结果值
        System.out.println("统计新增用户数量完毕");
        countDownLatch.countDown(); //标记已经完成一个任务
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
});

Thread countOrderThread = new Thread(() -> {
    try {
        System.out.println("正在统计订单数量");
        Thread.sleep(3000); //任务执行需要3秒
        map.put("countOrder", 20); //保存结果值
        System.out.println("统计订单数量完毕");
        countDownLatch.countDown(); //标记已经完成一个任务
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
});

Thread countGoodsThread = new Thread(() -> {
    try {
        System.out.println("正在商品销量");
        Thread.sleep(3000); //任务执行需要3秒
        map.put("countGoods", 300); //保存结果值
        System.out.println("统计商品销量完毕");
        countDownLatch.countDown(); //标记已经完成一个任务
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
});

Thread countmoneyThread = new Thread(() -> {
    try {
        System.out.println("正在总销售额");
        Thread.sleep(3000); //任务执行需要3秒
        map.put("countMoney", 40000); //保存结果值
        System.out.println("统计销售额完毕");
        countDownLatch.countDown(); //标记已经完成一个任务
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
});

//启动子线程执行任务
countUserThread.start();
countGoodsThread.start();
countOrderThread.start();
countmoneyThread.start();

//
countUserThread.join();
countGoodsThread.join();
countOrderThread.join();
countmoneyThread.join();

try {
    //主线程等待所有统计指标执行完毕
    countDownLatch.await();
}

```

```

        long endTime = System.currentTimeMillis();//记录结束时间
        System.out.println("-----统计指标全部完成-----");
        System.out.println("统计结果为: " + map);
        System.out.println("任务总执行时间为" + (endTime - startTime) + "ms");
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}
}
}

```

CountDownLatch与Thread.join的区别

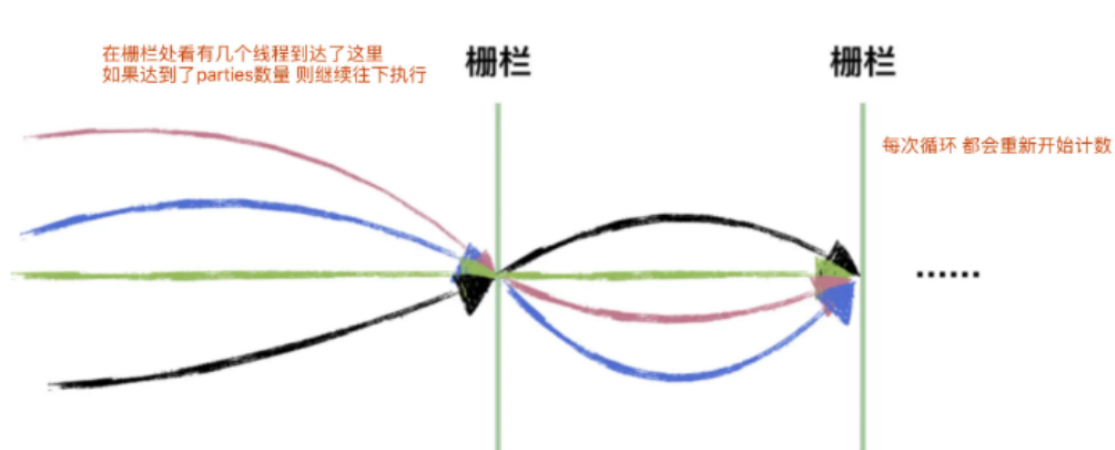
- CountDownLatch的作用就是允许一个或多个线程等待其他线程完成操作，看起来有点类似join()方法，但其提供了比 join() 更加灵活的API。
- CountDownLatch可以手动控制在n个线程里调用n次countDown()方法使计数器进行减一操作，也可以在一个线程里调用n次执行减一操作。
- 而 join() 的实现原理是不停检查join线程是否存活，如果 join 线程存活则让当前线程永远等待。所以两者之间相对来说还是CountDownLatch使用起来较为灵活。

9.7 CyclicBarrier 【基于reentrantlock+condition实现】

9.7.1 简介

与CountDownLatch相同用法，但内部逻辑相反CountDownLatch使用向下计数（-1），CyclicBarrier使用向上计数（+1）

回环栅栏，通过它可以实现让一组线程等待至某个状态（屏障点）之后再全部同时执行。叫做回环是因为当所有等待线程都被释放以后，CyclicBarrier可以被重用。



9.7.2 使用

构造方法

```
// parties表示屏障拦截的线程数量，每个线程调用 await 方法告诉 CyclicBarrier 我已经到达了屏障，然后当前线程被阻塞。  
public CyclicBarrier(int parties)  
// 用于在线程到达屏障时，优先执行 barrierAction，方便处理更复杂的业务场景(该线程的执行时机是在到达屏障之后再执行)  
public CyclicBarrier(int parties, Runnable barrierAction)
```

重要方法

```
//屏障 指定数量的线程全部调用await()方法时，这些线程不再阻塞  
// BrokenBarrierException 表示栅栏已经被破坏，破坏的原因可能是其中一个线程 await() 时被中断或者超时  
public int await()  
public int await(long timeout, TimeUnit unit) ;  
  
//循环 通过reset()方法可以进行重置  
public void reset()
```

CyclicBarrier应用场景

CyclicBarrier 可以用于多线程计算数据，最后合并计算结果的场景

基本使用：

```
/**  
 * CyclicBarrier(回环栅栏)允许一组线程互相等待，直到到达某个公共屏障点 (Common Barrier Point)  
 * 闭锁用于等待countDown事件，而栅栏用于等待其他线程。  
 */  
public class CyclicBarrierTest {  
  
    public static void main(String[] args) {  
  
        CyclicBarrier cyclicBarrier = new CyclicBarrier(3);  
  
        for (int i = 0; i < 3; i++) {  
            new Thread(new Runnable() {  
                @Override  
                public void run() {  
                    try {  
                        System.out.println(Thread.currentThread().getName() + "开始等待其他线程");  
                        cyclicBarrier.await();  
                        System.out.println(Thread.currentThread().getName() + "开始执行");  
  
                        //TODO 模拟业务处理  
                        Thread.sleep(5000);  
                        System.out.println(Thread.currentThread().getName() + "执行完毕");  
  
                    } catch (Exception e) {  
                        e.printStackTrace();  
                    }  
                }  
            }) {  
        }  
    }  
}
```

```

        }).start();

    }

}
}

```

示例2:

```

/**
 * 栅栏与闭锁的关键区别在于，所有的线程必须同时到达栅栏位置，才能继续执行。
 */
public class CyclicBarrierTest2 {

    //保存每个学生的平均成绩
    private ConcurrentHashMap<String, Integer> map=new
    ConcurrentHashMap<String,Integer>();

    private ExecutorService threadPool= Executors.newFixedThreadPool(3);

    private CyclicBarrier cb=new CyclicBarrier(3,()->{
        int result=0;
        Set<String> set = map.keySet();
        for(String s:set){
            result+=map.get(s);
        }
        System.out.println("三人平均成绩为:"+(result/3)+"分");

        threadPool.shutdown();
    });

    public void count(){
        for(int i=0;i<3;i++){
            threadPool.execute(new Runnable(){

                @Override
                public void run() {
                    //获取学生平均成绩
                    int score=(int)(Math.random()*40+60);
                    map.put(Thread.currentThread().getName(), score);
                    System.out.println(Thread.currentThread().getName()
                        +"同学的平均成绩为: "+score);

                    try {
                        //执行完运行await(),等待所有学生平均成绩都计算完毕
                        cb.await();
                    } catch (InterruptedException | BrokenBarrierException e) {
                        e.printStackTrace();
                    }
                }

            });
        }
    }
}

```

```

    }

    public static void main(String[] args) {
        cyclicBarrierTest2 cb=new CyclicBarrierTest2();
        cb.count();
    }
}

```

示例3:

```

public class CyclicBarrierTest3 {

    public static void main(String[] args) {

        AtomicInteger counter = new AtomicInteger();
        ThreadPoolExecutor threadPoolExecutor = new ThreadPoolExecutor(
            5, 5, 1000, TimeUnit.SECONDS,
            new ArrayBlockingQueue<>(100),
            (r) -> new Thread(r, counter.addAndGet(1) + " 号 "),
            new ThreadPoolExecutor.AbortPolicy());

        CyclicBarrier cyclicBarrier = new CyclicBarrier(5,
            () -> System.out.println("裁判: 比赛开始~~"));

        for (int i = 0; i < 10; i++) {
            threadPoolExecutor.submit(new Runner(cyclicBarrier));
        }

    }

    static class Runner extends Thread{
        private CyclicBarrier cyclicBarrier;
        public Runner (CyclicBarrier cyclicBarrier) {
            this.cyclicBarrier = cyclicBarrier;
        }

        @Override
        public void run() {
            try {
                int sleepMills = ThreadLocalRandom.current().nextInt(1000);
                Thread.sleep(sleepMills);
                System.out.println(Thread.currentThread().getName() + " 选手已就
位, 准备共用时: " + sleepMills + "ms" + cyclicBarrier.getNumberWaiting());
                cyclicBarrier.await();

            } catch (InterruptedException e) {
                e.printStackTrace();
            } catch (BrokenBarrierException e){
                e.printStackTrace();
            }
        }
    }
}

```

```
}
```

CountDownLatch与CyclicBarrier的区别

1. **CountDownLatch的计数器只能使用一次，而CyclicBarrier的计数器可以使用reset() 方法重置。**所以CyclicBarrier能处理更为复杂的业务场景，比如如果计算发生错误，可以重置计数器，并让线程们重新执行一次
2. CyclicBarrier还提供getNumberWaiting(可以获得CyclicBarrier阻塞的线程数量)、isBroken(用来知道阻塞的线程是否被中断)等方法。
3. CountDownLatch会阻塞主线程，CyclicBarrier不会阻塞主线程，只会阻塞子线程。
4. CountDownLatch和CyclicBarrier都能够实现线程之间的等待，只不过它们侧重点不同。
CountDownLatch一般用于一个或多个线程，等待其他线程执行完任务后，再执行。CyclicBarrier一般用于一组线程互相等待至某个状态，然后这一组线程再同时执行。
5. CyclicBarrier 还可以提供一个 barrierAction，合并多线程计算结果。
6. **CyclicBarrier是通过ReentrantLock的"独占锁"和Condition来实现一组线程的阻塞唤醒的，而CountDownLatch则是通过AQS的"共享锁"实现**

9.8 ReentrantReadWriteLock

9.8.1 读写锁介绍

现实中有这样一种场景：对共享资源有读和写的操作，且写操作没有读操作那么频繁（读多写少）。在没有写操作的时候，多个线程同时读一个资源没有任何问题，所以应该允许多个线程同时读取共享资源（读读可以并发）；但是如果一个线程想去写这些共享资源，就不应该允许其他线程对该资源进行读和写操作了（读写，写读，写写互斥）。在读多于写的情况下，读写锁能够提供比排它锁更好的并发性和吞吐量。

针对这种场景，JAVA的并发包提供了读写锁ReentrantReadWriteLock，它内部，维护了一对相关的锁，一个用于只读操作，称为读锁；一个用于写入操作，称为写锁，描述如下：

线程进入读锁的前提条件：

- 没有其他线程的写锁
- 没有写请求或者有写请求，但调用线程和持有锁的线程是同一个。

线程进入写锁的前提条件：

- 没有其他线程的读锁
- 没有其他线程的写锁

而读写锁有以下三个重要的特性：

- 公平选择性：支持非公平（默认）和公平的锁获取方式，吞吐量还是非公平优于公平。
- 可重入：读锁和写锁都支持线程重入。以读写线程为例：读线程获取读锁后，能够再次获取读锁。写线程在获取写锁之后能够再次获取写锁，同时也可以获取读锁。
- 锁降级：遵循获取写锁、再获取读锁最后释放写锁的次序，写锁能够降级成为读锁。

锁降级指的是写锁降级成为读锁。如果当前线程拥有写锁，然后将其释放，最后再获取读锁，这种分段完成的过程不能称之为锁降级。**锁降级是指把持住（当前拥有的）写锁，再获取到读锁，随后释放（先前拥有的）写锁的过程。**锁降级可以帮助我们拿到当前线程修改后的结果而不被其他线程所破坏，防止更新丢失。

9.8.2 ReentrantReadWriteLock的使用

读写锁接口ReadWriteLock

```
public interface ReadWriteLock {  
    /**  
     * Returns the lock used for reading.  
     *  
     * @return the lock used for reading  
     */  
    Lock readLock();  
  
    /**  
     * Returns the lock used for writing.  
     *  
     * @return the lock used for writing  
     */  
    Lock writeLock();  
}
```

如何使用读写锁

```
public class ReadWriteLockTest {  
  
    public static void main(String[] args) throws InterruptedException {  
        final Queue queue = new Queue();  
  
        for (int i = 0; i < 3; i++) {  
            new Thread(new Runnable() {  
                @Override  
                public void run() {  
                    queue.put(new Random().nextInt(10000));  
                }  
            }).start();  
        }  
  
        Thread.sleep(100);  
  
        for (int i = 0; i < 3; i++) {  
            new Thread(new Runnable() {  
                @Override  
                public void run() {  
                    queue.get();  
                }  
            })  
        }  
    }  
}
```



```

        }).start();
    }

}

}

}

@Slf4j
class Queue {
    //共享数据，只能有一个线程能写该数据，但可以有多个线程同时读该数据。
    private Object data = null;
    private ReadWriteLock readWriteLock = new ReentrantReadWriteLock();
    private Lock readLock = readWriteLock.readLock();
    private Lock writeLock = readWriteLock.writeLock();

    public void get() {
        log.debug(Thread.currentThread().getName() + " be ready to read data!");
        //上读锁，其他线程只能读不能写
        readLock.lock();
        try {
            Thread.sleep(1000);
            log.debug(Thread.currentThread().getName() + " have read data : " +
data);
        } catch (InterruptedException e) {
            e.printStackTrace();
        } finally {
            //释放读锁
            readLock.unlock();
        }
    }

    public void put(Object data) {
        log.debug(Thread.currentThread().getName() + " be ready to write
data!");
        //上写锁，不允许其他线程读也不允许写
        writeLock.lock();
        try {
            Thread.sleep(5000);
            this.data = data;
            log.debug(Thread.currentThread().getName() + " have write data: " +
data);
        } catch (InterruptedException e) {
            e.printStackTrace();
        } finally {
            //释放写锁
            writeLock.unlock();
        }
    }
}

```

10、阻塞队列

基本都是基于reentrantlock +condition实现阻塞的

put()方法条件元素，放满了，会进行等待

10.1 队列的基本认识

Queue接口

```
public interface Queue<E> extends Collection<E> {
    //添加一个元素，添加成功返回true，如果队列满了，就会抛出异常
    boolean add(E e);
    //添加一个元素，添加成功返回true，如果队列满了，返回false
    boolean offer(E e);
    //返回并删除队首元素，队列为空则抛出异常
    E remove();
    //返回并删除队首元素，队列为空则返回null
    E poll();
    //返回队首元素，但不移除，队列为空则抛出异常
    E element();
    //获取队首元素，但不移除，队列为空则返回null
    E peek();
}
```

BlockingQueue接口

BlockingQueue 继承了 Queue 接口，是队列的一种。Queue 和 BlockingQueue 都是在 Java 5 中加入的。**阻塞队列 (BlockingQueue) 是一个在队列基础上又支持了两个附加操作的队列**，常用解耦。两个附加操作：

- 支持阻塞的插入方法put: 队列满时，队列会阻塞插入元素的线程，直到队列不满。
- 支持阻塞的移除方法take: 队列空时，获取元素的线程会等待队列变为非空

```
public interface BlockingQueue<E> extends Queue<E> {
    /**...*/
    boolean add( @NotNull E e);

    /**...*/
    boolean offer( @NotNull E e);

    /**...*/
    void put( @NotNull E e) throws InterruptedException;

    /**...*/
    boolean offer(E e, long timeout, @NotNull TimeUnit unit)
        throws InterruptedException;

    /**...*/
    @NotNull E take() throws InterruptedException;

    /**...*/
    @Nullable E poll(long timeout, @NotNull TimeUnit unit)
        throws InterruptedException;

    /**...*/
    int remainingCapacity();

    /**...*/
    boolean remove(Object o);
}
```

BlockingQueue和JDK集合包中的Queue接口兼容，同时在其基础上**增加了阻塞功能**。

入队：

- (1) offer(E e): 如果队列没满，返回true，如果队列已满，返回false（不阻塞）
- (2) offer(E e, long timeout, TimeUnit unit): 可以设置阻塞时间，如果队列已满，则进行阻塞。超过阻塞时间，则返回false
- (3) **put(E e): 队列没满的时候是正常的插入，如果队列已满，则阻塞，直至队列空出位置**

出队：

- (1) poll(): 如果有数据，出队，如果没有数据，返回null（不阻塞）

(2) poll(long timeout, TimeUnit unit): 可以设置阻塞时间, 如果没有数据, 则阻塞, 超过阻塞时间, 则返回null

(3) **take(): 队列里有数据会正常取出数据并删除; 但是如果队列里无数据, 则阻塞, 直到队列里有数据**

当队列满了无法添加元素, 或者是队列空了无法移除元素时:

1. 抛出异常: add、remove、element
2. 返回结果但不抛出异常: offer、poll、peek
3. 阻塞: put、take

方法	抛出异常	返回特定值	阻塞	阻塞特定时间
入队	add(e)	offer(e)	put(e)	offer(e, time, unit)
出队	remove()	poll()	take()	poll(time, unit)
获取队首元素	element()	peek()	不支持	不支持

常见API测试

```
public class BlockingQueueTest {

    public static void main(String[] args) {

        takeTest();

    }

    /**
     * add 方法是往队列里添加一个元素, 如果队列满了, 就会抛出异常来提示队列已满。
     */
    private static void addTest() {
        BlockingQueue<Integer> blockingQueue = new ArrayBlockingQueue<Integer>
(2);
        System.out.println(blockingQueue.add(1));
        System.out.println(blockingQueue.add(2));
        System.out.println(blockingQueue.add(3));
    }

    /**
     * remove 方法的作用是删除元素并返回队列的头节点, 如果删除的队列是空的, remove 方法就会
抛出异常。
     */
    private static void removeTest() {
        ArrayBlockingQueue<Integer> blockingQueue = new
ArrayBlockingQueue<Integer>(2);
        blockingQueue.add(1);
        blockingQueue.add(2);
        System.out.println(blockingQueue.remove());
        System.out.println(blockingQueue.remove());
        System.out.println(blockingQueue.remove());
    }

    /**
```

```

    * element 方法是返回队列的头部节点，但是并不删除。如果队列为空，抛出异常
    */
    private static void elementTest() {
        ArrayBlockingQueue<Integer> blockingQueue = new
ArrayBlockingQueue<Integer>(2);
        blockingQueue.element();
    }

    /**
    * offer 方法用来插入一个元素。如果添加成功会返回 true，而如果队列已经满了，返回false
    */
    private static void offerTest(){
        ArrayBlockingQueue<Integer> blockingQueue = new
ArrayBlockingQueue<Integer>(2);
        System.out.println(blockingQueue.offer(1));
        System.out.println(blockingQueue.offer(2));
        System.out.println(blockingQueue.offer(3));
    }

    /**
    * poll 方法作用也是移除并返回队列的头节点。 如果队列为空，返回null
    */
    private static void pollTest() {
        ArrayBlockingQueue<Integer> blockingQueue = new
ArrayBlockingQueue<Integer>(3);
        blockingQueue.offer(1);
        blockingQueue.offer(2);
        blockingQueue.offer(3);
        System.out.println(blockingQueue.poll());
        System.out.println(blockingQueue.poll());
        System.out.println(blockingQueue.poll());
        System.out.println(blockingQueue.poll());
    }

    /**
    * peek 方法返回队列的头元素但并不删除。 如果队列为空，返回null
    */
    private static void peekTest() {
        ArrayBlockingQueue<Integer> blockingQueue = new
ArrayBlockingQueue<Integer>(2);
        System.out.println(blockingQueue.peek());
    }

    /**
    * put 方法的作用是插入元素。如果队列已满就无法继续插入,阻塞插入线程，直至队列空出位置
    */
    private static void putTest(){
        BlockingQueue<Integer> blockingQueue = new ArrayBlockingQueue<Integer>
(2);
        try {
            blockingQueue.put(1);
            blockingQueue.put(2);
            blockingQueue.put(3);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }

```

```

    }
}

/**
 * take 方法的作用是获取并移除队列的头结点。如果队列里无数据，则阻塞，直到队列里有数据
 */
private static void takeTest(){
    BlockingQueue<Integer> blockingQueue = new ArrayBlockingQueue<Integer>
(2);
    try {
        blockingQueue.take();
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}
}

```

10.2 阻塞队列特性

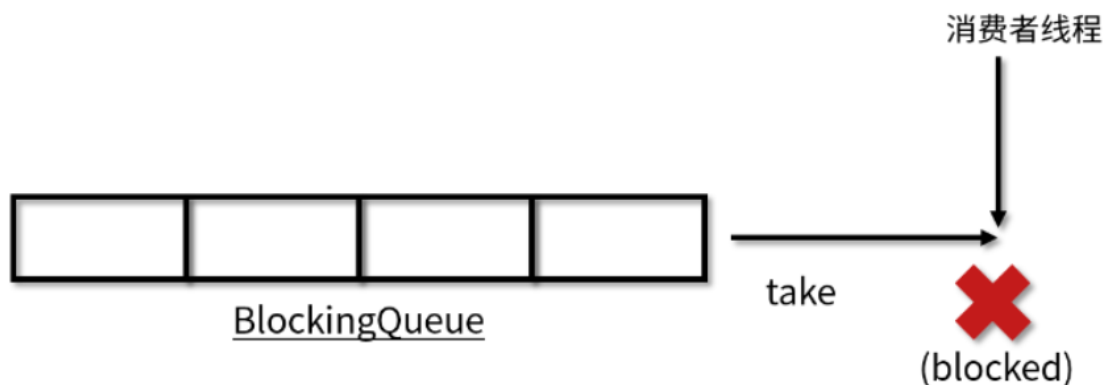
10.2.1 阻塞

阻塞

阻塞队列区别于其他类型的队列的最主要的特点就是“阻塞”这两个字，所以下面重点介绍阻塞功能：阻塞功能使得生产者和消费者两端的能力得以平衡，当有任何一端速度过快时，阻塞队列便会把过快的速度给降下来。实现阻塞最重要的两个方法是 take 方法和 put 方法。

take 方法

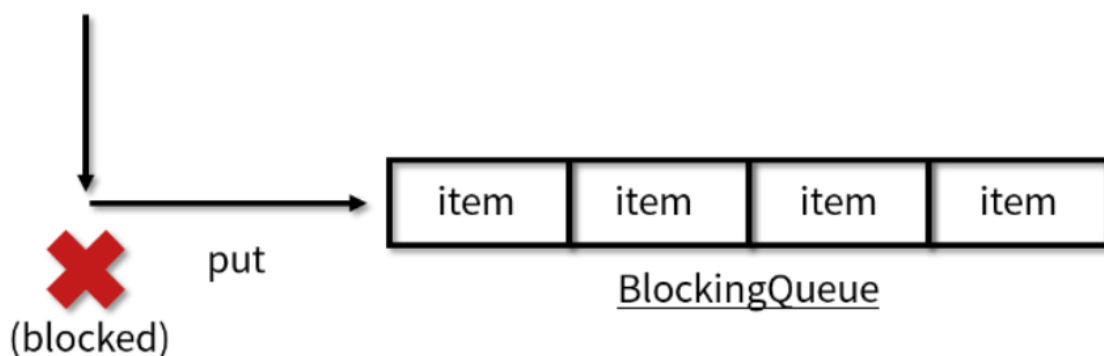
take 方法的功能是获取并移除队列的头结点，通常在队列里有数据的时候是可以正常移除的。可**一旦执行 take 方法的时候，队列里无数据，则阻塞，直到队列里有数据**。一旦队列里有数据了，就会立刻解除阻塞状态，并且取到数据。过程如图所示：



put 方法

put 方法插入元素时，如果队列没有满，那就和普通的插入一样是正常的插入，**但是如果队列已满，那么就无法继续插入，则阻塞**，直到队列里有了空闲空间。如果后续队列有了空闲空间，比如消费者消费了一个元素，那么此时队列就会解除阻塞状态，并把需要添加的数据添加到队列中。过程如图所示：

生产者线程

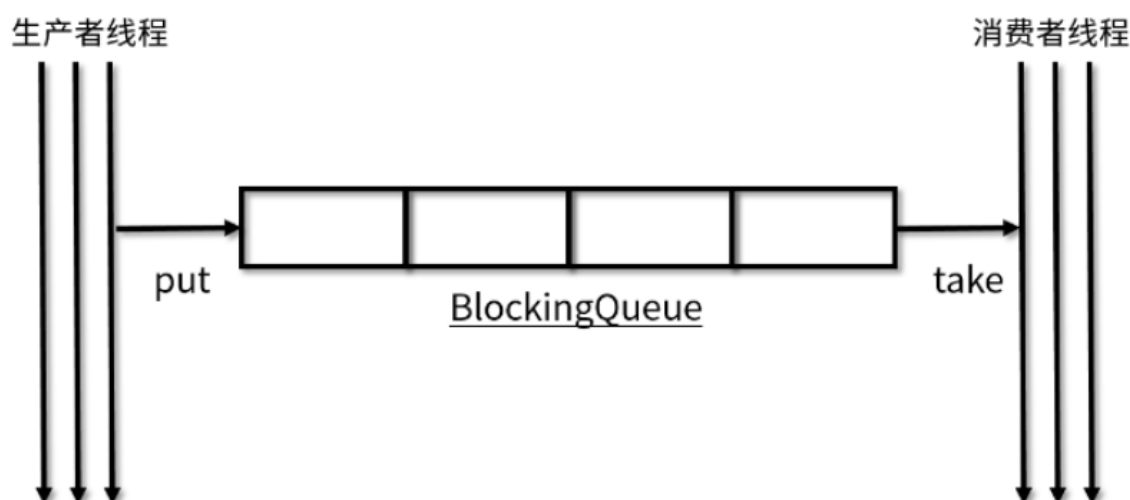


10.2.2 有界？

阻塞队列还有一个非常重要的属性，那就是容量的大小，分为**有界和无界**两种。无界队列意味着里面可以容纳非常多的元素，例如 `LinkedBlockingQueue` 的上限是 `Integer.MAX_VALUE`，是非常大的一个数，可以近似认为是无限容量，因为我们几乎无法把这个容量装满。但是有的阻塞队列是有界的，例如 `ArrayBlockingQueue` 如果容量满了，也不会扩容，所以一旦满了就无法再往里放数据了。

10.3 应用场景

`BlockingQueue` 是线程安全的，我们在很多场景下都可以利用线程安全的队列来优雅地解决我们业务自身的线程安全问题。比如说，使用生产者/消费者模式的时候，我们生产者只需要往队列里添加元素，而消费者只需要从队列里取出它们就可以了，如图所示：



因为阻塞队列是线程安全的，所以生产者和消费者都可以是多线程的，不会发生线程安全问题。**生产者/消费者直接使用线程安全的队列就可以，而不需要自己去考虑更多的线程安全问题。**这也就意味着，考虑锁等线程安全问题的重任从“你”转移到了“队列”上，降低了我们开发的难度和工作量。

同时，**队列它还能起到一个隔离的作用。**比如说我们开发一个银行转账的程序，那么生产者线程不需要关心具体的转账逻辑，只需要把转账任务，如账户和金额等信息放到队列中就可以，而不需要去关心银行这个类如何实现具体的转账业务。而作为银行这个类来讲，它会去从队列里取出来将要执行的具体的任务，再去通过自己的各种方法来完成本次转账。**这样就实现了具体任务与执行任务类之间的解耦**，任务被放在了阻塞队列中，而负责放任务的线程是无法直接访问到我们银行具体实现转账操作的对象的，实现了隔离，提高了安全性。

10.4 常见阻塞队列

BlockingQueue 接口的实现类都被放在了 juc 包中，它们的区别主要体现在存储结构上或对元素操作上的不同，但是对于take与put操作的原理，却是类似的。

队列	描述
ArrayBlockingQueue	基于数组结构实现的一个有界阻塞队列
LinkedBlockingQueue	基于链表结构实现的一个有界阻塞队列
PriorityBlockingQueue	支持按优先级排序的无界阻塞队列
DelayQueue	基于优先级队列（PriorityBlockingQueue）实现的无界阻塞队列
SynchronousQueue	不存储元素的阻塞队列
LinkedTransferQueue	基于链表结构实现的一个无界阻塞队列
LinkedBlockingDeque	基于链表结构实现的一个双端阻塞队列

10.4.1 ArrayBlockingQueue

ArrayBlockingQueue是最典型的**有界阻塞队列**，其内部是用数组存储元素的，**初始化时需要指定容量大小，利用 ReentrantLock 实现线程安全。**

在生产者-消费者模型中使用时，**如果生产速度和消费速度基本匹配的情况下，使用 ArrayBlockingQueue是个不错的选择**；当如果生产速度远远大于消费速度，则会导致队列填满，大量生产线程被阻塞。

使用独占锁ReentrantLock实现线程安全，入队和出队操作使用同一个锁对象，也就是只能有一个线程可以进行入队或者出队操作；这也就意味着生产者和消费者无法并行操作，在高并发场景下会成为性能瓶颈。



10.4.1.1 基本使用

```
public class ArrayBlockingQueueDemo {

    public static void main(String[] args) throws Exception {
        //使用ArrayBlockingQueue初始化一个BlockingQueue，指定容量的上限为1024
        BlockingQueue queue = new ArrayBlockingQueue(1024);

        Producer producer = new Producer(queue); //生产者
        Consumer consumer = new Consumer(queue); //消费者
    }
}
```

```

        new Thread(producer).start(); //开启生产者线程
        new Thread(consumer).start(); //开启消费者线程

        Thread.sleep(4000);
    }
}

public class Producer implements Runnable{

    protected BlockingQueue queue = null;

    public Producer(BlockingQueue queue) {
        this.queue = queue;
    }

    @Override
    public void run() {
        try {
            for(int i=0;i<3;i++){
                queue.put(i);
                System.out.println("produce "+i);
                Thread.sleep(10000);
            }
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}

public class Consumer implements Runnable{

    protected BlockingQueue queue = null;

    public Consumer(BlockingQueue queue) {
        this.queue = queue;
    }

    @Override
    public void run() {
        try {
            System.out.println("consumer "+queue.take());
            System.out.println("consumer "+queue.take());
            System.out.println("consumer "+queue.take());
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}

```


10.4.1.2 原理

利用了Lock锁的Condition通知机制进行阻塞控制。

里面是一个数组，这个数组是**环形数组**

为什么用环形数组？

如果不用环形数组，在数组中元素被消费者消费之后，会移动数组中元素，这时的时间复杂度为 $O(n)$ ，换成环形数组后，移除元素的时间复杂度为 $O(1)$

核心：一把锁，两个条件核心：一把锁，两个条件

```
//数据元素数组
final Object[] items;
//下一个待取出元素索引
int takeIndex;
//下一个待添加元素索引
int putIndex;
//元素个数
int count;
//内部锁
final ReentrantLock lock;
//消费者
private final Condition notEmpty;
//生产者
private final Condition notFull;

public ArrayBlockingQueue(int capacity) {
    this(capacity, false);
}
public ArrayBlockingQueue(int capacity, boolean fair) {
    ...
    lock = new ReentrantLock(fair); //公平，非公平
    notEmpty = lock.newCondition();
    notFull = lock.newCondition();
}
```

入队put方法

```
public void put(E e) throws InterruptedException {
    //检查是否为空
    checkNotNull(e);
    final ReentrantLock lock = this.lock;
    //加锁，如果线程中断抛出异常
    lock.lockInterruptibly();
    try {
        //阻塞队列已满，则将生产者挂起，等待消费者唤醒
        //设计注意点： 用while不用if是为了防止虚假唤醒
        while (count == items.length)
            notFull.await(); //队列满了，使用notFull等待（生产者阻塞）
        // 入队
        enqueue(e);
    } finally {
        lock.unlock(); // 唤醒消费者线程
    }
}
```

```

}

private void enqueue(E x) {
    final Object[] items = this.items;
    //入队 使用的putIndex
    items[putIndex] = x;
    if (++putIndex == items.length)
        putIndex = 0; //设计的精髓： 环形数组，putIndex指针到数组尽头了，返回头部
    count++;
    //notEmpty条件队列转同步队列，准备唤醒消费者线程，因为入队了一个元素，肯定不为空了
    notEmpty.signal();
}

```

出队take方法

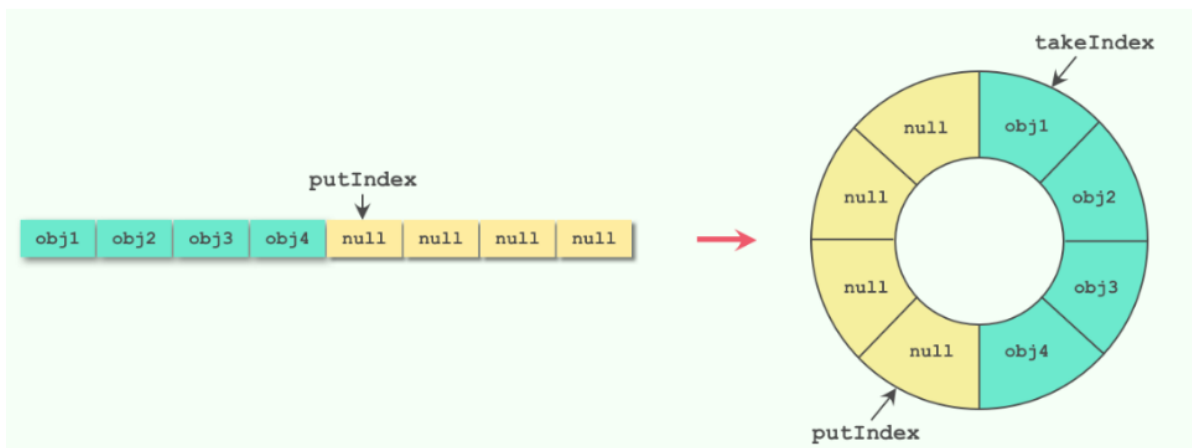
```

public E take() throws InterruptedException {
    final ReentrantLock lock = this.lock;
    //加锁，如果线程中断抛出异常
    lock.lockInterruptibly();
    try {
        //如果队列为空，则消费者挂起
        while (count == 0)
            notEmpty.await();
        //出队
        return dequeue();
    } finally {
        lock.unlock(); // 唤醒生产者线程
    }
}

private E dequeue() {
    final Object[] items = this.items;
    @SuppressWarnings("unchecked")
    E x = (E) items[takeIndex]; //取出takeIndex位置的元素
    items[takeIndex] = null;
    if (++takeIndex == items.length)
        takeIndex = 0; //设计的精髓： 环形数组，takeIndex 指针到数组尽头了，返回头部
    count--;
    if (itrs != null)
        itrs.elementDequeued();
    //notEmpty条件队列转同步队列，准备唤醒生产者线程，此时队列有空位
    notFull.signal();
    return x;
}

```

如果不用环形数组，在数组中元素被消费者消费之后，会移动数组中元素，这时的时间复杂度为 $O(n)$ 。换成环形数组后，移除元素的时间复杂度为 $O(1)$ 。



10.4.2 LinkedBlockingQueue

LinkedBlockingQueue是一个基于链表实现的阻塞队列，默认情况下，该阻塞队列的大小为 `Integer.MAX_VALUE`，由于这个数值特别大，所以 **LinkedBlockingQueue** 也被称作无界队列，代表它几乎没有界限，队列可以随着元素的添加而动态增长，但是**如果没有剩余内存，则队列将抛出OOM错误**。所以为了避免队列过大造成机器负载或者内存爆满的情况出现，我们在使用的时候建议手动传一个队列的大小。

LinkedBlockingQueue内部由**单链表**实现，只能从head取元素，从tail添加元素。

LinkedBlockingQueue采用两把锁的锁分离技术实现入队出队互不阻塞，添加元素和获取元素都有独立的锁，也就是说LinkedBlockingQueue是读写分离的，读写操作可以并行执行。



10.4.2.1 基本使用

```
public class LinkedBlockingQueueTest {

    private static LinkedBlockingQueue<String> linkedBlockingQueue;

    public static void main(String[] args) {

        linkedBlockingQueue = new LinkedBlockingQueue<String>(20);
        // 1、add 方法:队列已满，报java.lang.IllegalStateException: Queue full 错误
        System.out.println("-----add 方法-----");
        for (int i = 0; i < 5; i++) {
            // linkedBlockingQueue.add(String.valueOf(i));
        }
        System.out.println(linkedBlockingQueue.size());

        // 2、offer 方法，队列已满，程序正常运行，只是不再新增元素
        System.out.println("-----offer方法-----");
```

```

    for (int i = 0; i < 20; i++) {
        linkedBlockingQueue.offer(String.valueOf(i));
    }
    System.out.println(linkedBlockingQueue.size());

    System.out.println("-----poll 方法-----");
    // 3、poll 方法，弹出队顶元素，队列为空时返回null
    for (int i = 0; i < 6; i++) {
        String e = linkedBlockingQueue.poll();
        System.out.println("取出元素: " + e);
    }

    System.out.println("-----peek 方法-----");
    // 4、peek 方法，返回队列顶元素，但顶元素不弹出，队列为空时返回null
    for (int i = 0; i < 5; i++) {
        String e = linkedBlockingQueue.peek();
        System.out.println("取出元素: " + e);
    }

    System.out.println("-----take 方法-----");
    // 5、take 方法，当队列为空，阻塞
    for (int i = 0; i < 10; i++) {
        String e = null;
        try {
            e = linkedBlockingQueue.take();
        } catch (InterruptedException e1) {
            e1.printStackTrace();
        }
        System.out.println("取出元素: " + e);
    }

    System.out.println("-----put 方法-----");
    // 6、put 方法，当队列满时，阻塞
    for (int i = 0; i < 30; i++) {
        try {
            String e = String.valueOf(i);
            linkedBlockingQueue.put(e);
            System.out.println("放入元素: " + e);
        } catch (InterruptedException e1) {
            e1.printStackTrace();
        }
    }
    System.out.println("队列深度: " + linkedBlockingQueue.size());
}
}

```

10.4.2.2 原理

基本结构

```

// 容量,指定容量就是有界队列
private final int capacity;

```

```

// 元素数量
private final AtomicInteger count = new AtomicInteger();
// 链表头 本身是不存储任何元素的，初始化时item指向null
transient Node<E> head;
// 链表尾
private transient Node<E> last;
// take锁 锁分离，提高效率
private final ReentrantLock takeLock = new ReentrantLock();
// notEmpty条件
// 当队列无元素时，take锁会阻塞在notEmpty条件上，等待其它线程唤醒
private final Condition notEmpty = takeLock.newCondition();
// put锁
private final ReentrantLock putLock = new ReentrantLock();
// notFull条件
// 当队列满了时，put锁会阻塞在notFull上，等待其它线程唤醒
private final Condition notFull = putLock.newCondition();

//典型的单链表结构
static class Node<E> {
    E item; //存储元素
    Node<E> next; //后继节点 单链表结构
    Node(E x) { item = x; }
}

```

构造器

```

public LinkedBlockingQueue() {
    // 如果没传容量，就使用最大int值初始化其容量
    this(Integer.MAX_VALUE);
}

public LinkedBlockingQueue(int capacity) {
    if (capacity <= 0) throw new IllegalArgumentException();
    this.capacity = capacity;
    // 初始化head和last指针为空值节点
    last = head = new Node<E>(null);
}

```

入队put方法

```

public void put(E e) throws InterruptedException {
    // 不允许null元素
    if (e == null) throw new NullPointerException();
    int c = -1;
    // 新建一个节点
    Node<E> node = new Node<E>(e);
    final ReentrantLock putLock = this.putLock;
    final AtomicInteger count = this.count;
    // 使用put锁加锁
    putLock.lockInterruptibly();
    try {
        // 如果队列满了，就阻塞在notFull上等待被其它线程唤醒（阻塞生产者线程）
        while (count.get() == capacity) {

```

```

        notFull.await();
    }
    // 队列不满，就入队
    enqueue(node);
    c = count.getAndIncrement();// 队列长度加1，返回原值
    // 如果现队列长度小于容量，notFull条件队列转同步队列，准备唤醒一个阻塞在notFull条件
    上的线程(可以继续入队)
    // 这里为啥要唤醒一下呢？
    // 因为可能有很多线程阻塞在notFull这个条件上，而取元素时只有取之前队列是满的才会唤醒
    notFull,此处不用等到取元素时才唤醒
    if (c + 1 < capacity)
        notFull.signal();
    } finally {
        putLock.unlock(); // 真正唤醒生产者线程
    }
    // 如果原队列长度为0，现在加了一个元素后立即唤醒阻塞在notEmpty上的线程
    if (c == 0)
        signalNotEmpty();
}
private void enqueue(Node<E> node) {
    // 直接加到last后面,last指向入队元素
    last = last.next = node;
}
private void signalNotEmpty() {
    final ReentrantLock takeLock = this.takeLock;
    takeLock.lock();// 加take锁
    try {
        notEmpty.signal();// notEmpty条件队列转同步队列，准备唤醒阻塞在notEmpty上的线程
    } finally {
        takeLock.unlock(); // 真正唤醒消费者线程
    }
}
}

```

出队take方法

```

public E take() throws InterruptedException {
    E x;
    int c = -1;
    final AtomicInteger count = this.count;
    final ReentrantLock takeLock = this.takeLock;
    // 使用takeLock加锁
    takeLock.lockInterruptibly();
    try {
        // 如果队列无元素，则阻塞在notEmpty条件上（消费者线程阻塞）
        while (count.get() == 0) {
            notEmpty.await();
        }
        // 否则，出队
        x = dequeue();
        c = count.getAndDecrement();//长度-1，返回原值
        if (c > 1)// 如果取之前队列长度大于1，notEmpty条件队列转同步队列，准备唤醒阻塞在
        notEmpty上的线程，原因与入队同理
            notEmpty.signal();
    } finally {
        takeLock.unlock(); // 真正唤醒消费者线程
    }
}

```

```

    }
    // 为什么队列是满的才唤醒阻塞在notFull上的线程呢?
    // 因为唤醒是需要加putLock的, 这是为了减少锁的次数, 所以, 这里索性在放完元素就检测一下, 未
    // 满就唤醒其它notFull上的线程,
    // 这也是锁分离带来的代价
    // 如果取之前队列长度等于容量(已满), 则唤醒阻塞在notFull的线程
    if (c == capacity)
        signalNotFull();
    return x;
}
private E dequeue() {
    // head节点本身是不存储任何元素的
    // 这里把head删除, 并把head下一个节点作为新的值
    // 并把其值置空, 返回原来的值
    Node<E> h = head;
    Node<E> first = h.next;
    h.next = h; // 方便GC
    head = first;
    E x = first.item;
    first.item = null;
    return x;
}
private void signalNotFull() {
    final ReentrantLock putLock = this.putLock;
    putLock.lock();
    try {
        notFull.signal(); // notFull条件队列转同步队列, 准备唤醒阻塞在notFull上的线程
    } finally {
        putLock.unlock(); // 解锁, 这才会真正的唤醒生产者线程
    }
}
}

```

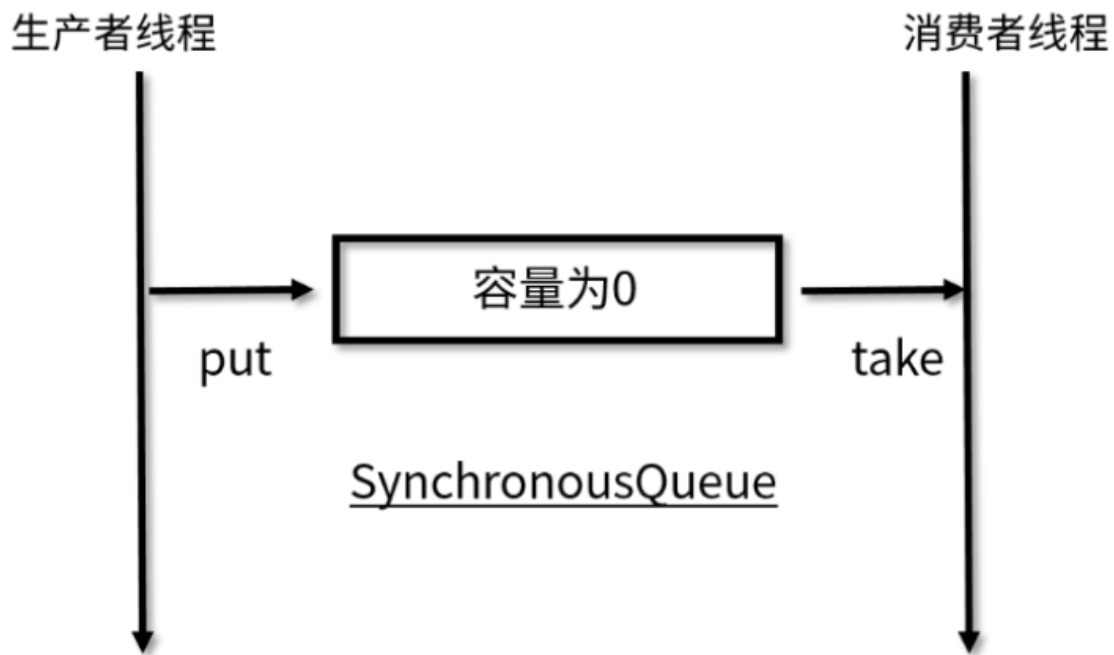
LinkedBlockingQueue与ArrayBlockingQueue对比

LinkedBlockingQueue是一个阻塞队列, 内部由**两个ReentrantLock**来实现出入队列的线程安全, 由各自的Condition对象的await和signal来实现等待和唤醒功能。它和ArrayBlockingQueue的不同点在于:

- 队列大小有所不同, ArrayBlockingQueue是有界的初始化必须指定大小, 而LinkedBlockingQueue可以是有界的也可以是无界的(Integer.MAX_VALUE), 对于后者而言, 当添加速度大于移除速度时, 在无界的情况下, 可能会造成内存溢出等问题。
- 数据存储容器不同, **ArrayBlockingQueue采用的是数组作为数据存储容器, 而LinkedBlockingQueue采用的则是以Node节点作为连接对象的链表。**
- 由于ArrayBlockingQueue采用的是数组的存储容器, 因此在插入或删除元素时不会产生或销毁任何额外的对象实例, 而LinkedBlockingQueue则会生成一个额外的Node对象。这可能在长时间需要高效并发地处理大批量数据的时, 对于GC可能存在较大影响。
- 两者的实现队列添加或移除的锁不一样, **ArrayBlockingQueue实现的队列中的锁是没有分离的, 即添加操作和移除操作采用的同一个ReenterLock锁, 而LinkedBlockingQueue实现的队列中的锁是分离的, 其添加采用的是putLock, 移除采用的则是takeLock**, 这样能大大提高队列的吞吐量, 也意味着在高并发的情况下生产者和消费者可以并行地操作队列中的数据, 以此来提高整个队列的并发性能。

10.4.3 SynchronousQueue

SynchronousQueue是一个没有数据缓冲的BlockingQueue，生产者线程对其的插入操作put必须等待消费者的移除操作take。



如图所示，SynchronousQueue 最大的不同之处在于，**它的容量为 0**，所以没有一个地方来暂存元素，导致**每次取数据都要先阻塞，直到有数据被放入；同理，每次放数据的时候也会阻塞，直到有消费者来取。**

需要注意的是，SynchronousQueue 的容量不是 1 而是 0，**因为 SynchronousQueue 不需要去持有元素，它所做的就是直接传递**（direct handoff）。由于每当需要传递的时候，SynchronousQueue 会把元素直接从生产者传给消费者，在此期间并不需要做存储，所以如果运用得当，它的效率是很高的。

10.4.3.1 适用场景

SynchronousQueue**非常适合传递性场景做交换工作**，生产者的线程和消费者的线程同步传递某些信息、事件或者任务。

SynchronousQueue的一个使用场景是在线程池里。如果我们不确定来自生产者请求数量，但是这些请求需要很快的处理掉，那么配合SynchronousQueue为每个生产者请求分配一个消费线程是处理效率最高的办法。Executors.newCachedThreadPool()就使用了SynchronousQueue，这个线程池根据需要（新任务到来时）创建新的线程，如果有空闲线程则会重复使用，线程空闲了60秒后会被回收。

10.4.3.2 基本使用

```
public class SynchronousQueueDemo {
    public static void main(String[] args) {
        //新建一个SynchronousQueue同步队列
        final BlockingQueue<Integer> synchronousQueue = new SynchronousQueue<>
            (false);

        //启动一个生产者线程插入对象
        SynchronousQueueProducer queueProducer = new
            SynchronousQueueProducer(synchronousQueue);
        new Thread(queueProducer).start();
    }
}
```



```

        //启动两个消费者线程移除对象
        SynchronousQueueConsumer queueConsumer1 = new
SynchronousQueueConsumer(synchronousQueue);
        new Thread(queueConsumer1).start();

        SynchronousQueueConsumer queueConsumer2 = new
SynchronousQueueConsumer(synchronousQueue);
        new Thread(queueConsumer2).start();
    }
}

public class SynchronousQueueProducer implements Runnable {

    protected BlockingQueue<Integer> blockingQueue;
    public SynchronousQueueProducer(BlockingQueue<Integer> queue) {
        this.blockingQueue = queue;
    }

    @Override
    public void run() {
        int i = 0;
        while (true) {
            System.out.println(Thread.currentThread().getName() + " Put: " + ++i);
            try {
                blockingQueue.put(i);
                Thread.sleep(1000); //每隔一秒生产一次
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}

public class SynchronousQueueConsumer implements Runnable {
    protected BlockingQueue<Integer> blockingQueue;

    public SynchronousQueueConsumer(BlockingQueue<Integer> queue) {
        this.blockingQueue = queue;
    }

    @Override
    public void run() {
        while (true) {
            try {
                Integer data = blockingQueue.take();
                System.out.println(Thread.currentThread().getName() + " take(): " +
data);
                Thread.sleep(5000); //每隔5秒消费一次
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}

```

}

10.4.3.3 原理



10.4.4 DelayQueue

DelayQueue 是一个支持延时获取元素的阻塞队列，内部采用优先队列 PriorityQueue 存储元素，同时元素必须实现 Delayed 接口；在创建元素时可以指定多久才可以从队列中获取当前元素，只有在延迟期满时才能从队列中提取元素。延迟队列的特点是：**不是先进先出，而是会按照延迟时间的长短来排序，下一个即将执行的任务会排到队列的最前面。**

它是无界队列，放入的元素必须实现 Delayed 接口，而 Delayed 接口又继承了 Comparable 接口，所以自然就拥有了比较和排序的能力，代码如下：

```
~~~java
public interface Delayed extends Comparable<Delayed> {
    //getDelay 方法返回的是“还剩下多长的延迟时间才会被执行”，
    //如果返回 0 或者负数则代表任务已过期。
    //元素会根据延迟时间的长短被放到队列的不同位置，越靠近队列头代表越早过期。
    long getDelay(TimeUnit unit);
}
~~~
```



10.4.4.1 基本使用

1、简单使用使用延迟队列实现订单简单关闭

```
@Data
public class OrderInfo implements Delayed,Serializable {

    private static final long serialVersionUID = 1L;
    private String orderNo;// 订单号
    private String expTime;// 订单过期时间
    private String createTime;//订单创建时间

    /**
     * 用于延时队列内部比较排序：当前订单的过期时间 与 队列中对象的过期时间 比较
     */
    @Override
    public int compareTo(Delayed o) {
        SimpleDateFormat formatter = new SimpleDateFormat("yyyy-MM-dd
HH:mm:ss");
        long nowThreadtime = 0;
        long queueThreadtime = 0;
        try {
            nowThreadtime = formatter.parse(this.expTime).getTime();
            queueThreadtime = formatter.parse(((OrderInfo)
o).expTime).getTime();
        } catch (ParseException e) {
            e.printStackTrace();
        }
        return
Long.valueOf(nowThreadtime).compareTo(Long.valueOf(queueThreadtime));
    }

    /**
     * 时间单位：秒
     * 延迟关闭时间 = 过期时间 - 当前时间
     */
    @Override
    public long getDelay(TimeUnit unit) {
        SimpleDateFormat formatter = new SimpleDateFormat("yyyy-MM-dd
HH:mm:ss");
        long time = 0;
        try {
            time = formatter.parse(this.expTime).getTime();
        } catch (ParseException e) {
            e.printStackTrace();
        }
        return time - System.currentTimeMillis();
    }
}

public class TestDelayQueueDemo01 {

    private static DelayQueue<OrderInfo> queue = new DelayQueue<OrderInfo>();
```

```

    public static void main(String[] args) throws Exception{
        SimpleDateFormat formatter = new SimpleDateFormat("yyyy-MM-dd
HH:mm:ss");

        OrderInfo orderInfo01 = new OrderInfo();
        orderInfo01.setOrderNo("1111");
        Date now = new Date();
        orderInfo01.setCreateTime(formatter.format(now));
        //设置过期时间 30s后过期
        Calendar c = Calendar.getInstance();
        c.setTimeInMillis(now.getTime() + 30000);

        orderInfo01.setExpTime(formatter.format(c.getTime()));

        queue.put(orderInfo01);
        //2s后启动线程
        TimeUnit.SECONDS.sleep(2);

        new Thread()->{
            while (true) {
                if (queue.size() > 0) {
                    //取出超时订单
                    try {
                        OrderInfo orderInfo = queue.take();
                        System.out.println(orderInfo.getOrderNo() + "已超时，并关
闭");

                    } catch (InterruptedException e) {
                        throw new RuntimeException(e);
                    }
                }
            }
        }.start();
    }
}

```

2、使用延迟队列实现超时订单关闭

```

/**
 * 订单对象
 */
public class OrderInfo implements Delayed,Serializable {

    private static final long serialVersionUID = 1L;
    private String orderNo;// 订单号
    private String status;// 订单状态
    private String expTime;// 订单过期时间
    private String createTime;//订单创建时间

    /**

```

```

    * 用于延时队列内部比较排序：当前订单的过期时间 与 队列中对象的过期时间 比较
    */
    @Override
    public int compareTo(Delayed o) {
        SimpleDateFormat formatter = new SimpleDateFormat("yyyy-MM-dd
HH:mm:ss");
        long nowThreadtime = 0;
        long queueThreadtime = 0;
        try {
            nowThreadtime = formatter.parse(this.expTime).getTime();
            queueThreadtime = formatter.parse(((OrderInfo)
o).expTime).getTime();
        } catch (ParseException e) {
            e.printStackTrace();
        }
        return
Long.valueOf(nowThreadtime).compareTo(Long.valueOf(queueThreadtime));
    }

    /**
     * 时间单位：秒
     * 延迟关闭时间 = 过期时间 - 当前时间
     */
    @Override
    public long getDelay(TimeUnit unit) {
        SimpleDateFormat formatter = new SimpleDateFormat("yyyy-MM-dd
HH:mm:ss");
        long time = 0;
        try {
            time = formatter.parse(this.expTime).getTime();
        } catch (ParseException e) {
            e.printStackTrace();
        }
        return time - System.currentTimeMillis();
    }

    public String getOrderNo() {
        return orderNo;
    }

    public void setOrderNo(String orderNo) {
        this.orderNo = orderNo;
    }

    public String getExpTime() {
        return expTime;
    }

    public void setExpTime(String overTime) {
        this.expTime = overTime;
    }

    public String getStatus() {
        return status;
    }

```

```

    public void setStatus(String status) {
        this.status = status;
    }

    public String getCreateTime() {
        return createTime;
    }

    public void setCreateTime(String createTime) {
        this.createTime = createTime;
    }
}

/**
 * 使用延时队列DelayQueue实现订单超时关闭
 * 后台守护线程不断的执行检测工作
 *
 */

public class OrderOverTimeClose {

    private volatile static OrderOverTimeClose oderOverTimeClose = null;

    private OrderOverTimeClose() {

    }

    /**
     * 单例模式，双检查锁模式，在并发环境下对象只被初始化一次
     */
    public static OrderOverTimeClose getInstance() {
        if (oderOverTimeClose == null) {
            synchronized (OrderOverTimeClose.class) {
                if (oderOverTimeClose == null) {
                    oderOverTimeClose = new OrderOverTimeClose();
                }
            }
        }
        return oderOverTimeClose;
    }

    /**
     * 守护线程
     */
    private Thread mainThread;

    /**
     * 启动方法
     */
    public void init() {
        mainThread = new Thread(() -> execute());
        mainThread.setDaemon(true);
        mainThread.setName("守护线程-->");
    }
}

```

```

        mainThread.start();
    }

    /**
     * 创建空延时队列
     */
    private DelayQueue<OrderInfo> queue = new DelayQueue<OrderInfo>();

    /**
     * 读取延时队列，关闭超时订单
     */
    private void execute() {
        while (true) {
            try {
                if (queue.size() > 0) {
                    //从队列里获取超时的订单
                    OrderInfo orderInfo = queue.take();
                    // 检查订单状态，是否已经成功，成功则将订单从队列中删除。
                    if (Objects.equals(orderInfo.getStatus(), "成功")) {
                        //TODO 支付成功的订单处理逻辑
                        System.out.println("线程: " +
Thread.currentThread().getName() + ", 订单号: "
                                + orderInfo.getOrderNo() + ", 订单状态: "
                                + orderInfo.getStatus() + ", 订单创建时间: "
                                + orderInfo.getCreateTime()
                                + ", 订单超时时间: " + orderInfo.getExpTime() + ",
当前时间: "
                                + OrderPay.getTime(System.currentTimeMillis()));
                        Thread.sleep(2000);
                    } else {
                        // TODO 超时未支付订单处理逻辑
                        System.out.println("线程: " +
Thread.currentThread().getName() + ", 订单号: "
                                + orderInfo.getOrderNo() + ", 变更订单状态为: 超时关
闭"
                                + ", 订单创建时间: "
                                + orderInfo.getCreateTime()
                                + ", 订单超时时间: " + orderInfo.getExpTime() + ",
当前时间: "
                                + OrderPay.getTime(System.currentTimeMillis()));
                        Thread.sleep(2000);
                    }
                }
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }

    /**
     * 插入订单到延时队列中
     */
    public void orderPutQueue(OrderInfo orderInfo, String createTime,
String overTime) {
        System.out.println("订单号: " + orderInfo.getOrderNo()

```

```

        +", 订单状态: "+ orderInfo.getStatus()
        +", 订单创建时间: "+ createTime
        + ", 订单过期时间: " + overTime);
    queue.add(orderInfo);
}

}

/**
 * 模拟订单支付, 创建订单
 */
public class OrderPay {

    static String[] str = new String[]{ "成功", "支付中", "订单初始化" };

    public static String getTime(long time) {
        SimpleDateFormat formatter = new SimpleDateFormat("yyyy-MM-dd
HH:mm:ss");
        Date date = new Date(time);
        String currentTime = formatter.format(date);
        return currentTime;
    }

    public static void main(String[] args) throws InterruptedException {
        OrderOverTimeClose.getInstance().init();

        ExecutorService service = Executors.newFixedThreadPool(10);
        for (int i = 0; i < 20; i++) {
            Runnable run = new Runnable() {
                @Override
                public void run() {
                    // 创建初始订单
                    long createTime = System.currentTimeMillis();
                    String currentTime = getTime(createTime);
                    //设置订单过期时间
                    String overTime = getTime(createTime + 10000); // 10s后超时
                    String orderNo = String.valueOf(new Random().nextLong());
                    OrderInfo order = new OrderInfo();
                    order.setOrderNo(orderNo);
                    order.setExpTime(overTime);
                    int random_index = (int) (Math.random() * str.length);
                    //设置订单状态
                    order.setStatus(str[random_index]);
                    order.setCreateTime(currentTime);
                    //插入订单到延时队列中
                    OrderOverTimeClose.getInstance().orderPutQueue(order,
currentTime, overTime);
                }
            };
            service.execute(run);
            try {
                Thread.sleep(3000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}

```



```

    }
}
}
}

```

3、使用延迟队列实现缓存过期清除

```

public class DelayQueueDemo {

    static class Cache implements Runnable {

        private boolean stop = false;

        private Map<String, String> itemMap = new HashMap<>();

        private DelayQueue<CacheItem> delayQueue = new DelayQueue<>();

        public Cache () {
            // 开启内部线程检测是否过期
            new Thread(this).start();
        }

        /**
         * 添加缓存
         *
         * @param key
         * @param value
         * @param exprieTime&emsp;过期时间,单位秒
         */
        public void put (String key, String value, long exprieTime) {
            CacheItem cacheItem = new CacheItem(key, exprieTime);

            // 此处忽略添加重复 key 的处理
            delayQueue.add(cacheItem);
            itemMap.put(key, value);
        }

        public String get (String key) {
            return itemMap.get(key);
        }

        public void shutdown () {
            stop = true;
        }

        @Override
        public void run() {
            while (!stop) {
                CacheItem cacheItem = delayQueue.poll();
                if (cacheItem != null) {
                    // 元素过期，从缓存中移除
                    itemMap.remove(cacheItem.getKey());
                }
            }
        }
    }
}

```

```

        System.out.println("key : " + cacheItem.getKey() + " 过期并移
除");
    }
}

    System.out.println("cache stop");
}
}

static class CacheItem implements Delayed {

    private String key;

    /**
     * 过期时间(单位秒)
     */
    private long exprieTime;

    private long currentTime;

    public CacheItem(String key, long exprieTime) {
        this.key = key;
        this.exprieTime = exprieTime;
        this.currentTime = System.currentTimeMillis();
    }

    @Override
    public long getDelay(TimeUnit unit) {
        // 计算剩余的过期时间
        // 大于 0 说明未过期
        return exprieTime -
TimeUnit.MILLISECONDS.toSeconds(System.currentTimeMillis() - currentTime);
    }

    @Override
    public int compareTo(Delayed o) {
        // 过期时间长的放置在队列尾部
        if (this.getDelay(TimeUnit.MICROSECONDS) >
o.getDelay(TimeUnit.MICROSECONDS)) {
            return 1;
        }
        // 过期时间短的放置在队列头
        if (this.getDelay(TimeUnit.MICROSECONDS) <
o.getDelay(TimeUnit.MICROSECONDS)) {
            return -1;
        }

        return 0;
    }

    public String getKey() {
        return key;
    }
}
}

```

```

public static void main(String[] args) throws InterruptedException {

    Cache cache = new Cache();

    // 添加缓存元素
    cache.put("a", "1", 5);
    cache.put("b", "2", 4);
    cache.put("c", "3", 3);

    while (true) {
        String a = cache.get("a");
        String b = cache.get("b");
        String c = cache.get("c");

        System.out.println("a : " + a + ", b : " + b + ", c : " + c);

        // 元素均过期后退出循环
        if (StringUtils.isEmpty(a) && StringUtils.isEmpty(b) &&
            StringUtils.isEmpty(c)) {
            break;
        }

        TimeUnit.MILLISECONDS.sleep(1000);
    }

    cache.shutdown();
}
}

```

10.4.4.2 原理

```

//用于保证队列操作的线程安全
private final transient ReentrantLock lock = new ReentrantLock();
// 优先级队列,存储元素, 用于保证延迟低的优先执行
private final PriorityQueue<E> q = new PriorityQueue<E>();
// 用于标记当前是否有线程在排队 (仅用于取元素时) leader 指向的是第一个从队列获取元素阻塞的线程
private Thread leader = null;
// 条件, 用于表示现在是否有可取的元素 当新元素到达, 或新线程可能需要成为leader时被通知
private final Condition available = lock.newCondition();

public DelayQueue() {}
public DelayQueue(Collection<? extends E> c) {
    this.addAll(c);
}

```

入队put方法

```

public void put(E e) {
    offer(e);
}

```

```

public boolean offer(E e) {
    final ReentrantLock lock = this.lock;
    lock.lock();
    try {
        // 入队
        q.offer(e);
        if (q.peek() == e) {
            // 若入队的元素位于队列头部，说明当前元素延迟最小
            // 将 leader 置空
            leader = null;
            // available条件队列转同步队列，准备唤醒阻塞在available上的线程
            available.signal();
        }
        return true;
    } finally {
        lock.unlock(); // 解锁，真正唤醒阻塞的线程
    }
}

```

出队take方法

```

public E take() throws InterruptedException {
    final ReentrantLock lock = this.lock;
    lock.lockInterruptibly();
    try {
        for (;;) {
            E first = q.peek(); // 取出堆顶元素
            if (first == null) // 如果堆顶元素为空，说明队列中还没有元素，直接阻塞等待
                available.await();
            else {
                long delay = first.getDelay(NANOSECONDS); // 堆顶元素的到期时间

                if (delay <= 0) // 如果小于0说明已到期，直接调用poll()方法弹出堆顶元素
                    return q.poll();

                // 如果delay大于0，则下面要阻塞了
                // 将first置为空方便gc
                first = null;
                // 如果前面有其它线程在等待，直接进入等待
                if (leader != null)
                    available.await();
                else {
                    // 如果leader为null，把当前线程赋值给它
                    Thread thisThread = Thread.currentThread();
                    leader = thisThread;
                    try {
                        // 等待delay时间后自动醒过来
                        // 醒过来后把leader置空并重新进入循环判断堆顶元素是否到期
                        // 这里即使醒过来后也不一定能获取到元素
                        // 因为有可能其它线程先一步获取了锁并弹出了堆顶元素
                        // 条件锁的唤醒分成两步，先从Condition的队列里出队
                        // 再入队到AQS的队列中，当其它线程调用LockSupport.unpark(t)的时
候才会真正唤醒

                        available.awaitNanos(delay);
                    } finally {

```

```

        // 如果leader还是当前线程就把它置为空，让其它线程有机会获取元素
        if (leader == thisThread)
            leader = null;
    }
}
}
}
} finally {
    // 成功出队后，如果leader为空且堆顶还有元素，就唤醒下一个等待的线程
    if (leader == null && q.peek() != null)
        // available条件队列转同步队列，准备唤醒阻塞在available上的线程
        available.signal();
    // 解锁，真正唤醒阻塞的线程
    lock.unlock();
}
}

```

10.4.5 PriorityBlockingQueue

PriorityBlockingQueue是一个无界的基于数组的优先级阻塞队列，数组的默认长度是11，虽然指定了数组的长度，但是可以无限的扩充，直到资源消耗尽为止，每次出队都返回优先级别最高的或者最低的元素。默认情况下元素采用自然顺序升序排序，当然我们也可以通过构造函数来指定Comparator来对元素进行排序。需要注意的是PriorityBlockingQueue不能保证同优先级元素的顺序。

优先级队列PriorityQueue：队列中每个元素都有一个优先级，出队的时候，优先级最高的先出。



10.4.5.1 基本使用

```

public class PriorityBlockingQueueDemo {

    public static void main(String[] args) throws InterruptedException {
        // 创建优先级阻塞队列 Comparator为null, 自然排序
        PriorityBlockingQueue<Integer> queue = new PriorityBlockingQueue<Integer>
(5);

        // 自定义Comparator
        // PriorityBlockingQueue queue = new PriorityBlockingQueue<Integer>(
        //     5, new Comparator<Integer>() {
        //         @Override
        //         public int compare(Integer o1, Integer o2) {
        //             return o2 - o1;
        //         }
        //     });
    }
}

```

```

Random random = new Random();
System.out.println("put:");
for (int i = 0; i < 5; i++) {
    int j = random.nextInt(100);
    System.out.print(j+" ");
    queue.put(j);
}

System.out.println("\ntake:");
for (int i = 0; i < 5; i++) {
    System.out.print(queue.take()+" ");
}

}
}

```

10.4.5.2 原理

10.4.6 LinkedTransferQueue



10.4.6.1 基本使用

```

public class LinkedTransferQueueDemo {

    public static void main(String[] args) {
        TransferQueue<String> queue = new LinkedTransferQueue<String>();
        Thread producer = new Thread(new LinkedTransferQueueProducer(queue));
        producer.setDaemon(true);
        producer.start();

        for (int i = 0; i < 10; i++) {
            Thread consumer = new Thread(new
LinkedTransferQueueConsumer(queue));
            consumer.setDaemon(true);
            consumer.start();

            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}

```

```

    }

}

public class LinkedTransferQueueProducer implements Runnable {
    private final TransferQueue<String> queue;
    public LinkedTransferQueueProducer(TransferQueue<String> queue) {
        this.queue = queue;
    }

    private String produce() {
        return "Your lucky number " + (new Random().nextInt(100));
    }
    @Override
    public void run() {
        try {
            while (true) {
                if (queue.hasWaitingConsumer()) {
                    queue.transfer(produce());
                }
                TimeUnit.SECONDS.sleep(1);
            }
        } catch (InterruptedException e){

        }

    }

}

public class LinkedTransferQueueConsumer implements Runnable{
    private final TransferQueue<String> queue;
    public LinkedTransferQueueConsumer(TransferQueue<String> queue) {
        this.queue = queue;
    }
    @Override
    public void run() {
        try {
            System.out.println("Consumer " + Thread.currentThread().getName() +
queue.take());

        } catch (InterruptedException e) {
            e.printStackTrace();
        }

    }

}

```

10.4.6.2 原理



10.4.7 LinkedBlockingDeque



11、Future&CompletableFuture

11.1 Callable&Future&FutureTask

11.1.1 Callable&Future&FutureTask介绍

直接继承Thread或者实现Runnable接口都可以创建线程, 但是这两种方法都有一个问题就是: 没有返回值, 也就是不能获取执行完的结果。因此java1.5就提供了Callable接口来实现这一场景, 而Future和FutureTask就可以和Callable接口配合起来使用。

Callable和Runnable的区别

```
@FunctionalInterface
public interface Runnable {
    public abstract void run();
}
@FunctionalInterface
public interface Callable<V> {
    V call() throws Exception;
}
```

Runnable 的缺陷:

- 不能返回一个返回值
- 不能抛出 checked Exception

Callable的call方法可以有返回值, 可以声明抛出异常。

和 Callable 配合的有一个 Future 类, 通过 Future 可以了解任务执行情况, 或者取消任务的执行, 还可获取任务执行的结果, 这些功能都是 Runnable 做不到的, Callable 的功能要比 Runnable 强大。

```
@Slf4j
public class FutureDemo {

    public static void main(String[] args) throws ExecutionException,
        InterruptedException {
```



```

        new Thread(() -> {
            log.debug("通过Runnable方式执行任务");
        }).start();

//        new Thread(new Runnable() {
//            @Override
//            public void run() {
//
//            }
//        }).start();

        FutureTask task = new FutureTask(new Callable() {
            @Override
            public Object call() throws Exception {
                log.debug("通过Callable方式执行任务");
                Thread.sleep(3000);
                return "返回任务结果";
            }
        });

        new Thread(task).start();
        log.debug("结果: {}", task.get());

    }
}

```

11.1.2 Future

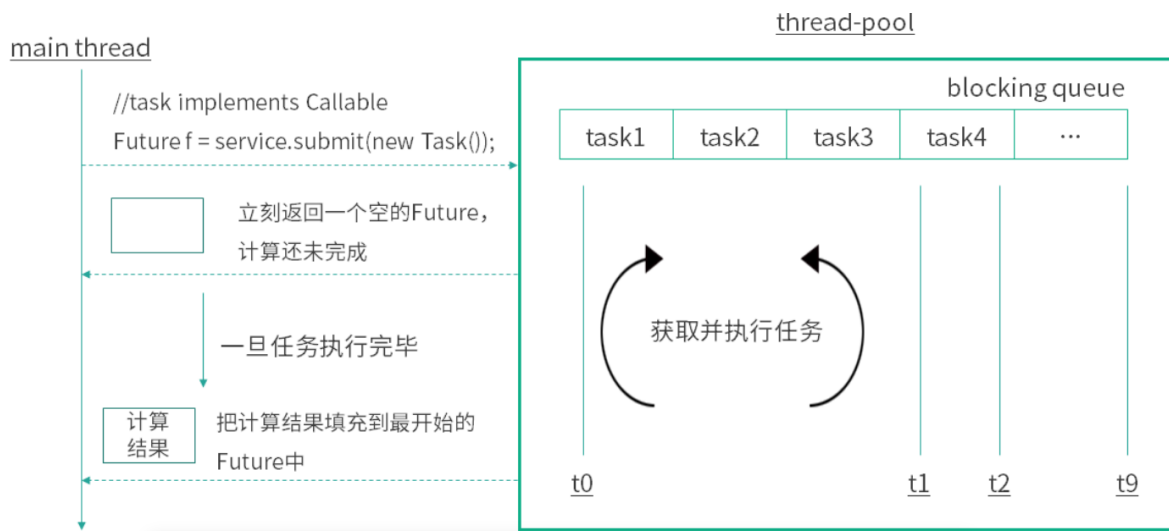
Future 的主要功能

Future就是对于具体的Runnable或者Callable任务的执行结果进行取消、查询是否完成、获取结果。必要时可以通过get方法获取执行结果，该方法会阻塞直到任务返回结果。

```

boolean cancel (boolean mayInterruptIfRunning) //取消任务的执行。参数指定是否立即中断任务执行，或者等等任务结束
boolean isCancelled () //任务是否已经取消，任务正常完成前将其取消，则返回 true
boolean isDone () //任务是否已经完成。需要注意的是如果任务正常终止、异常或取消，都将返回true
V get () throws InterruptedException, ExecutionException //等待任务执行结束，然后获得V类型的结果。InterruptedException 线程被中断异常， ExecutionException任务执行异常，如果任务被取消，还会抛出CancellationException
V get (long timeout, TimeUnit unit) throws InterruptedException, ExecutionException, TimeoutException//同上面的get功能一样，多了设置超时时间。参数 timeout指定超时时间， unit指定时间的单位，在枚举类TimeUnit中有相关的定义。如果计算超时，将抛出 TimeoutException

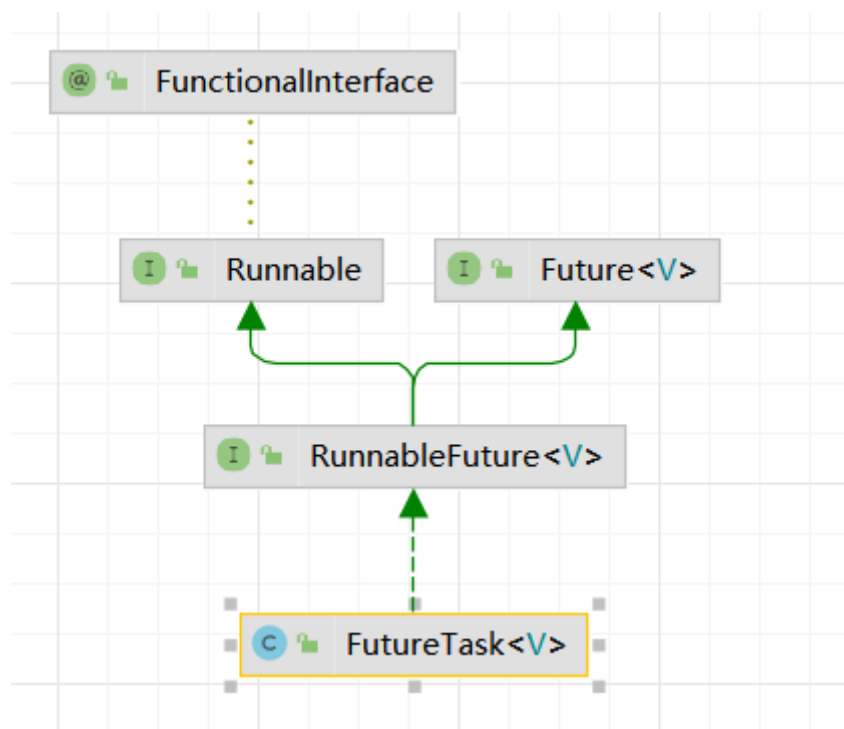
```



11.1.3 利用 FutureTask 创建 Future

Future实际采用**FutureTask**实现，该对象相当于是消费者和生产者的桥梁，消费者通过 `FutureTask` 存储任务的处理结果，更新任务的状态：未开始、正在处理、已完成等。而生产者拿到的 `FutureTask` 被转型为 `Future` 接口，可以阻塞式获取任务的处理结果，非阻塞式获取任务处理状态。

`FutureTask`既可以被当做`Runnable`来执行，也可以被当做`Future`来获取`Callable`的返回结果。



使用:

把 `Callable` 实例当作 `FutureTask` 构造函数的参数，生成 `FutureTask` 的对象，然后把这个对象当作一个 `Runnable` 对象，放到线程池中或另起线程去执行，最后还可以通过 `FutureTask` 获取任务执行的结果。

```

public class FutureTaskDemo {

    public static void main(String[] args) throws ExecutionException,
        InterruptedException {
        Task task = new Task();
        //构建futureTask
        FutureTask<Integer> futureTask = new FutureTask<>(task);
        //作为Runnable入参
    }
}
  
```

```

        new Thread(futureTask).start();

        System.out.println("task运行结果: "+futureTask.get());
    }

    static class Task implements Callable<Integer> {

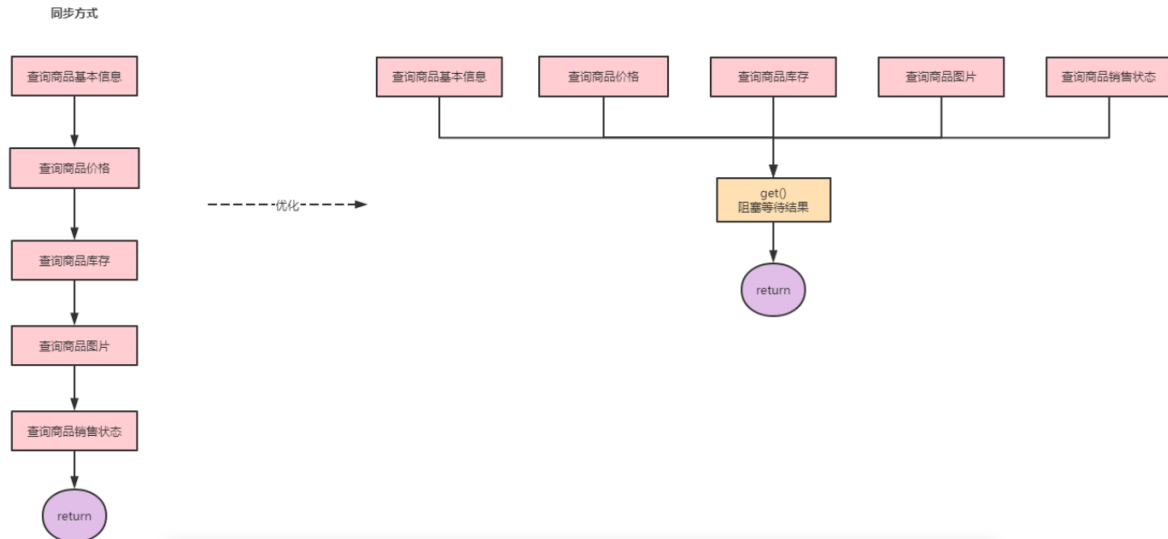
        @Override
        public Integer call() throws Exception {
            System.out.println("子线程正在计算");
            int sum = 0;
            for (int i = 0; i < 100; i++) {
                sum += i;
            }
            return sum;
        }
    }
}

```

11.1.4 使用案例

促销活动中商品信息查询

在维护促销活动时需要查询商品信息（包括商品基本信息、商品价格、商品库存、商品图片、商品销售状态等）。这些信息分布在不同的业务中心，由不同的系统提供服务。如果采用同步方式，假设一个接口需要50ms，那么一个商品查询下来就需要200ms-300ms，这对于我们来说是不满意的。如果使用Future改造则需要的就是最长耗时服务的接口，也就是50ms左右。



```

public class FutureTaskDemo2 {

    public static void main(String[] args) throws ExecutionException,
        InterruptedException {

        FutureTask<String> ft1 = new FutureTask<>(new T1Task());
        FutureTask<String> ft2 = new FutureTask<>(new T2Task());
        FutureTask<String> ft3 = new FutureTask<>(new T3Task());
        FutureTask<String> ft4 = new FutureTask<>(new T4Task());
        FutureTask<String> ft5 = new FutureTask<>(new T5Task());

```

//构建线程池

```
ExecutorService executorService = Executors.newFixedThreadPool(5);
executorService.submit(ft1);
executorService.submit(ft2);
executorService.submit(ft3);
executorService.submit(ft4);
executorService.submit(ft5);
//获取执行结果
System.out.println(ft1.get());
System.out.println(ft2.get());
System.out.println(ft3.get());
System.out.println(ft4.get());
System.out.println(ft5.get());

executorService.shutdown();

}

static class T1Task implements Callable<String> {
    @Override
    public String call() throws Exception {
        System.out.println("T1:查询商品基本信息...");
        TimeUnit.MILLISECONDS.sleep(5000);
        return "商品基本信息查询成功";
    }
}

static class T2Task implements Callable<String> {
    @Override
    public String call() throws Exception {
        System.out.println("T2:查询商品价格...");
        TimeUnit.MILLISECONDS.sleep(50);
        return "商品价格查询成功";
    }
}

static class T3Task implements Callable<String> {
    @Override
    public String call() throws Exception {
        System.out.println("T3:查询商品库存...");
        TimeUnit.MILLISECONDS.sleep(50);
        return "商品库存查询成功";
    }
}

static class T4Task implements Callable<String> {
    @Override
    public String call() throws Exception {
        System.out.println("T4:查询商品图片...");
        TimeUnit.MILLISECONDS.sleep(50);
        return "商品图片查询成功";
    }
}

static class T5Task implements Callable<String> {
    @Override
```

```

        public String call() throws Exception {
            System.out.println("T5:查询商品销售状态...");
            TimeUnit.MILLISECONDS.sleep(50);
            return "商品销售状态查询成功";
        }
    }
}

```

Future 注意事项

- 当 for 循环批量获取 Future 的结果时容易 block, **get 方法调用时应使用 timeout 限制**
- Future 的生命周期不能后退。一旦完成了任务, 它就永久停在了“已完成”的状态, 不能从头再来

11.1.5 Future的局限性

从本质上说, **Future表示一个异步计算的结果**。它提供了isDone()来检测计算是否已经完成, 并且在计算结束后, 可以通过get()方法来获取计算结果。在异步计算中, Future确实是个非常优秀的接口。但是, 它的本身也确实存在着许多限制:

- **并发执行多任务**: Future只提供了get()方法来获取结果, 并且是阻塞的。所以, 除了等待你别无他法;
- **无法对多个任务进行链式调用**: 如果你希望在计算任务完成后执行特定动作, 比如发邮件, 但Future却没有提供这样的能力;
- **无法组合多个任务**: 如果你运行了10个任务, 并期望在它们全部执行结束后执行特定动作, 那么在Future中这是无能为力的;
- **没有异常处理**: Future接口中没有关于异常处理的方法

11.2 CompletionService

Callable+Future 可以实现多个task并行执行, 但是如果遇到前面的task执行较慢时需要阻塞等待前面的task执行完后才能取得结果。而**CompletionService的主要功能就是一边生成任务, 一边获取任务的返回值。让两件事分开执行, 任务之间不会互相阻塞, 可以实现先执行完的先取结果, 不再依赖任务顺序了。**

CompletionService

  poll(): Future<V>

  poll(long, TimeUnit): Future<V>

  submit(Callable<V>): Future<V>

  submit(Runnable, V): Future<V>

  take(): Future<V>

11.2.1 CompletionService原理

内部通过阻塞队列+FutureTask，实现了任务先完成可优先获取到，即结果按照完成先后顺序排序，内部有一个先进先出的阻塞队列，用于保存已经执行完成的Future，通过调用它的take方法或poll方法可以获取到一个已经执行完成的Future，进而通过调用Future接口实现类的get方法获取最终的结果

11.2.2 案例

询价应用：向不同电商平台询价，并保存价格

11.2.2.1 采用“ThreadPoolExecutor+Future”的方案：同步执行询价然后再保存

```
@Slf4j
public class FutureTaskDemo4 {

    public static void main(String[] args) throws Exception {

        //创建线程池
        ExecutorService executor = Executors.newFixedThreadPool(10);
        //异步向电商S1询价
        Future<Integer> f1 = executor.submit(() -> getPriceByS1());
        //异步向电商S2询价
        Future<Integer> f2 = executor.submit(() -> getPriceByS2());
        //异步向电商S3询价
        Future<Integer> f3 = executor.submit(() -> getPriceByS3());

        // 获取电商S1报价并保存
        save(f1.get());
        // 获取电商S2报价并保存
        save(f2.get());
        // 获取电商S3报价并保存
        save(f3.get());

        executor.shutdown();
    }

    private static void save(Integer r) {
        log.debug("保存询价结果:{}", r);
    }

    private static Integer getPriceByS1() throws InterruptedException {
        TimeUnit.MILLISECONDS.sleep(5000);
        log.debug("电商S1询价信息1200");
        return 1200;
    }

    private static Integer getPriceByS2() throws InterruptedException {
        TimeUnit.MILLISECONDS.sleep(8000);
        log.debug("电商S2询价信息1000");
        return 1000;
    }

    private static Integer getPriceByS3() throws InterruptedException {
        TimeUnit.MILLISECONDS.sleep(3000);
        log.debug("电商S3询价信息800");
        return 800;
    }
}
```

```
}  
}
```

如果获取电商S1报价的耗时很长，那么即便获取电商S2报价的耗时很短，也无法让保存S2报价的操作先执行，因为这个主线程都阻塞在了f1.get()操作上。

11.2.2.2 采用“ThreadPoolExecutor+Future”的方案：异步执行询价然后再保存

```
/**  
 * 电商询价业务  
 */  
@Slf4j  
public class FutureTaskDemo5 {  
  
    public static void main(String[] args) throws Exception {  
  
        //创建线程池  
        ExecutorService executor = Executors.newFixedThreadPool(10);  
        //异步向电商S1询价  
        Future<Integer> f1 = executor.submit(() -> getPriceByS1());  
        //异步向电商S2询价  
        Future<Integer> f2 = executor.submit(() -> getPriceByS2());  
        //异步向电商S3询价  
        Future<Integer> f3 = executor.submit(() -> getPriceByS3());  
  
        // 获取电商S1报价并异步保存  
        executor.execute(() -> {  
            try {  
                save(f1.get());  
            } catch (InterruptedException e) {  
                throw new RuntimeException(e);  
            } catch (ExecutionException e) {  
                throw new RuntimeException(e);  
            }  
        });  
        // 获取电商S2报价并异步保存  
        executor.execute(() -> {  
            try {  
                save(f2.get());  
            } catch (InterruptedException e) {  
                throw new RuntimeException(e);  
            } catch (ExecutionException e) {  
                throw new RuntimeException(e);  
            }  
        });  
        // 获取电商S3报价并异步保存  
        executor.execute(() -> {  
            try {  
                save(f3.get());  
            } catch (InterruptedException e) {  
                throw new RuntimeException(e);  
            } catch (ExecutionException e) {  
                throw new RuntimeException(e);  
            }  
        });  
    }  
}
```

```

        executor.shutdown();
    }

    private static void save(Integer r) {
        log.debug("保存询价结果:{}", r);
    }

    private static Integer getPriceByS1() throws InterruptedException {
        TimeUnit.MILLISECONDS.sleep(5000);
        log.debug("电商S1询价信息1200");
        return 1200;
    }

    private static Integer getPriceByS2() throws InterruptedException {
        TimeUnit.MILLISECONDS.sleep(8000);
        log.debug("电商S2询价信息1000");
        return 1000;
    }

    private static Integer getPriceByS3() throws InterruptedException {
        TimeUnit.MILLISECONDS.sleep(3000);
        log.debug("电商S3询价信息800");
        return 800;
    }
}

```

11.2.2.3 使用CompletionService实现先获取的报价先保存到数据库

```

@Slf4j
public class CompletionServiceDemo {

    public static void main(String[] args) throws InterruptedException,
    ExecutionException {

        //创建线程池
        ExecutorService executor = Executors.newFixedThreadPool(10);
        //创建CompletionService
        CompletionService<Integer> cs = new ExecutorCompletionService<>
        (executor);
        //异步向电商S1询价
        cs.submit(() -> getPriceByS1());
        //异步向电商S2询价
        cs.submit(() -> getPriceByS2());
        //异步向电商S3询价
        cs.submit(() -> getPriceByS3());
        //将询价结果异步保存到数据库
        for (int i = 0; i < 3; i++) {
            //从阻塞队列获取futureTask
            Integer r = cs.take().get();
            executor.execute(() -> save(r));
        }
    }
}

```



```

        executor.shutdown();
    }

    private static void save(Integer r) {
        log.debug("保存询价结果:{}", r);
    }

    private static Integer getPriceByS1() throws InterruptedException {
        TimeUnit.MILLISECONDS.sleep(5000);
        log.debug("电商S1询价信息1200");
        return 1200;
    }

    private static Integer getPriceByS2() throws InterruptedException {
        TimeUnit.MILLISECONDS.sleep(8000);
        log.debug("电商S2询价信息1000");
        return 1000;
    }

    private static Integer getPriceByS3() throws InterruptedException {
        TimeUnit.MILLISECONDS.sleep(3000);
        log.debug("电商S3询价信息800");
        return 800;
    }
}

```

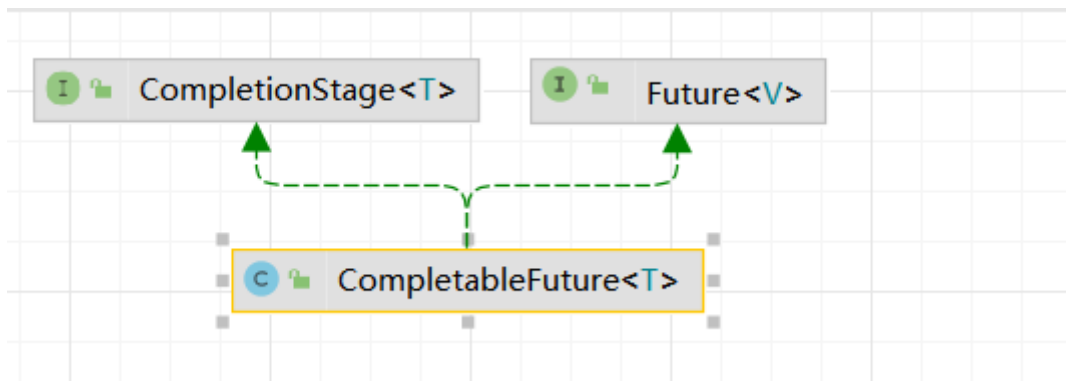
应用场景总结

- 当需要批量提交异步任务的时候建议你使用CompletionService。CompletionService将线程池Executor和阻塞队列BlockingQueue的功能融合在了一起，能够让批量异步任务的管理更简单。
- CompletionService能够让异步任务的执行结果有序化。先执行完的先进入阻塞队列，利用这个特性，你可以轻松实现后续处理的有序性，避免无谓的等待。
- 线程池隔离。CompletionService支持自己创建线程池，这种隔离性能避免几个特别耗时的任务拖垮整个应用的风险。

11.3 CompletableFuture

简单的任务，用Future获取结果还好，但我们并行提交的多个异步任务，往往并不是独立的，**很多时候业务逻辑处理存在串行[依赖]、并行、聚合的关系**。如果要我们手动用Future实现，是非常麻烦的。

CompletableFuture是Future接口的扩展和增强。CompletableFuture实现了Future接口，并在此基础上进行了丰富地扩展，完美地弥补了Future上述的种种问题。更为重要的是，**CompletableFuture实现了对任务的编排能力**。借助这项能力，我们可以轻松地组织不同任务的运行顺序、规则以及方式。从某种程度上说，这项能力是它的核心能力。而在以往，虽然通过CountDownLatch等工具类也可以实现任务的编排，但需要复杂的逻辑处理，不仅耗费精力且难以维护。



CompletionStage接口: 执行某一个阶段，可向下执行后续阶段。异步执行，默认线程池是 ForkJoinPool.commonPool()

应用场景

描述依赖关系:

1. thenApply() 把前面异步任务的结果，交给后面的Function
2. thenCompose()用来连接两个有依赖关系的任务，结果由第二个任务返回

描述and聚合关系:

1. thenCombine:任务合并，有返回值
2. thenAcceptBoth:两个任务执行完成后，将结果交给thenAcceptBoth消耗，无返回值。
3. runAfterBoth:两个任务都执行完成后，执行下一步操作（Runnable）。

描述or聚合关系:

1. applyToEither:两个任务谁执行的快，就使用那一个结果，有返回值。
2. acceptEither: 两个任务谁执行的快，就消耗那一个结果，无返回值。
3. runAfterEither: 任意一个任务执行完成，进行下一步操作(Runnable)。

并行执行:

CompletableFuture类自己也提供了anyOf()和allOf()用于支持多个CompletableFuture并行执行

创建异步操作

CompletableFuture 提供了四个静态方法来创建一个异步操作

```
public static CompletableFuture<Void> runAsync(Runnable runnable)
public static CompletableFuture<Void> runAsync(Runnable runnable, Executor executor)
public static <U> CompletableFuture<U> supplyAsync(Supplier<U> supplier)
public static <U> CompletableFuture<U> supplyAsync(Supplier<U> supplier,
Executor executor)
```

这四个方法区别在于:

- runAsync 方法以Runnable函数式接口类型为参数，**没有返回结果**，supplyAsync 方法Supplier函数式接口类型为参数，返回结果类型为U；Supplier 接口的 get() 方法是有返回值的（会阻塞）
- 没有指定Executor的方法会使用ForkJoinPool.commonPool() 作为它的线程池执行异步代码。如果指定线程池，则使用指定的线程池运行。
- 默认情况下 CompletableFuture 会使用公共的 ForkJoinPool 线程池，这个线程池默认创建的线程数是 CPU 的核数（也可以通过JVM option:-

Djava.util.concurrent.ForkJoinPool.common.parallelism 来设置 ForkJoinPool 线程池的线程数)。如果所有 CompletableFuture 共享一个线程池，那么一旦有任务执行一些很慢的 I/O 操作，就会导致线程池中所有线程都阻塞在 I/O 操作上，从而造成线程饥饿，进而影响整个系统的性能。所以，**强烈建议你根据不同的业务类型创建不同的线程池，以避免互相干扰**

分类	方法	说明	返回值
异步执行一个线程	CompletableFuture.runAsync(Runnable)	默认使用ForkJoinPool.commonPool线程池	runAsync返回CompletableFuture<Void>:无返回值
	CompletableFuture.supplyAsync(Supplier)	也可指定Executor参数指定线程池	supplyAsync返回CompletableFuture<T>:有返回值
	调用get(), join()返回阻塞等待线程结束		
	thenApply	thenApplySync:可指定线程池。其他方法类似	有返回值
	thenAccept	获取前一个线程的结果。消费。	无返回值
	thenRun	忽略前一个线程的结果。执行额外的逻辑	无返回值
	whenComplete	获取前一个线程的结果或异常。消费。	不影响上一线程返回值
	exceptionally	前面线程异常时。执行。一般跟whenComplete配合使用。捕获异常范围跟包括前面所有异步线程如thenApply().thenAccept().exceptionally()	有返回值
2个线程依次执行	handle	相当于whenComplete+exceptionally 根据是否产生异常内部if else分支处理业务逻辑	有返回值
	thenCombine	2个线程都要有返回值。等待都结束。结果合并转换。	有返回值
等待2个线程都执行完	thenAcceptBoth	2个线程都要有返回值。等待都结束。结果合并消费。	无返回值
	runAfterBoth	2个线程不需要有返回值。等待都结束。执行其他逻辑。	无返回值
等待2个线程任一执行完	applyToEither	2个线程都要有返回值。等待任一先结束。转换其结果。	有返回值
	acceptToEither	2个线程都要有返回值。等待任一先结束。消费其结果。	无返回值
多个线程等待	runAfterEither	2个线程不需要有返回值。等待任一结束。执行其他逻辑。	无返回值
	CompletableFuture.anyOf(cf1.cf2.cf3).join()	多个线程任一执行完即返回。	有返回值 Object
	CompletableFuture.allOf(cf1.cf2.cf3).join()	多个线程全部执行完返回。	无返回值

11.3.1 runAsync&supplyAsync

```
public class CompletableFutureDemo01 {

    public static void main(String[] args) throws ExecutionException,
    InterruptedException {

        Runnable runnable = () -> System.out.println("执行无返回结果的异步任务");
        CompletableFuture.runAsync(runnable);

        CompletableFuture<String> future = CompletableFuture.supplyAsync(() -> {
            System.out.println("执行有返回值的异步任务");
            try {
                Thread.sleep(5000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
            return "Hello world";
        });
        String result = future.join();
        System.out.println(result);

    }
}
```

11.3.2 whenComplete&exceptionally

```
public class CompletableFutureDemo02 {

    public static void main(String[] args) throws ExecutionException,
    InterruptedException {

        CompletableFuture<String> future = CompletableFuture.supplyAsync(() -> {
```

```

        try {
            TimeUnit.SECONDS.sleep(1);
        } catch (InterruptedException e) {
        }
        if (new Random().nextInt(10) % 2 == 0) {
            int i = 12 / 0;
        }
        System.out.println("执行结束! ");
        return "test";
    });

    //正常的，和异常的都会走这里
    future.whenComplete(new BiConsumer<String, Throwable>() {
        @Override
        public void accept(String t, Throwable action) {
            System.out.println(t+" 执行完成! ");
        }
    });

    //出异常走这里
    future.exceptionally(new Function<Throwable, String>() {
        @Override
        public String apply(Throwable t) {
            System.out.println("执行失败: " + t.getMessage());
            return "异常xxxx";
        }
    }).join();
}
}

```

11.3.3 烧水泡茶

- 使用Future实现

```

/**
 *
 * 烧水泡茶程序
 */
public class FutureTaskDemo3 {

    public static void main(String[] args) throws ExecutionException,
    InterruptedException {
        // 创建任务T2的FutureTask
        FutureTask<String> ft2 = new FutureTask<>(new T2Task());
        // 创建任务T1的FutureTask
        FutureTask<String> ft1 = new FutureTask<>(new T1Task(ft2));

        // 线程T1执行任务ft1
        Thread T1 = new Thread(ft1);
        T1.start();
        // 线程T2执行任务ft2
        Thread T2 = new Thread(ft2);
        T2.start();
        // 等待线程T1执行结果
    }
}

```

```

        System.out.println(ft1.get());
    }
}

// T1Task需要执行的任务：
// 洗水壶、烧开水、泡茶
class T1Task implements Callable<String> {
    FutureTask<String> ft2;
    // T1任务需要T2任务的FutureTask
    T1Task(FutureTask<String> ft2){
        this.ft2 = ft2;
    }
    @Override
    public String call() throws Exception {
        System.out.println("T1:洗水壶...");
        TimeUnit.SECONDS.sleep(1);

        System.out.println("T1:烧开水...");
        TimeUnit.SECONDS.sleep(15);
        // 获取T2线程的茶叶
        String tf = ft2.get();
        System.out.println("T1:拿到茶叶:"+tf);

        System.out.println("T1:泡茶...");
        return "上茶:" + tf;
    }
}

// T2Task需要执行的任务：
// 洗茶壶、洗茶杯、拿茶叶
class T2Task implements Callable<String> {
    @Override
    public String call() throws Exception {
        System.out.println("T2:洗茶壶...");
        TimeUnit.SECONDS.sleep(1);

        System.out.println("T2:洗茶杯...");
        TimeUnit.SECONDS.sleep(2);

        System.out.println("T2:拿茶叶...");
        TimeUnit.SECONDS.sleep(1);
        return "龙井";
    }
}

```

- 使用CompletableFuture实现

```

/**
 * 烧水泡茶程序
 */
public class CompletableFutureDemo2 {

    public static void main(String[] args) {

        //任务1: 洗水壶->烧开水
    }
}

```

```

CompletableFuture<Void> f1 = CompletableFuture
    .runAsync(() -> {
        System.out.println("T1:洗水壶...");
        sleep(1, TimeUnit.SECONDS);

        System.out.println("T1:烧开水...");
        sleep(15, TimeUnit.SECONDS);
    });
//任务2: 洗茶壶->洗茶杯->拿茶叶
CompletableFuture<String> f2 = CompletableFuture
    .supplyAsync(() -> {
        System.out.println("T2:洗茶壶...");
        sleep(1, TimeUnit.SECONDS);

        System.out.println("T2:洗茶杯...");
        sleep(2, TimeUnit.SECONDS);

        System.out.println("T2:拿茶叶...");
        sleep(1, TimeUnit.SECONDS);
        return "龙井";
    });
//任务3: 任务1和任务2完成后执行: 泡茶
CompletableFuture<String> f3 = f1.thenCombine(f2, (_, tf) -> {
    System.out.println("T1:拿到茶叶:" + tf);
    System.out.println("T1:泡茶...");
    return "上茶:" + tf;
});
//等待任务3执行结果
System.out.println(f3.join());
}

static void sleep(int t, TimeUnit u){
    try {
        u.sleep(t);
    } catch (InterruptedException e) {
    }
}
}

```

11.3.4 综合案例

```

@Slf4j
public class CompletableFutureDemo {

    public static void main(String[] args) {

        ExecutorService executorService = Executors.newFixedThreadPool(10);

        log.debug("张三 进入餐厅, 点了份西红柿炒番茄");
        CompletableFuture<String> cf = CompletableFuture.supplyAsync(()->{
            log.debug("厨师炒菜");
            sleep(2, TimeUnit.SECONDS);

```

```

        return "西红柿炒番茄好了";
    }, executorService).thenCombine(CompletableFuture.supplyAsync(() -> {
        log.debug("服务员蒸饭");
        sleep(3, TimeUnit.SECONDS);
        return "米饭好了";
    }),(dish, rice) -> { //收到前面2个返回, 进行合并转换
        log.debug("服务员打饭");
        sleep(1, TimeUnit.SECONDS);
        return dish + ", " + rice;
    });

log.debug(" 张三 在刷抖音");
log.debug("{} , 张三 开吃", cf.join()); //等待前面的执行完毕

log.debug("张三 吃完饭去结账, 要求开发票");
cf = CompletableFuture.supplyAsync(() -> {
    log.debug("服务员收款");
    sleep(1, TimeUnit.SECONDS);
    return "20";
}).thenApply(money -> { //获取前一个执行结果 进行转换
    log.debug("服务员开发票, 面额{}元", money);
    sleep(2, TimeUnit.SECONDS);
    return money + "元发票";
});

log.debug("张三 接到朋友电话");
log.debug("张三 拿到{}", 准备回家", cf.join());

log.debug("张三 走出餐厅, 来到公交车站, 等待11路或者 21公交到来");
cf = CompletableFuture.supplyAsync(() -> {
    log.debug("11路公交正在赶来");
    sleep(2, TimeUnit.SECONDS);
    return "11路到了";
}).applyToEither(CompletableFuture.supplyAsync(() -> {
    log.debug("21路公交正在赶来");
    sleep(1, TimeUnit.SECONDS);
    return "21路到了";
}), bus -> { //等待2个线程中任意一个先结束, 进行转换
    if (bus.startsWith("21")) {
        throw new RuntimeException("21 撞树了.....");
    }
    return bus;
}).exceptionally(e -> { //出现一异常走这里
    log.debug(e.getMessage());
    log.debug("张三 叫出租车");
    sleep(3, TimeUnit.SECONDS);
    return "出租车到了";
});

log.debug("{} , 张三 坐车回家", cf.join()); //等待上面的执行结束

```

```

}

```

```

static void sleep(int t, TimeUnit u){
    try {
        u.sleep(t);
    } catch (InterruptedException e) {
    }
}
}

```

12、其他

12.1 死锁

```

@Slf4j
public class DeadLockTest {

    private static String a = "a";
    private static String b = "b";

    public static void main(String[] args) {
        Thread threadA = new Thread()->{
            synchronized (a) {
                log.debug("threadA进入a同步块，执行中...");
                try {
                    Thread.sleep(2000);
                    //a.wait(5000); // 加入这句，注释上面的sleep，可以解决死锁， 条件队列
作用： 打破死锁的循环
                    synchronized (b) {
                        log.debug("threadA进入b同步块，执行中...");
                    }
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }
        }, "threadA");

        Thread threadB = new Thread()->{
            synchronized (b) {
                log.debug("threadB进入b同步块，执行中...");
                try {
                    //b.wait(5000);
                    Thread.sleep(2000);
                    synchronized (a) {
                        log.debug("threadB进入a同步块，执行中...");
                    }
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }
        }, "threadB");
    }
}

```



```

        threadA.start();
        threadB.start();

    }

}

```

12.2 线程饥饿

定长[线程池](#)的使用过程中会存在饥饿现象，也就是当多线程情况下，当池中所有线程都被占用后，被占用的线程又需要空闲线程去进行下一步的操作，此时又获取不到池中空闲的线程，此时就出现了饥饿现象。

```

/**
 * 定长线程池饥饿示例
 */
@Slf4j
public class HungryTest {

    static final List<String> FOODS = Arrays.asList("猪脚饭", "宫保鸡丁", "鱼香肉
    丝", "麻婆豆腐");

    static final Random RANDOM = new Random();

    static ExecutorService pool = Executors.newFixedThreadPool(2);

    //随机做菜
    public static String cooking() {
        return FOODS.get(RANDOM.nextInt(FOODS.size()));
    }

    public static void main(String[] args) throws InterruptedException {
        // 服务员需要点菜、以及自己去做菜
        HungryTest.test();
    }

    public static void test() {
        pool.execute(() -> {
            //服务员开始点菜
            log.info("开始给顾客点菜");
            Future<String> food = pool.submit(() -> {
                log.info("开始做菜");
                return cooking();
            });

            //该服务员点完菜上菜
            try {
                log.info("上菜:{}", food.get());
            }

```

```

        } catch (InterruptedException e) {
            e.printStackTrace();
        } catch (ExecutionException e) {
            e.printStackTrace();
        }
    });

    pool.execute(() -> {
        //服务员开始点菜
        log.info("开始给顾客点菜");
        Future<String> food = pool.submit(() -> {
            log.info("开始做菜");
            return cooking();
        });

        //该服务员点完菜上菜
        try {
            log.info("上菜:{}", food.get());
        } catch (InterruptedException e) {
            e.printStackTrace();
        } catch (ExecutionException e) {
            e.printStackTrace();
        }
    });
}
}

```

12.3 活锁

活锁指的是任务或者执行者没有被阻塞，由于某些条件没有满足，导致一直重复尝试，失败，尝试，失败。

```

/**
 * 活锁
 */
@Slf4j
public class LiveLockTest {

    /**
     * 定义一个勺子，owner 表示这个勺子的拥有者
     */
    static class Spoon {
        Diner owner;

        public Spoon(Diner diner) {
            this.owner = diner;
        }

        public String getOwnerName() {
            return owner.getName();
        }
    }
}

```

```

        public void setOwner(Diner diner) {
            this.owner = diner;
        }

        //表示正在用餐
        public void use() {
            log.info( "{} 用这个勺子吃饭.", owner.getName());
        }
    }

    /**
     * 定义一个晚餐类
     */
    static class Diner {

        private boolean isHungry;
        //用餐者的名字
        private String name;

        public Diner(boolean isHungry, String name) {
            this.isHungry = isHungry;
            this.name = name;
        }

        //和某人吃饭
        public void eatwith(Diner diner, Spoon sharedSpoon) {
            try {
                synchronized (sharedSpoon) {
                    while (isHungry) {
                        //当前用餐者和勺子拥有者不是同一个人，则进行等待
                        while (!sharedSpoon.getOwnerName().equals(name)) {
                            sharedSpoon.wait();
                        }
                    }
                    if (diner.isHungry()) {
                        log.info( "{}: 亲爱的我饿了，然后{}把勺子给了{}",
                                diner.getName(), name, diner.getName());
                        sharedSpoon.setOwner(diner);
                        //用餐
                        sharedSpoon.use();
                        //唤醒等待的线程
                        sharedSpoon.notifyAll();
                    } else {
                        //用餐
                        sharedSpoon.use();
                        sharedSpoon.setOwner(diner);
                        isHungry = false;
                    }
                }
                Thread.sleep(500);
            }
        }
    }

    } catch (InterruptedException e) {
        log.info("{} is interrupted.", name);
    }
}

```

```

        public boolean isHungry() {
            return isHungry;
        }

        public String getName() {
            return name;
        }
    }

    public static void main(String[] args) {
        final Diner husband = new Diner(true, "丈夫");
        final Diner wife = new Diner(true, "妻子");
        //最开始牛郎持有勺子
        final Spoon sharedSpoon = new Spoon(husband);

        //织女和牛郎吃饭
        Thread h = new Thread(()->wife.eatWith(husband, sharedSpoon));
        h.start();

        //牛郎和织女吃饭
        Thread w = new Thread(()->husband.eatWith(wife, sharedSpoon));
        w.start();

        try {
            Thread.sleep(10000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        h.interrupt();
        w.interrupt();
    }
}

```